LaTeX's hook management*

Frank Mittelbach†

October 19, 2026

Contents

1 Introduction **2**

2 Package writer interface **2**
2.1 LaTeX 2ε interfaces . . . 3
2.1.1 Declaring hooks . . . 3
2.1.2 Special declarations for generic hooks . . . 4
2.1.3 Using hooks in code . . . 4
2.1.4 Updating code for hooks . . . 5
2.1.5 Hook names and default labels . . . 8
2.1.6 The `top-level` label . . . 10
2.1.7 Defining relations between hook code . . . 11
2.1.8 Querying hooks . . . 13
2.1.9 Displaying hook code . . . 14
2.1.10 Debugging hook code . . . 15
2.2 L3 programming layer (`expl3`) interfaces . . . 15
2.3 On the order of hook code execution . . . 18
2.4 The use of "reversed" hooks . . . 20
2.5 Difference between "normal" and "one-time" hooks . . . 21
2.6 Generic hooks provided by packages . . . 21
2.7 Hooks with arguments . . . 22
2.8 Private LaTeX kernel hooks . . . 24
2.9 Legacy LaTeX 2ε interfaces . . . 24

3 LaTeX 2ε commands and environments augmented by hooks **25**
3.1 Generic hooks . . . 25
3.1.1 Generic hooks for all environments . . . 26
3.1.2 Generic hooks for commands . . . 27
3.1.3 Generic hooks provided by file loading operations . . . 27
3.2 Hooks provided by `\begin{document}` . . . 28
3.3 Hooks provided by `\end{document}` . . . 28
3.4 Hooks provided by `\shipout` operations . . . 30
3.5 Hooks provided for paragraphs . . . 30
3.6 Hooks provided in NFSS commands . . . 30
3.7 Hook provided by the mark mechanism . . . 31

*This module has version v1.1p dated 2026-06-01, © LaTeX Project.
†Code improvements for speed and other goodies by Phelype Oleinik

4 The Implementation **31**
4.1 Debugging . . . 31
4.2 Borrowing from internals of other kernel modules . . . 32
4.3 Declarations . . . 32
4.4 Providing new hooks . . . 34
4.4.1 The data structures of a hook . . . 34
4.4.2 On the existence of hooks . . . 35
4.4.3 Setting hooks up . . . 36
4.4.4 Disabling and providing hooks . . . 42
4.5 Parsing a label . . . 44
4.6 Adding or removing hook code . . . 48
4.7 Setting rules for hooks code . . . 69
4.8 Specifying code for next invocation . . . 91
4.9 Using the hook . . . 93
4.10 Querying a hook . . . 99
4.11 Messages . . . 103
4.12 LaTeX 2_ε package interface commands . . . 106
4.13 Deprecated that needs cleanup at some point . . . 109
4.14 Internal commands needed elsewhere . . . 111

Index **113**

1 Introduction

Hooks are points in the code of commands or environments where it is possible to add processing code into existing commands. This can be done by different packages that do not know about each other, and to allow for hopefully safe processing it is necessary to sort different chunks of code added by different packages into a suitable processing order.

This is done by the packages adding chunks of code (via `\AddToHook`) and labeling their code with some label by default using the package name as a label.

At `\begin{document}` all code for a hook is then sorted according to some rules (given by `\DeclareHookRule`) for fast execution without processing overhead. If the hook code is modified afterwards (or the rules are changed), a new version for fast processing is generated.

Some hooks are used already in the preamble of the document. If that happens then the hook is prepared for execution (and sorted) already at that point.

2 Package writer interface

The hook management system is offered as a set of CamelCase commands for traditional LaTeX 2_ε packages (and for use in the document preamble if needed) as well as `expl3` commands for modern packages, that use the L3 programming layer of LaTeX. Behind the scenes, a single set of data structures is accessed so that packages from both worlds can coexist and access hooks in other packages.

2.1 LaTeX 2ε interfaces

2.1.1 Declaring hooks

With a few exceptions, hooks have to be declared before they can be used. The exceptions are the generic hooks for commands and environments (executed at `\begin` and `\end`), and the generic hooks run when loading files (see section 3.1).

`\NewHook` `\NewHook {⟨hook⟩}`

Creates a new ⟨*hook*⟩. If this hook is declared within a package it is suggested that its name is always structured as follows: ⟨*package-name*⟩`/`⟨*hook-name*⟩. If necessary you can further subdivide the name by adding more `/` parts. If a hook name is already taken, an error is raised and the hook is not created.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5. The string `??` can't be used as a hook name because it has a special significance as a placeholder in hook rules.

`\NewReversedHook` `\NewReversedHook {⟨hook⟩}`

Like `\NewHook` declares a new ⟨*hook*⟩. the difference is that the code chunks for this hook are in reverse order by default (those added last are executed first). Any rules for the hook are applied after the default ordering. See sections 2.3 and 2.4 for further details.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\NewMirroredHookPair` `\NewMirroredHookPair {⟨hook-1⟩} {⟨hook-2⟩}`

A shorthand for `\NewHook{`⟨*hook-1*⟩`}\NewReversedHook{`⟨*hook-2*⟩`}`.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\NewHookWithArguments` `\NewHookWithArguments {⟨hook⟩} {⟨number⟩}`

Creates a new ⟨*hook*⟩ whose code takes ⟨*number*⟩ arguments, and otherwise works exactly like `\NewHook`. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\NewReversedHookWithArguments` `\NewReversedHookWithArguments {⟨hook⟩} {⟨number⟩}`

Like `\NewReversedHook`, but creates a hook whose code takes ⟨*number*⟩ arguments. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\NewMirroredHookPairWithArguments` `\NewMirroredHookPairWithArguments {⟨hook-1⟩} {⟨hook-2⟩} {⟨number⟩}`

A shorthand for `\NewHookWithArguments{`⟨*hook-1*⟩`}{`⟨*number*⟩`}` `\NewReversedHookWithArguments{`⟨*hook-2*⟩`}{`⟨*number*⟩`}`. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

2.1.2 Special declarations for generic hooks

The declarations here should normally not be used. They are available to provide support for special use cases mainly involving generic command hooks.

`\DisableGenericHook` `\DisableGenericHook {⟨hook⟩}`

After this declaration[1] the ⟨*hook*⟩ is no longer usable: Any further attempt to add code to it will result in an error and any use, e.g., via `\UseHook`, will simply do nothing.

This is intended to be used with generic command hooks (see `ltcmdhooks-doc`) as depending on the definition of the command such generic hooks may be unusable. If that is known, a package developer can disable such hooks up front.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\ActivateGenericHook` `\ActivateGenericHook {⟨hook⟩}`

This declaration activates a generic hook provided by a package/class (e.g., one used in code with `\UseHook` or `\UseOneTimeHook`) without it being explicitly declared with `\NewHook`). If the hook is already activated, this command does nothing.

Note that this command does not undo the effect of `\DisableGenericHook`. See section 2.6 for a discussion of when this declaration is appropriate.

2.1.3 Using hooks in code

Using a hook that is executing the code that has been associated with it is only allowed if the hook has been previously declared with `\NewHook`. For performance reason there are no runtime checks for this and it is the responsibility of the programmer of a package to ensure that all hooks that are used in a package (with one of the commands in this section) are declared first.

`\UseHook` `\UseHook {⟨hook⟩}`

Execute the code stored in the ⟨*hook*⟩.

Before `\begin{document}` the fast execution code for a hook is not set up, so in order to use a hook there it is explicitly initialized first. As that involves assignments using a hook at those times is not 100% the same as using it after `\begin{document}`.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

`\UseHookWithArguments` `\UseHookWithArguments {⟨hook⟩} {⟨number⟩} {⟨arg1⟩} ... {⟨argn⟩}`

Execute the code stored in the ⟨*hook*⟩ and pass the arguments {⟨arg_1⟩} through {⟨arg_n⟩} to the ⟨*hook*⟩. Otherwise, it works exactly like `\UseHook`. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

[1] In the 2020/06 release this command was called `\DisableHook`, but that name was misleading as it shouldn't be used to disable non-generic hooks.

`\UseOneTimeHook` `\UseOneTimeHook {⟨hook⟩}`

Some hooks are only used (and can be only used) in one place, for example, those in `\begin{document}` or `\end{document}`. From that point onwards, adding to the hook through a defined `\⟨addto-cmd⟩` command (e.g., `\AddToHook` or `\AtBeginDocument`, etc.) would have no effect (as would the use of such a command inside the hook code itself). It is therefore customary to redefine `\⟨addto-cmd⟩` to simply process its argument, i.e., essentially make it behave like `\@firstofone`.

`\UseOneTimeHook` does that: it records that the hook has been consumed and any further attempt to add to it will result in executing the code to be added immediately.

Using `\UseOneTimeHook` several times with the same `{⟨hook⟩}` means that it only executes the first time it is used. For example, if it is used in a command that can be called several times then the hook executes during only the *first* invocation of that command; this allows its use as an "initialization hook".

Mixing `\UseHook` and `\UseOneTimeHook` for the same `{⟨hook⟩}` should be avoided, but if this is done then neither will execute after the first `\UseOneTimeHook`.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally. See section 2.1.5 for details.

`\UseOneTimeHookWithArguments` `\UseOneTimeHookWithArguments {⟨hook⟩} {⟨number⟩} {⟨arg1⟩} ... {⟨argn⟩}`

Works exactly like `\UseOneTimeHook`, but passes arguments `{⟨arg1⟩}` through `{⟨argn⟩}` to the ⟨*hook*⟩. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input.

It should be noted that after a one-time hook is used, it is no longer possible to use `\AddToHookWithArguments` or similar with that hook. `\AddToHook` continues to work as normal. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally. See section 2.1.5 for details.

2.1.4 Updating code for hooks

In contrast to the commands from the previous section, declarations such as `\AddToHook` or `\DeclareHookRule` can be used even when the hook is not yet declared. The rationale is that the hook declaration may be in some package that is loaded later, or perhaps not loaded at all.

A side effect of this design is that misspellings do not raise an error but are simply regarded as declarations for hooks with a different name.

`\AddToHook` `\AddToHook {⟨hook⟩} [⟨label⟩] {⟨code⟩}`

Adds ⟨*code*⟩ to the ⟨*hook*⟩ labeled by ⟨*label*⟩. When the optional argument ⟨*label*⟩ is not provided, the ⟨*default label*⟩ is used (see section 2.1.5). If `\AddToHook` is used in a package/class, the ⟨*default label*⟩ is the package/class name, otherwise it is `top-level` (the `top-level` label is treated differently: see section 2.1.6).

If there already exists code under the ⟨*label*⟩ then the new ⟨*code*⟩ is appended to the existing one (even if this is a reversed hook). If you want to replace existing code under the ⟨*label*⟩, first apply `\RemoveFromHook`.

The hook doesn't have to exist for code to be added to it. However, if it is not declared, then obviously the added ⟨*code*⟩ will never be executed. This allows for hooks to work regardless of package loading order and enables packages to add to hooks from other packages without worrying whether they are actually used in the current document. See section 2.1.8.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\AddToHookWithArguments` `\AddToHookWithArguments {⟨hook⟩} [⟨label⟩] {⟨code⟩}`

Works exactly like `\AddToHook`, except that the ⟨*code*⟩ can access the arguments passed to the hook using `#1`, `#2`, ..., `#n` (up to the number of arguments declared for the hook). If the ⟨*code*⟩ should contain *parameter tokens* (`#`) that are not supposed to be understood as the arguments of the hook, such tokens should be doubled. For example, with `\AddToHook` one can write:

```
\AddToHook{myhook}{\def\foo#1{Hello, #1!}}
```

but to achieve the same with `\AddToHookWithArguments`, one should write:

```
\AddToHookWithArguments{myhook}{\def\foo##1{Hello, ##1!}}
```

because in the latter case, `#1` refers to the first argument of the hook `myhook`. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\RemoveFromHook` `\RemoveFromHook {⟨hook⟩} [⟨label⟩]`

Removes any code labeled by ⟨*label*⟩ from the ⟨*hook*⟩. When the optional argument ⟨*label*⟩ is not provided, the ⟨*default label*⟩ is used (see section 2.1.5).

If there is no code under the ⟨*label*⟩ in the ⟨*hook*⟩, or if the ⟨*hook*⟩ does not exist, a warning is issued when you attempt to `\RemoveFromHook`, and the command is ignored. `\RemoveFromHook` should be used only when you know exactly what labels are in a hook. Typically this will be when some code gets added to a hook by a package, then later this code is removed by that same package. If you want to prevent the execution of code from another package, use the `voids` rule instead (see section 2.1.7).

If the optional ⟨*label*⟩ argument is `*`, then all code chunks are removed. This is rather dangerous as it may well drop code from other packages (that one may not know about); it should therefore not be used in packages but only in document preambles!

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

Important: *The* `\RemoveFromHook` *command should be only used if one has full control over the code chunk to be removed. In particular it should not be used to remove code chunks from other packages! For this the* `voids` *relation is provided.*

In contrast to the `voids` relationship between two labels in a `\DeclareHookRule` this is a destructive operation as the labeled code is removed from the hook data structure, whereas the relationship setting can be undone by providing a different relationship later.

A useful application for this declaration inside the document body is when one wants to temporarily add code to hooks and later remove it again, e.g.,

```
\AddToHook{env/quote/begin}{\small}
\begin{quote}
  A quote set in a smaller typeface
\end{quote}
...
\RemoveFromHook{env/quote/begin}
... now back to normal for further quotes
```

Note that you can't cancel the setting with

```
\AddToHook{env/quote/begin}{}
```

because that only "adds" a further empty chunk of code to the hook. Adding `\normalsize` would work but that means the hook then contained `\small\normalsize` which means two font size changes for no good reason.

The above is only needed if one wants to typeset several quotes in a smaller typeface. If the hook is only needed once then `\AddToHookNext` is simpler, because it resets itself after one use.

`\AddToHookNext` `\AddToHookNext {⟨hook⟩} {⟨code⟩}`

Adds ⟨*code*⟩ to the next invocation of the ⟨*hook*⟩. The code is executed after the normal hook code has finished and it is executed only once, i.e. it is deleted after it was used.

Using this declaration is a global operation, i.e., the code is not lost even if the declaration is used inside a group and the next invocation of the hook happens after the end of that group. If the declaration is used several times before the hook is executed then all code is executed in the order in which it was declared.[2]

If this declaration is used with a one-time hook then the code is only ever used if the declaration comes before the hook's invocation. This is because, in contrast to `\AddToHook`, the code in this declaration is not executed immediately in the case when the invocation of the hook has already happened—in other words, this code will truly execute only on the next invocation of the hook (and in the case of a one-time hook there is no such "next invocation"). This gives you a choice: should my code execute always, or should it execute only at the point where the one-time hook is used (and not at all if this is impossible)? For both of these possibilities there are use cases.

It is possible to nest this declaration using the same hook (or different hooks): e.g.,

`\AddToHookNext{`⟨*hook*⟩`}{`⟨*code-1*⟩`\AddToHookNext{`⟨*hook*⟩`}{`⟨*code-2*⟩`}}`

will execute ⟨*code-1*⟩ next time the ⟨*hook*⟩ is used and at that point puts ⟨*code-2*⟩ into the ⟨*hook*⟩ so that it gets executed on following time the hook is run.

A hook doesn't have to exist for code to be added to it. This allows for hooks to work regardless of package loading order. See section 2.1.8.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

[2] There is no mechanism to reorder such code chunks (or delete them).

\AddToHookNextWithArguments `\AddToHookNextWithArguments {⟨hook⟩} {⟨code⟩}`

Works exactly like `\AddToHookNext`, but the ⟨*code*⟩ can contain references to the arguments of the ⟨*hook*⟩ as described for `\AddToHookWithArguments` above. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\ClearHookNext `\ClearHookNext {⟨hook⟩}`

Normally `\AddToHookNext` is only used when you know precisely where it will apply and why you want some extra code at that point. However, there are a few use cases in which such a declaration needs to be canceled, for example, when discarding a page with `\DiscardShipoutBox` (but even then not always), and in such situations `\ClearHookNext` can be used.

2.1.5 Hook names and default labels

It is best practice to use `\AddToHook` in packages or classes *without specifying a* ⟨*label*⟩ because then the package or class name is automatically used, which is helpful if rules are needed, and avoids mistyping the ⟨*label*⟩.

Using an explicit ⟨*label*⟩ is only necessary in very specific situations, e.g., if you want to add several chunks of code into a single hook and have them placed in different parts of the hook (by providing some rules).

The other case is when you develop a larger package with several sub-packages. In that case you may want to use the same ⟨*label*⟩ throughout the sub-packages in order to avoid that the labels change if you internally reorganize your code.

Except for `\UseHook`, `\UseOneTimeHook` and `\IfHookEmptyTF` (and their expl3 interfaces `\hook_use:n`, `\hook_use_once:n` and `\hook_if_empty:nTF`), all ⟨*hook*⟩ and ⟨*label*⟩ arguments are processed in the same way: first, spaces are trimmed around the argument, then it is fully expanded until only character tokens remain. If the full expansion of the ⟨*hook*⟩ or ⟨*label*⟩ contains a non-expandable non-character token, a low-level TEX error is raised (namely, the ⟨*hook*⟩ is expanded using TEX's `\csname`...`\endcsname`, as such, Unicode characters are allowed in ⟨*hook*⟩ and ⟨*label*⟩ arguments). The arguments of `\UseHook`, `\UseOneTimeHook`, and `\IfHookEmptyTF` are processed much in the same way except that spaces are not trimmed around the argument, for better performance.

It is not enforced, but highly recommended that the hooks defined by a package, and the ⟨*labels*⟩ used to add code to other hooks contain the package name to easily identify the source of the code chunk and to prevent clashes. This should be the standard practice, so this hook management code provides a shortcut to refer to the current package in the name of a ⟨*hook*⟩ and in a ⟨*label*⟩. If the ⟨*hook*⟩ name or the ⟨*label*⟩ consist just of a single dot (`.`), or starts with a dot followed by a slash (`./`) then the dot denotes the ⟨*default label*⟩ (usually the current package or class name—see `\SetDefaultHookLabel`). A "`.`" or "`./`" anywhere else in a ⟨*hook*⟩ or in ⟨*label*⟩ is treated literally and is not replaced.

For example, inside the package `mypackage.sty`, the default label is `mypackage`, so the instructions:

```
\NewHook   {./hook}
\AddToHook {./hook}[.]{code}     % Same as \AddToHook{./hook}{code}
```

```
\AddToHook {./hook}[./sub]{code}
\DeclareHookRule{begindocument}{.}{before}{babel}
\AddToHook {file/foo.tex/after}{code}
```

are equivalent to:

```
\NewHook   {mypackage/hook}
\AddToHook {mypackage/hook}[mypackage]{code}
\AddToHook {mypackage/hook}[mypackage/sub]{code}
\DeclareHookRule{begindocument}{mypackage}{before}{babel}
\AddToHook {file/foo.tex/after}{code}                  % unchanged
```

The ⟨*default label*⟩ is automatically set equal to the name of the current package or class at the time the package is loaded. If the hook command is used outside of a package, or the current file wasn't loaded with `\usepackage` or `\documentclass`, then the `top-level` is used as the ⟨*default label*⟩. This may have exceptions—see `\PushDefaultHookLabel`.

This syntax is available in all ⟨*label*⟩ arguments and most ⟨*hook*⟩ arguments, both in the LaTeX 2_ε interface, and the LaTeX3 interface described in section 2.2.

Important:
The dot-syntax is ***not*** *available with* `\UseHook` *and some other commands that are typically used within code!*

Note, however, that the replacement of `.` by the ⟨*default label*⟩ takes place when the hook command is executed, so actions that are somehow executed after the package ends will have the wrong ⟨*default label*⟩ if the dot-syntax is used. For that reason, this syntax is not available in `\UseHook` (and `\hook_use:n`) because the hook is most of the time used outside of the package file in which it was defined. This syntax is also not available in the hook conditionals `\IfHookEmptyTF` (and `\hook_if_empty:nTF`), because these conditionals are used in some performance-critical parts of the hook management code, and because they are usually used to refer to other package's hooks, so the dot-syntax doesn't make much sense.

In some cases, for example in large packages, one may want to separate the code in logical parts, but still use the main package name as the ⟨*label*⟩, then the ⟨*default label*⟩ can be set using `\PushDefaultHookLabel{...}` ... `\PopDefaultHookLabel` or `\SetDefaultHookLabel{...}`.

\PushDefaultHookLabel
\PopDefaultHookLabel

```
\PushDefaultHookLabel {⟨default label⟩}
  ⟨code⟩
\PopDefaultHookLabel
```

`\PushDefaultHookLabel` sets the current ⟨*default label*⟩ to be used in ⟨*label*⟩ arguments, or when replacing a leading "`.`" (see above). `\PopDefaultHookLabel` reverts the ⟨*default label*⟩ to its previous value.

Inside a package or class, the ⟨*default label*⟩ is equal to the package or class name, unless explicitly changed. Everywhere else, the ⟨*default label*⟩ is `top-level` (see section 2.1.6) unless explicitly changed.

The effect of `\PushDefaultHookLabel` holds until the next `\PopDefaultHookLabel`. `\usepackage` (and `\RequirePackage` and `\documentclass`) internally use

```
\PushDefaultHookLabel{⟨package name⟩}
  ⟨package code⟩
\PopDefaultHookLabel
```

to set the ⟨*default label*⟩ for the package or class file. Inside the ⟨*package code*⟩ the ⟨*default label*⟩ can also be changed with `\SetDefaultHookLabel`. `\input` and other file input-related commands from the LaTeX kernel do not use `\PushDefaultHookLabel`, so code within files loaded by these commands does *not* get a dedicated ⟨*label*⟩! (that is, the ⟨*default label*⟩ is the current active one when the file was loaded.)

Packages that provide their own package-like interfaces (Ti*k*Z's `\usetikzlibrary`, for example) can use `\PushDefaultHookLabel` and `\PopDefaultHookLabel` to set dedicated labels and to emulate `\usepackage`-like hook behavior within those contexts.

The `top-level` label is treated differently, and is reserved to the user document, so it is not allowed to change the ⟨*default label*⟩ to `top-level`.

\SetDefaultHookLabel

```
\SetDefaultHookLabel {⟨default label⟩}
```

Similarly to `\PushDefaultHookLabel`, sets the current ⟨*default label*⟩ to be used in ⟨*label*⟩ arguments, or when replacing a leading "`.`". The effect holds until the label is changed again or until the next `\PopDefaultHookLabel`. The difference between `\PushDefaultHookLabel` and `\SetDefaultHookLabel` is that the latter does not save the current ⟨*default label*⟩.

This command is useful when a large package is composed of several smaller packages, but all should have the same ⟨*label*⟩, so `\SetDefaultHookLabel` can be used at the beginning of each package file to set the correct label.

`\SetDefaultHookLabel` is not allowed in the main document, where the ⟨*default label*⟩ is `top-level` and there is no `\PopDefaultHookLabel` to end its effect. It is also not allowed to change the ⟨*default label*⟩ to `top-level`.

2.1.6 The `top-level` label

The `top-level` label, assigned to code added from the main document, is different from other labels. Code added to hooks (usually `\AtBeginDocument`) in the preamble is almost always to change something defined by a package, so it should go at the very end of the hook.

Therefore, code added in the `top-level` is always executed at the end of the hook, regardless of where it was declared. If the hook is reversed (see `\NewReversedHook`), the `top-level` chunk is executed at the very beginning instead.

Rules regarding `top-level` have no effect: if a user wants to have a specific set of rules for a code chunk, they should use a different label to said code chunk, and provide a rule for that label instead.

The `top-level` label is exclusive for the user, so trying to add code with that label from a package results in an error.

2.1.7 Defining relations between hook code

The default assumption is that code added to hooks by different packages are independent and the order in which they are executed is irrelevant. While this is true in many cases it is obviously false in others.

Before the hook management system was introduced packages had to take elaborate precautions to determine whether some other package had also been loaded (before or after) and then to find some ways to alter its behavior accordingly. In addition is was often the user's responsibility to load packages in the right order so that alterations made by packages were done in thsat same order; and in some cases even altering the loading order wouldn't resolve the conflicts.

With the new hook management system it is now possible to define rules (i.e., relationships) between code chunks added by different packages and to specify explicitly the order in which they should be processed.

The rules can be declared for hooks before the hook has been declared with `\NewHook` and they are allowed to refer to code labels that do not yet exist, e.g., because a package defining the code chunk with that label has not yet been loaded. When the hook code is finally sorted for fast execution, all rules that apply are acted on and the others are ignored.

This offers the flexibility needed to handle complicated relationships between code from different packages and to set this up beforehand in a way that is independent of whether or not the packages are actually loaded in a specific document. The downside of this is that misspellings of hook names or code labels will not raise any error, instead the rule will simply never apply!

`\DeclareHookRule` `\DeclareHookRule {⟨hook⟩} {⟨label1⟩} {⟨relation⟩} {⟨label2⟩}`

Defines a relation between ⟨*label1*⟩ and ⟨*label2*⟩ for a given ⟨*hook*⟩. If ⟨*hook*⟩ is `??` this defines a default relation for all hooks that use the two labels, i.e., that have chunks of code labeled with ⟨*label1*⟩ and ⟨*label2*⟩.

Currently, the supported relations are the following:

`before` or `<` Code for ⟨*label1*⟩ comes before code for ⟨*label2*⟩.

`after` or `>` Code for ⟨*label1*⟩ comes after code for ⟨*label2*⟩.

`incompatible-warning` Only code for either ⟨*label1*⟩ or ⟨*label2*⟩ can appear for that hook (a way to say that two packages—or parts of them—are incompatible). A warning is raised if both labels appear in the same hook.

`incompatible-error` Like `incompatible-error` but instead of a warning a LaTeX error is raised, and the code for both labels are dropped from that hook until the conflict is resolved.

`voids` Code for ⟨*label1*⟩ overwrites code for ⟨*label2*⟩. More precisely, code for ⟨*label2*⟩ is dropped for that hook. This can be used, for example if one package is a superset in functionality of another one and therefore wants to undo code in some hook and replace it with its own version.

`unrelated` The order of code for ⟨*label1*⟩ and ⟨*label2*⟩ is irrelevant. This rule is there to undo an incorrect rule specified earlier.

There can only be a single relation between two labels for a given hook, i.e., a later `\DeclareHookRule` overwrites any previous declaration. In all cases rules specific to a given hook take precedence over default rules that use `??` as the ⟨*hook*⟩.

If a default rule is applied, it is done before reversing the label order in a reversed hook, e.g., `before` in a default rule effectively becomes `after` in such a hook. In contrast, a rule for a specific hook is always applied to the state after any reversal (i.e., the state you see when using `\ShowHook` on that hook).

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\ClearHookRule` `\ClearHookRule {⟨hook⟩} {⟨label1⟩} {⟨label2⟩}`

Syntactic sugar for saying that ⟨*label1*⟩ and ⟨*label2*⟩ are unrelated for the given ⟨*hook*⟩.

\DeclareDefaultHookRule

`\DeclareDefaultHookRule {⟨label1⟩} {⟨relation⟩} {⟨label2⟩}`

This sets up a relation between ⟨*label1*⟩ and ⟨*label2*⟩ for all hooks unless overwritten by a specific rule for a hook. Useful for cases where one package has a specific relation to some other package, e.g., is `incompatible` or always needs a special ordering `before` or `after`. (Technically it is just a shorthand for using `\DeclareHookRule` with `??` as the hook name.)

If such a rule is applied to a reversed hook it behaves as if the rule is reversed (e.g., `after` becomes `before`) because those rules are applied first and then the order is reversed. The rationale is that in hook pairs (in which the ordering in one is reversed) default rules have to be reversed too in nearly all scenarios. If this is not the case, a default rule can't be used or has to be overwritten with an explicit `\DeclareHookRule` for that specific hook.

Declaring default rules is only supported in the document preamble.[3]

The ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

2.1.8 Querying hooks

Simpler data types, like token lists, have three possible states; they can:

- exist and be empty;
- exist and be non-empty; and
- not exist (in which case emptiness doesn't apply);

Hooks are a bit more complicated: a hook may exist or not, and independently it may or may not be empty. This means that even a hook that doesn't exist may be non-empty and it can also be disabled.

This seemingly strange state may happen when, for example, package A defines hook `A/foo`, and package B adds some code to that hook. However, a document may load package B before package A, or may not load package A at all. In both cases some code is added to hook `A/foo` without that hook being defined yet, thus that hook is said to be non-empty, whereas it doesn't exist. Therefore, querying the existence of a hook doesn't imply its emptiness, neither does the other way around.

Given that code or rules can be added to a hook even if it doesn't physically exist yet, means that a querying its existence has no real use case (in contrast to other variables that can only be update if they have already been declared). For that reason only the test for emptiness has a public interface.

A hook is said to be empty when no code was added to it, either to its permanent code pool, or to its "next" token list. The hook doesn't need to be declared to have code added to its code pool. A hook is said to exist when it was declared with `\NewHook` or some variant thereof. Generic hooks such as `file` and `env` hooks are automatically declared when code is added to them.

[3] Trying to do so, e.g., via `\DeclareHookRule` with `??` has bad side-effects and is not supported (though not explicitly caught for performance reasons).

\IfHookEmptyTF ⋆ \IfHookEmptyTF {⟨*hook*⟩} {⟨*true code*⟩} {⟨*false code*⟩}
\IfHookEmptyT ⋆
\IfHookEmptyF ⋆

Tests if the ⟨*hook*⟩ is empty (*i.e.*, no code was added to it using either `\AddToHook` or `\AddToHookNext`) or such code was removed again (via `\RemoveFromHook`), and branches to either ⟨*true code*⟩ or ⟨*false code*⟩ depending on the result.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

2.1.9 Displaying hook code

If one has to adjust the code execution in a hook using a hook rule it is helpful to get some information about the code associated with a hook, its current order and the existing rules.

\ShowHook \ShowHook {⟨*hook*⟩}
\LogHook \LogHook {⟨*hook*⟩}

Displays information about the ⟨*hook*⟩ such as

- the code chunks (and their labels) added to it,
- any rules set up to order them,
- the computed order in which the chunks are executed,
- any code executed on the next invocation only.

`\LogHook` prints the information to the `.log` file, and `\ShowHook` prints them to the terminal/command window and starts TEX's prompt (only in `\errorstopmode`) to wait for user action.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

Suppose a hook `example-hook` whose output of `\ShowHook{example-hook}` is:

```
1  -> The hook 'example-hook':
2  > Code chunks:
3  >     foo -> [code from package 'foo']
4  >     bar -> [from package 'bar']
5  >     baz -> [package 'baz' is here]
6  > Document-level (top-level) code (executed last):
7  >     -> [code from 'top-level']
8  > Extra code for next invocation:
9  >     -> [one-time code]
10 > Rules:
11 >     foo|baz with relation >
12 >     baz|bar with default relation <
13 > Execution order (after applying rules):
14 >     baz, foo, bar.
```

In the listing above, lines 3 to 5 show the three code chunks added to the hook and their respective labels in the format

⟨*label*⟩ -> ⟨*code*⟩

Line 7 shows the code chunk added by the user in the main document (labeled `top-level`) in the format

```
Document-level (top-level) code (executed ⟨first|last⟩):
  -> ⟨top-level code⟩
```

This code will be either the first or last code executed by the hook (`last` if the hook is normal, `first` if it is reversed). This chunk is not affected by rules and does not take part in sorting.

Line 9 shows the code chunk for the next execution of the hook in the format

```
-> ⟨next-code⟩
```

This code will be used and disappear at the next `\UseHook{example-hook}`, in contrast to the chunks mentioned earlier, which can only be removed from that hook by doing `\RemoveFromHook{⟨label⟩}[example-hook]`.

Lines 11 and 12 show the rules declared that affect this hook in the format

```
⟨label-1⟩|⟨label-2⟩ with ⟨default?⟩ relation ⟨relation⟩
```

which means that the ⟨*relation*⟩ applies to ⟨*label-1*⟩ and ⟨*label-2*⟩, in that order, as detailed in `\DeclareHookRule`. If the relation is `default` it means that this rule applies to ⟨*label-1*⟩ and ⟨*label-2*⟩ in *all* hooks, (unless overridden by a non-default relation).

Finally, line 14 lists the labels in the hook after sorting; that is, in the order they will be executed when the hook is used.

2.1.10 Debugging hook code

`\DebugHooksOn` `\DebugHooksOff`

`\DebugHooksOn ... \DebugHooksOff`

Turn the debugging of hook code on or off. This displays most changes made to the hook data structures. The output is rather coarse and not really intended for normal use, but it can be helpful in case hooks do not work as expected. See also 2.1.9 for commands to inspect individual hooks.

2.2 L3 programming layer (`expl3`) interfaces

This is a quick summary of the LaTeX3 programming interfaces for use with packages written in `expl3`. In contrast to the LaTeX 2ε interfaces they always use mandatory arguments only, e.g., you always have to specify the ⟨*label*⟩ for a code chunk. We therefore suggest to use the declarations discussed in the previous section even in `expl3` packages, but the choice is yours.

`\hook_new:n` `\hook_new_reversed:n` `\hook_new_pair:nn`

`\hook_new:n {⟨hook⟩}`
`\hook_new_reversed:n {⟨hook⟩}`
`\hook_new_pair:nn {⟨hook-1⟩} {⟨hook-2⟩}`

Creates a new ⟨*hook*⟩ with normal or reverse ordering of code chunks. `\hook_new_pair:nn` creates a pair of such hooks with `{`⟨*hook-2*⟩`}` being a reversed hook. If a hook name is already taken, an error is raised and the hook is not created.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_new_with_args:nn `\hook_new_with_args:nn {⟨hook⟩} {⟨number⟩}`
\hook_new_reversed_with_args:nn `\hook_new_reversed_with_args:nn {⟨hook⟩} {⟨number⟩}`
\hook_new_pair_with_args:nnn `\hook_new_pair_with_args:nnn {⟨hook-1⟩} {⟨hook-2⟩} {⟨number⟩}`

Creates a new ⟨*hook*⟩ with normal or reverse ordering of code chunks, that takes ⟨*number*⟩ arguments from the input stream when it is used. `\hook_new_pair_with_args:nn` creates a pair of such hooks with `{⟨hook-2⟩}` being a reversed hook. If a hook name is already taken, an error is raised and the hook is not created.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_disable_generic:n `\hook_disable_generic:n {⟨hook⟩}`

Marks `{⟨hook⟩}` as disabled. Any further attempt to add code to it or declare it, will result in an error and any call to `\hook_use:n` will simply do nothing.

This declaration is intended for use with generic hooks that are known not to work (see `ltcmdhooks-doc`) if they receive code.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_activate_generic:n `\hook_activate_generic:n {⟨hook⟩}`

This is like `\hook_new:n` but it does nothing if the hook was previously declared with `\hook_new:n`. This declaration should be used only in special situations, e.g., when a command from another package needs to be altered and it is not clear whether a generic `cmd` hook (for that command) has been previously explicitly declared.

Normally `\hook_new:n` should be used instead of this.

\hook_use:n `\hook_use:n {⟨hook⟩}`
\hook_use:nnw `\hook_use:nnw {⟨hook⟩} {⟨number⟩} {⟨arg1⟩} ... {⟨argn⟩}`

Executes the `{⟨hook⟩}` code followed (if set up) by the code for next invocation only, then empties that next invocation code. `\hook_use:nnw` should be used for hooks declared with arguments, and should be followed by as many brace groups as the declared number of arguments. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

\hook_use_once:n `\hook_use_once:n {⟨hook⟩}`
\hook_use_once:nnw `\hook_use_once:nnw {⟨hook⟩} {⟨number⟩} {⟨arg1⟩} ... {⟨argn⟩}`

Changes the `{⟨hook⟩}` status so that from now on any addition to the hook code is executed immediately. Then execute any `{⟨hook⟩}` code already set up. `\hook_use_once:nnw` should be used for hooks declared with arguments, and should be followed by as many brace groups as the declared number of arguments. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

\hook_gput_code:nnn `\hook_gput_code:nnn {⟨hook⟩} {⟨label⟩} {⟨code⟩}`
\hook_gput_code_with_args:nnn `\hook_gput_code_with_args:nnn {⟨hook⟩} {⟨label⟩} {⟨code⟩}`

Adds a chunk of ⟨*code*⟩ to the ⟨*hook*⟩ labeled ⟨*label*⟩. If the label already exists the ⟨*code*⟩ is appended to the already existing code.

If `\hook_gput_code_with_args:nnn` is used, the ⟨*code*⟩ can access the arguments passed to `\hook_use:nnw` (or `\hook_use_once:nnw`) with `#1`, `#2`, ..., `#n` (up to the number of arguments declared for the hook). In that case, if an actual parameter token should be added to the code, it should be doubled.

If code is added to an external ⟨*hook*⟩ (of the kernel or another package) then the convention is to use the package name as the ⟨*label*⟩ not some internal module name or some other arbitrary string.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_gput_next_code:nn `\hook_gput_next_code:nn {⟨hook⟩} {⟨code⟩}`
\hook_gput_next_code_with_args:nn `\hook_gput_next_code_with_args:nn {⟨hook⟩} {⟨code⟩}`

Adds a chunk of ⟨*code*⟩ for use only in the next invocation of the ⟨*hook*⟩. Once used it is gone.

If `\hook_gput_next_code_with_args:nn` is used, the ⟨*code*⟩ can access the arguments passed to `\hook_use:nnw` (or `\hook_use_once:nnw`) with `#1`, `#2`, ..., `#n` (up to the number of arguments declared for the hook). In that case, if an actual parameter token should be added to the code, it should be doubled.

This is simpler than `\hook_gput_code:nnn`, the code is simply appended to the hook in the order of declaration at the very end, i.e., after all standard code for the hook got executed. Thus if one needs to undo what the standard does one has to do that as part of ⟨*code*⟩.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_gclear_next_code:n `\hook_gclear_next_code:n {⟨hook⟩}`

Undo any earlier `\hook_gput_next_code:nn`.

\hook_gremove_code:nn `\hook_gremove_code:nn {⟨hook⟩} {⟨label⟩}`

Removes any code for ⟨*hook*⟩ labeled ⟨*label*⟩.

If there is no code under the ⟨*label*⟩ in the ⟨*hook*⟩, or if the ⟨*hook*⟩ does not exist, a warning is issued when you attempt to use `\hook_gremove_code:nn`, and the command is ignored.

If the second argument is `*`, then all code chunks are removed. This is rather dangerous as it drops code from other packages one may not know about, so think twice before using that!

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_gset_rule:nnnn `\hook_gset_rule:nnnn {⟨hook⟩} {⟨label1⟩} {⟨relation⟩} {⟨label2⟩}`

Relate ⟨*label1*⟩ with ⟨*label2*⟩ when used in ⟨*hook*⟩. See `\DeclareHookRule` for the allowed ⟨*relation*⟩s. If ⟨*hook*⟩ is `??` a default rule is specified.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5. The dot-syntax is parsed in both ⟨*label*⟩ arguments, but it usually makes sense to be used in only one of them.

\hook_if_empty_p:n ⋆
\hook_if_empty:nTF ⋆ `\hook_if_empty:nTF {⟨hook⟩} {⟨true code⟩} {⟨false code⟩}`

Tests if the ⟨*hook*⟩ is empty (*i.e.*, no code was added to it using either `\AddToHook` or `\AddToHookNext`), and branches to either ⟨*true code*⟩ or ⟨*false code*⟩ depending on the result.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

\hook_show:n `\hook_show:n {⟨hook⟩}`
\hook_log:n `\hook_log:n {⟨hook⟩}`

Displays information about the ⟨*hook*⟩ such as

- the code chunks (and their labels) added to it,
- any rules set up to order them,
- the computed order in which the chunks are executed,
- any code executed on the next invocation only.

`\hook_log:n` prints the information to the `.log` file, and `\hook_show:n` prints them to the terminal/command window and starts TEX's prompt (only if `\errorstopmode`) to wait for user action.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_debug_on: `\hook_debug_on:`
\hook_debug_off:

Turns the debugging of hook code on or off. This displays changes to the hook data.

2.3 On the order of hook code execution

Chunks of code for a ⟨*hook*⟩ under different labels are supposed to be independent if there are no special rules set up that define a relation between the chunks. This means that you can't make assumptions about the order of execution!

Suppose you have the following declarations:

```
\NewHook{myhook}
\AddToHook{myhook}[packageA]{\typeout{A}}
\AddToHook{myhook}[packageB]{\typeout{B}}
\AddToHook{myhook}[packageC]{\typeout{C}}
```

then executing the hook with `\UseHook` will produce the typeout `A B C` in that order. In other words, the execution order is computed to be `packageA`, `packageB`, `packageC` which you can verify with `\ShowHook{myhook}`:

```
-> The hook 'myhook':
> Code chunks:
>     packageA -> \typeout {A}
>     packageB -> \typeout {B}
>     packageC -> \typeout {C}
> Document-level (top-level) code (executed last):
>     ---
> Extra code for next invocation:
>     ---
> Rules:
>     ---
> Execution order:
>     packageA, packageB, packageC.
```

The reason is that the code chunks are internally saved in a property list and the initial order of such a property list is the order in which key-value pairs got added. However, that is only true if nothing other than adding happens!

Suppose, for example, you want to replace the code chunk for `packageA`, e.g.,

```
\RemoveFromHook{myhook}[packageA]
\AddToHook{myhook}[packageA]{\typeout{A alt}}
```

then your order becomes `packageB`, `packageC`, `packageA` because the label got removed from the property list and then re-added (at its end).

While that may not be too surprising, the execution order is also sometimes altered if you add a redundant rule, e.g. if you specify

```
\DeclareHookRule{myhook}{packageA}{before}{packageB}
```

instead of the previous lines we get

```
-> The hook 'myhook':
> Code chunks:
>     packageA -> \typeout {A}
>     packageB -> \typeout {B}
>     packageC -> \typeout {C}
> Document-level (top-level) code (executed last):
>     ---
> Extra code for next invocation:
>     ---
> Rules:
>     packageB|packageA with relation >
> Execution order (after applying rules):
>     packageA, packageC, packageB.
```

As you can see the code chunks are still in the same order, but in the execution order for the labels `packageB` and `packageC` have swapped places. The reason is that, with the rule there are two orders that satisfy it, and the algorithm for sorting happened to pick a different one compared to the case without rules (where it doesn't run at all as there is nothing to resolve). Incidentally, if we had instead specified the redundant rule

```
\DeclareHookRule{myhook}{packageB}{before}{packageC}
```

the execution order would not have changed.

In summary: it is not possible to rely on the order of execution unless there are rules that partially or fully define the order (in which you can rely on them being fulfilled).

2.4 The use of "reversed" hooks

You may have wondered why you can declare a "reversed" hook with `\NewReversedHook` and what that does exactly.

In short: the execution order of a reversed hook (without any rules!) is exactly reversed to the order you would have gotten for a hook declared with `\NewHook`.

This is helpful if you have a pair of hooks where you expect to see code added that involves grouping, e.g., starting an environment in the first and closing that environment in the second hook. To give a somewhat contrived example[4], suppose there is a package adding the following:

```
\AddToHook{env/quote/before}[package-1]{\begin{itshape}}
\AddToHook{env/quote/after} [package-1]{\end{itshape}}
```

As a result, all quotes will be in italics. Now suppose further that another `package-too` makes the quotes also in blue and therefore adds:

```
\usepackage{color}
\AddToHook{env/quote/before}[package-too]{\begin{color}{blue}}
\AddToHook{env/quote/after} [package-too]{\end{color}}
```

Now if the `env/quote/after` hook would be a normal hook we would get the same execution order in both hooks, namely:

```
package-1, package-too
```

(or vice versa) and as a result, would get:

```
\begin{itshape}\begin{color}{blue} ...
\end{itshape}\end{color}
```

and an error message saying that `\begin{color}` was ended by `\end{itshape}`. With `env/quote/after` declared as a reversed hook the execution order is reversed and so all environments are closed in the correct sequence and `\ShowHook` would give us the following output:

```
-> The hook 'env/quote/after':
> Code chunks:
>     package-1 -> \end {itshape}
>     package-too -> \end {color}
> Document-level (top-level) code (executed first):
>     ---
> Extra code for next invocation:
>     ---
> Rules:
>     ---
> Execution order (after reversal):
>     package-too, package-1.
```

If there is a matching default rule (done with `\DeclareDefaultHookRule` or with `??` for the hook name) then this default rule is applied before the reversal so that the order in the reversed hook mirrors the one in the normal hook. However, all rules specific to a hook happen always after the reversal of the execution order, so if you alter the order you will probably have to alter it in both hooks, not just in one, but that depends on the use case.

[4] There are simpler ways to achieve the same effect.

2.5 Difference between “normal” and “one-time” hooks

When executing a hook a developer has the choice of using either `\UseHook` or `\UseOneTimeHook` (or their `expl3` equivalents `\hook_use:n` and `\hook_use_once:n`). This choice affects how `\AddToHook` is handled after the hook has been executed for the first time.

With normal hooks adding code via `\AddToHook` means that the code chunk is added to the hook data structure and then used each time `\UseHook` is called.

With one-time hooks it this is handled slightly differently: After `\UseOneTimeHook` has been called, any further attempts to add code to the hook via `\AddToHook` will simply execute the ⟨*code*⟩ immediately.

This has some consequences one needs to be aware of:

- If ⟨*code*⟩ is added to a normal hook after the hook was executed and it is never executed again for one or the other reason, then this new ⟨*code*⟩ will never be executed.
- In contrast if that happens with a one-time hook the ⟨*code*⟩ is executed immediately.

In particular this means that construct such as

```
\AddToHook{myhook}
          { ⟨code-1⟩ \AddToHook{myhook}{⟨code-2⟩} ⟨code-3⟩ }
```

works for one-time hooks[5] (all three code chunks are executed one after another), but it makes little sense with a normal hook, because with a normal hook the first time `\UseHook{myhook}` is executed it would

- execute ⟨*code-1*⟩,
- then execute `\AddToHook{myhook}{code-2}` which adds the code chunk ⟨*code-2*⟩ to the hook for use on the next invocation,
- and finally execute ⟨*code-3*⟩.

The second time `\UseHook` is called it would execute the above and in addition ⟨*code-2*⟩ as that was added as a code chunk to the hook in the meantime. So each time the hook is used another copy of ⟨*code-2*⟩ is added and so that code chunk is executed ⟨*# of invocations*⟩ − 1 times.

2.6 Generic hooks provided by packages

The hook management system also implements a category of hooks that are called “Generic Hooks”. Normally a hook has to be explicitly declared before it can be used in code. This ensures that different packages are not using the same hook name for unrelated purposes—something that would result in absolute chaos. However, there are a number of “standard” hooks where it is unreasonable to declare them beforehand, e.g, each and every command has (in theory) an associated `before` and `after` hook. In such cases, i.e., for command, environment or file hooks, they can be used simply by adding code to them with `\AddToHook`. For more specialized generic hooks, e.g., those provided

[5] This is sometimes used with `\AtBeginDocument` which is why it is supported.

by babel, you have to additionally enable them with `\ActivateGenericHook` as explained below.

The generic hooks provided by LaTeX are those for `cmd`, `env`, `file`, `include`, `package`, and `class`, and all these are available out of the box: you only have to use `\AddToHook` to add code to them, but you don't have to add `\UseHook` or `\UseOneTimeHook` to your code, because this is already done for you (or, in the case of `cmd` hooks, the command's code is patched at `\begin{document}`, if necessary).

However, if you want to provide further generic hooks in your own code, the situation is slightly different. To do this you should use `\UseHook` or `\UseOneTimeHook`, but *without declaring the hook* with `\NewHook`. As mentioned earlier, a call to `\UseHook` with an undeclared hook name does nothing. So as an additional setup step, you need to explicitly activate your generic hook. Note that a generic hook produced in this way is always a normal hook.

For a truly generic hook, with a variable part in the hook name, such upfront activation would be difficult or impossible, because you typically do not know what kind of variable parts may come up in real documents.

For example, babel provides hooks such as `babel/`⟨*language*⟩`/afterextras`. However, language support in babel is often done through external language packages. Thus doing the activation for all languages inside the core babel code is not a viable approach. Instead it needs to be done by each language package (or by the user who wants to use a particular hook).

Because the hooks are not declared with `\NewHook` their names should be carefully chosen to ensure that they are (likely to be) unique. Best practice is to include the package or command name, as was done in the babel example above.

Generic hooks defined in this way are always normal hooks (i.e., you can't implement reversed hooks this way). This is a deliberate limitation, because it speeds up the processing considerably.

2.7 Hooks with arguments

Sometimes it is necessary to pass contextual information to a hook, and, for one reason or another, it is not feasible to store such information in macros. To serve this purpose, hooks can be declared with arguments, so that the programmer can pass along the data necessary for the code in the hook to function properly.

A hook with arguments works mostly like a regular hook, and most commands that work for regular hooks, also work for hooks that take arguments. The differences are when the hook is declared (`\NewHookWithArguments` is used instead of `\NewHook`), then code can be added with both `\AddToHook` and `\AddToHookWithArguments`, and when the hook is used (`\UseHookWithArguments` instead of `\UseHook`).

A hook with arguments must be declared as such (before it is first used, as all regular hooks) using `\NewHookWithArguments{`⟨*hook*⟩`}{`⟨*number*⟩`}`. All code added to that hook can then use `#1` to access the first argument, `#2` to access the second, and so forth up to the number of arguments declared. However, it is still possible to add code with references to the arguments of a hook that was not yet declared (we will discuss that later). At their core, hooks are macros, so TeX's limit of 9 arguments applies, and a low-level TeX error is raised if you try to reference an argument number that doesn't exist.

To use a hook with arguments, just write `\UseHookWithArguments{`⟨*hook*⟩`}{`⟨*number*⟩`}` followed by a braced list of the arguments. For example, if the hook `test` takes three arguments, write:

```
\UseHookWithArguments{test}{3}{arg-1}{arg-2}{arg-3}
```

then, in the ⟨*code*⟩ of the hook, all instances of `#1` will be replaced by `arg-1`, `#2` by `arg-2` and so on. If, at the point of usage, the programmer provides more arguments than the hook is declared to take, the excess arguments are simply ignored by the hook. Behavior is unpredictable[6] if too few arguments are provided. If the hook isn't declared, ⟨*number*⟩ arguments are removed from the input stream.

Adding code to a hook with arguments can be done with `\AddToHookWithArguments` as well as with the regular `\AddToHook`, to achieve different outcomes. The main difference when it comes to adding code to a hook, in this case, is firstly the possibility of accessing a hook's arguments, of course, and second, how parameter tokens (`#`$_6$) are treated.

Using `\AddToHook` in a hook that takes arguments will work as it does for all other hooks. This allows a package developer to add arguments to a hook that otherwise had none without having to worry about compatibility. This means that, for example:

```
\AddToHook{test}{\def\foo#1{Hello, #1!}}
```

will define the same macro `\foo` regardless if the hook `test` takes arguments or not.

Using `\AddToHookWithArguments` allows the ⟨*code*⟩ added to access the arguments of the hook with `#1`, `#2`, and so forth, up to the number of the arguments declared in the hook. This means that if one wants to add a `#`$_6$ to the ⟨*code*⟩ that token must be doubled in the input. The same definition from above, using `\AddToHookWithArguments`, needs to be rewritten:

```
\AddToHookWithArguments{test}{\def\foo##1{Hello, ##1!}}
```

Extending the above example to use the hook arguments, we could rewrite something like (now from declaration to usage, to get the whole picture):

```
\NewHookWithArguments{test}{1}
\AddToHookWithArguments{test}{%
  \typeout{Defining foo with "#1"}
  \def\foo##1{Hello, ##1! Some text after: #1}%
}
\UseHook{test}{Howdy!}
\ShowCommand\foo
```

Running the code above prints in the terminal:

```
Defining foo with "Howdy!"
> \foo=macro:
#1->Hello, #1! Some text after: Howdy!.
```

[6] The hook *will* take the declared number of arguments, and what will happen depends on what was grabbed, and what the hook code does with its arguments.

Note how `##1` in the call to `\AddToHookWithArguments` became `#1`, and the `#1` was replaced by the argument passed to the hook. Should the hook be used again, with a different argument, the definition would naturally change.

It is possible to add code referencing a hook's arguments before such hook is declared and the number of hooks is fixed. However, if some code is added to the hook, that references more arguments than will be declared for the hook, there will be a low-level TEX error about an "Illegal parameter number" at the time the hook is declared, which will be hard to track down because at that point TEX can't know whence the offending code came from. Thus it is important that package writers explicitly document how many arguments (if any) each hook can take, so users of those packages know how many arguments can be referenced, and equally important, what each argument means.

2.8 Private LATEX kernel hooks

There are a few places where it is absolutely essential for LATEX to function correctly that code is executed in a precisely defined order. Even that could have been implemented with the hook management (by adding various rules to ensure the appropriate ordering with respect to other code added by packages). However, this makes every document unnecessary slow, because there has to be sorting even though the result is predetermined. Furthermore it forces package writers to unnecessarily add such rules if they add further code to the hook (or break LATEX).

For that reason such code is not using the hook management, but instead private kernel commands directly before or after a public hook with the following naming convention: `\@kernel@before@`⟨*hook*⟩ or `\@kernel@after@`⟨*hook*⟩. For example, in `\enddocument` you find

```
\UseHook{enddocument}%
\@kernel@after@enddocument
```

which means first the user/package-accessible `enddocument` hook is executed and then the internal kernel hook. As their name indicates these kernel commands should not be altered by third-party packages, so please refrain from that in the interest of stability and instead use the public hook next to it.[7]

2.9 Legacy LATEX 2ε interfaces

LATEX 2ε offered a small number of hooks together with commands to add to them. They are listed here and are retained for backwards compatibility.

With the new hook management, several additional hooks have been added to LATEX and more will follow. See the next section for what is already available.

[7] As with everything in TEX there is not enforcement of this rule, and by looking at the code it is easy to find out how the kernel adds to them. The main reason of this section is therefore to say "please don't do that, this is unconfigurable code!"

\AtBeginDocument `\AtBeginDocument [⟨label⟩] {⟨code⟩}`

If used without the optional argument ⟨*label*⟩, it works essentially like before, i.e., it is adding ⟨*code*⟩ to the hook `begindocument` (which is executed inside `\begin{document}`). However, all code added this way is labeled with the label `top-level` (see section 2.1.6) if done outside of a package or class or with the package/class name if called inside such a file (see section 2.1.5).

This way one can add code to the hook using `\AddToHook` or `\AtBeginDocument` using a different label and explicitly order the code chunks as necessary, e.g., run some code before or after another package's code. When using the optional argument the call is equivalent to running `\AddToHook {begindocument} [`⟨*label*⟩`] {`⟨*code*⟩`}`.

`\AtBeginDocument` is a wrapper around the `begindocument` hook (see section 3.2), which is a one-time hook. As such, after the `begindocument` hook is executed at `\begin{document}` any attempt to add ⟨*code*⟩ to this hook with `\AtBeginDocument` or with `\AddToHook` will cause that ⟨*code*⟩ to execute immediately instead. See section 2.5 for more on one-time hooks.

For important packages with known order requirement we may over time add rules to the kernel (or to those packages) so that they work regardless of the loading-order in the document.

\AtEndDocument `\AtEndDocument [⟨label⟩] {⟨code⟩}`

Like `\AtBeginDocument` but for the `enddocument` hook.

The few hooks that existed previously in LaTeX 2$_\varepsilon$ used internally commands such as `\@begindocumenthook` and packages sometimes augmented them directly rather than working through `\AtBeginDocument`. For that reason there is currently support for this, that is, if the system detects that such an internal legacy hook command contains code it adds it to the new hook system under the label `legacy` so that it doesn't get lost.

However, over time the remaining cases of direct usage need updating because in one of the future release of LaTeX we will turn this legacy support off, as it does unnecessary slow down the processing.

3 LaTeX 2$_\varepsilon$ commands and environments augmented by hooks

In this section we describe the standard hooks that are now offered by LaTeX, or give pointers to other documents in which they are described. This section will grow over time (and perhaps eventually move to usrguide3).

3.1 Generic hooks

As stated earlier, with the exception of generic hooks, all hooks must be declared with `\NewHook` before they can be used. All generic hooks have names of the form "⟨*type*⟩/⟨*name*⟩/⟨*position*⟩", where ⟨*type*⟩ is from the predefined list shown below, and ⟨*name*⟩ is the variable part whose meaning will depend on the ⟨*type*⟩. The last component, ⟨*position*⟩, has more complex possibilities: it can always be `before` or `after`; for `env` hooks, it can also be `begin` or `end`; and for `include` hooks it can also be `end`. Each specific hook is documented below, or in `ltcmdhooks-doc.pdf` or `ltfilehook-doc.pdf`.

The generic hooks provided by LaTeX belong to one of the six types:

env Hooks executed before and after environments – ⟨*name*⟩ is the name of the environment, and available values for ⟨*position*⟩ are `before`, `begin`, `end`, and `after`;

cmd Hooks added to and executed before and after commands – ⟨*name*⟩ is the name of the command, and available values for ⟨*position*⟩ are `before` and `after`;

file Hooks executed before and after reading a file – ⟨*name*⟩ is the name of the file (with extension), and available values for ⟨*position*⟩ are `before` and `after`;

package Hooks executed before and after loading packages – ⟨*name*⟩ is the name of the package, and available values for ⟨*position*⟩ are `before` and `after`;

class Hooks executed before and after loading classes – ⟨*name*⟩ is the name of the class, and available values for ⟨*position*⟩ are `before` and `after`;

include Hooks executed before and after `\included` files – ⟨*name*⟩ is the name of the included file (without the `.tex` extension), and available values for ⟨*position*⟩ are `before`, `end`, and `after`.

Each of the hooks above are detailed in the following sections and in linked documentation.

3.1.1 Generic hooks for all environments

Every environment ⟨*env*⟩ has now four associated hooks coming with it:

`env/`⟨*env*⟩`/before` This hook is executed as part of `\begin` as the very first action, in particular prior to starting the environment group. Its scope is therefore not restricted by the environment.

`env/`⟨*env*⟩`/begin` This hook is executed as part of `\begin` directly in front of the code specific to the environment start (e.g., the third argument of `\NewDocumentEnvironment` and the second argument of `\newenvironment`). Its scope is the environment body.

`env/`⟨*env*⟩`/end` This hook is executed as part of `\end` directly in front of the code specific to the end of the environment (e.g., the forth argument of `\NewDocumentEnvironment` and the third argument of `\newenvironment`).

`env/`⟨*env*⟩`/after` This hook is executed as part of `\end` after the code specific to the environment end and after the environment group has ended. Its scope is therefore not restricted by the environment.

The hook is implemented as a reversed hook so if two packages add code to `env/`⟨*env*⟩`/before` and to `env/`⟨*env*⟩`/after` they can add surrounding environments and the order of closing them happens in the right sequence.

Given that these generic hook names involve `/` as part of their name they would not work if one tries to define an environment using a name that involves a `/`.[8]

Generic environment hooks are never one-time hooks even with environments that are supposed to appear only once in a document.[9] In contrast to other hooks there is also no need to declare them using `\NewHook`.

[8] Officially, LaTeX names for environments should only consist of a sequence of letters, numbers, and the character `*`, i.e., this is not a new restriction.

[9] Thus if one adds code to such hooks after the environment has been processed, it will only be executed if the environment appears again and if that doesn't happen the code will never get executed.

The hooks are only executed if `\begin{`⟨*env*⟩`}` and `\end{`⟨*env*⟩`}` is used. If the environment code is executed via low-level calls to `\`⟨*env*⟩ and `\end`⟨*env*⟩ (e.g., to avoid the environment grouping) they are not available. If you want them available in code using this method, you would need to add them yourself, i.e., write something like

```
\UseHook{env/quote/before}\quote
    ...
\endquote\UseHook{env/quote/after}
```

to add the outer hooks, etc.

Largely for compatibility with existing packages, the following four commands are also available to set the environment hooks; but for new packages we recommend directly using the hook names and `\AddToHook`.

`\BeforeBeginEnvironment` `\BeforeBeginEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This declaration adds to the `env/`⟨*env*⟩`/before` hook using the ⟨*label*⟩. If ⟨*label*⟩ is not given, the ⟨*default label*⟩ is used (see section 2.1.5).

`\AtBeginEnvironment` `\AtBeginEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This is like `\BeforeBeginEnvironment` but it adds to the `env/`⟨*env*⟩`/begin` hook.

`\AtEndEnvironment` `\AtEndEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This is like `\BeforeBeginEnvironment` but it adds to the `env/`⟨*env*⟩`/end` hook.

`\AfterEndEnvironment` `\AfterEndEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This is like `\BeforeBeginEnvironment` but it adds to the `env/`⟨*env*⟩`/after` hook.

3.1.2 Generic hooks for commands

Similar to environments there are now (at least in theory) two generic hooks available for any LaTeX command. These are

`cmd/`⟨*name*⟩`/before` This hook is executed at the very start of the command execution.

`cmd/`⟨*name*⟩`/after` This hook is executed at the very end of the command body. It is implemented as a reversed hook.

In practice there are restrictions and especially the `after` hook works only with a subset of commands. Details about these restrictions are documented in `ltcmdhooks-doc.pdf` or with code in `ltcmdhooks-code.pdf`.

3.1.3 Generic hooks provided by file loading operations

There are several hooks added to LaTeX's process of loading file via its high-level interfaces such as `\input`, `\include`, `\usepackage`, `\RequirePackage`, etc. These are documented in `ltfilehook-doc.pdf` or with code in `ltfilehook-code.pdf`.

3.2 Hooks provided by `\begin{document}`

Until 2020 `\begin{document}` offered exactly one hook that one could add to using `\AtBeginDocument`. Experiences over the years have shown that this single hook in one place was not enough and as part of adding the general hook management system a number of additional hooks have been added at this point. The places for these hooks have been chosen to provide the same support as offered by external packages, such as etoolbox and others that augmented `\document` to gain better control.

Supported are now the following hooks (all of them one-time hooks):

`begindocument/before` This hook is executed at the very start of `\document`, one can think of it as a hook for code at the end of the preamble section and this is how it is used by etoolbox's `\AtEndPreamble`.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

`begindocument` This hook is added to by using `\AddToHook{begindocument}` or by using `\AtBeginDocument` and it is executed after the `.aux` file has been read and most initialization are done, so they can be altered and inspected by the hook code. It is followed by a small number of further initializations that shouldn't be altered and are therefore coming later.

The hook should not be used to add material for typesetting as we are still in LaTeX's initialization phase and not in the document body. If such material needs to be added to the document body use the next hook instead.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

`begindocument/end` This hook is executed at the end of the `\document` code in other words at the beginning of the document body. The only command that follows it is `\ignorespaces`.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

The generic hooks executed by `\begin` also exist, i.e., `env/document/before` and `env/document/begin`, but with this special environment it is better use the dedicated one-time hooks above.

3.3 Hooks provided by `\end{document}`

LaTeX 2$_\varepsilon$ has always provided `\AtEndDocument` to add code to the `\end{document}`, just in front of the code that is normally executed there. While this was a big improvement over the situation in LaTeX 2.09, it was not flexible enough for a number of use cases and so packages, such as etoolbox, atveryend and others patched `\enddocument` to add additional points where code could be hooked into.

Patching using packages is always problematical as leads to conflicts (code availability, ordering of patches, incompatible patches, etc.). For this reason a number of additional hooks have been added to the `\enddocument` code to allow packages to add code in various places in a controlled way without the need for overwriting or patching the core code.

Supported are now the following hooks (all of them one-time hooks):

enddocument The hook associated with `\AtEndDocument`. It is immediately called at the beginning of `\enddocument`.

When this hook is executed there may be still unprocessed material (e.g., floats on the deferlist) and the hook may add further material to be typeset. After it, `\clearpage` is called to ensure that all such material gets typeset. If there is nothing waiting the `\clearpage` has no effect.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

enddocument/afterlastpage As the name indicates this hook should not receive code that generates material for further pages. It is the right place to do some final housekeeping and possibly write out some information to the `.aux` file (which is still open at this point to receive data). Just before this hook `\write` is redefined and is now always `\immediate\write`. This means that writing e.g. a label or something to the `.toc` works despite the fact that no more pages are output. It is also the correct place to set up any testing code to be run when the `.aux` file is re-read in the next step.

After this hook has been executed the `.aux` file is closed for writing and then read back in to do some tests (e.g., looking for missing references or duplicated labels, etc.).

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

enddocument/afteraux At this point, the `.aux` file has been reprocessed and so this is a possible place for final checks and display of information to the user. However, for the latter you might prefer the next hook, so that your information is displayed after the (possibly longish) list of files if that got requested via `\listfiles`.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

enddocument/info This hook is meant to receive code that write final information messages to the terminal. It follows immediately after the previous hook (so both could have been combined, but then packages adding further code would always need to also supply an explicit rule to specify where it should go.

This hook already contains some code added by the kernel (under the labels `kernel/filelist` and `kernel/warnings`), namely the list of files when `\listfiles` has been used and the warnings for duplicate labels, missing references, font substitutions etc.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

enddocument/end Finally, this hook is executed just in front of the final call to `\@@end`.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).is it even possible to add code after this one?

There is also the hook `shipout/lastpage`. This hook is executed as part of the last `\shipout` in the document to allow package to add final `\special`'s to that page. Where this hook is executed in relation to those from the above list can vary from document to

document. Furthermore to determine correctly which of the `\shipout`s is the last one, LaTeX needs to be run several times, so initially it might get executed on the wrong page. See section 3.4 for where to find the details.

It is in also possible to use the generic `env/document/end` hook which is executed by `\end`, i.e., just in front of the first hook above. Note however that the other generic `\end` environment hook, i.e., `env/document/after` will never get executed, because by that time LaTeX has finished the document processing.

3.4 Hooks provided by `\shipout` operations

There are several hooks and mechanisms added to LaTeX's process of generating pages. These are documented in `ltshipout-doc.pdf` or with code in `ltshipout-code.pdf`.

3.5 Hooks provided for paragraphs

The paragraph processing has been augmented to include a number of internal and public hooks. These are documented in `ltpara-doc.pdf` or with code in `ltpara-code.pdf`.

3.6 Hooks provided in NFSS commands

In languages that need to support for more than one script in parallel (and thus several sets of fonts, e.g., supporting both Latin and Japanese fonts), NFSS font commands such as `\sffamily` need to switch both the Latin family to "Sans Serif" and in addition alter a second set of fonts.

To support this, several NFSS commands have hooks to which such support can be added.

`rmfamily` After `\rmfamily` has done its initial checks and prepared a font series update, this hook is executed before `\selectfont`.

`sffamily` This is like the `rmfamily` hook, but for the `\sffamily` command.

`ttfamily` This is like the `rmfamily` hook, but for the `\ttfamily` command.

`normalfont` The `\normalfont` command resets the font encoding, family, series and shape to their document defaults. It then executes this hook and finally calls `\selectfont`.

`expand@font@defaults` The internal `\expand@font@defaults` command expands and saves the current defaults for the metafamilies (rm/sf/tt) and the metaseries (bf/md). If the NFSS machinery has been augmented, e.g., for Chinese or Japanese fonts, then further defaults may need to be set at this point. This can be done in this hook which is executed at the end of this macro.

`bfseries/defaults`, `bfseries` If the `\bfdefault` was explicitly changed by the user, its new value is used to set the bf series defaults for the metafamilies (rm/sf/tt) when `\bfseries` is called. The `bfseries/defaults` hook allows further adjustments to be made in this case. This hook is only executed if such a change is detected. In contrast, the `bfseries` hook is always executed just before `\selectfont` is called to change to the new series.

`mdseries/defaults`, `mdseries` These two hooks are like the previous ones but they are in the `\mdseries` command.

selectfont This hook is executed inside `\selectfont`, after the current values for *encoding*, *family*, *series*, *shape*, and *size* are evaluated and the new font is selected (and if necessary loaded). After the hook has executed, NFSS will still do any updates necessary for a new *size* (such as changing the size of `\strut`) and any updates necessary to a change in *encoding*.

This hook is intended for use cases where, in parallel to a change in the main font, some other fonts need to be altered (e.g., in CJK processing where you may need to deal with several different alphabets).

3.7 Hook provided by the mark mechanism

See `ltmarks-doc.pdf` for details.

insertmark This hook allows for a special setup while `\InsertMark` inserts a mark. It is executed in group so local changes only apply to the mark being inserted.

4 The Implementation

```
⟨@@=hook⟩
⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
⟨latexrelease⟩\NewModuleRelease{2020/10/01}{lthooks}
⟨latexrelease⟩                    {The~hook~management~system}
```

4.1 Debugging

\g__hook_debug_bool

Holds the current debugging state.

```
\bool_new:N \g__hook_debug_bool
```

(*End of definition for* `\g__hook_debug_bool`.)

\hook_debug_on:
\hook_debug_off:
__hook_debug:n
__hook_debug_gset:

Turns debugging on and off by redefining `\__hook_debug:n`.

```
\cs_new_eq:NN \__hook_debug:n \use_none:n
\cs_new_protected:Npn \hook_debug_on:
  {
    \bool_gset_true:N \g__hook_debug_bool
    \__hook_debug_gset:
  }
\cs_new_protected:Npn \hook_debug_off:
  {
    \bool_gset_false:N \g__hook_debug_bool
    \__hook_debug_gset:
  }
\cs_new_protected:Npn \__hook_debug_gset:
  {
    \cs_gset_protected:Npe \__hook_debug:n ##1
      { \bool_if:NT \g__hook_debug_bool {##1} }
  }
```

(*End of definition for* `\hook_debug_on:` *and others. These functions are documented on page* 18.)

4.2 Borrowing from internals of other kernel modules

__hook_str_compare:nn Private copy of __str_if_eq:nn

```
\cs_new_eq:NN \__hook_str_compare:nn \__str_if_eq:nn
```

(*End of definition for* __hook_str_compare:nn.)

4.3 Declarations

\l__hook_tmpa_bool Scratch boolean used throughout the package.

```
\bool_new:N \l__hook_tmpa_bool
```

(*End of definition for* \l__hook_tmpa_bool.)

\l__hook_return_tl
\l__hook_tmpa_tl
\l__hook_tmpb_tl
Scratch variables used throughout the package.

```
\tl_new:N \l__hook_return_tl
\tl_new:N \l__hook_tmpa_tl
\tl_new:N \l__hook_tmpb_tl
```

(*End of definition for* \l__hook_return_tl, \l__hook_tmpa_tl, *and* \l__hook_tmpb_tl.)

\g__hook_all_seq In a few places we need a list of all hook names ever defined so we keep track if them in this sequence.

```
\seq_new:N \g__hook_all_seq
```

(*End of definition for* \g__hook_all_seq.)

\l__hook_cur_hook_tl Stores the name of the hook currently being sorted.

```
\tl_new:N \l__hook_cur_hook_tl
```

(*End of definition for* \l__hook_cur_hook_tl.)

\l__hook_work_prop A property list holding a copy of the \g__hook_⟨*hook*⟩_code_prop of the hook being sorted to work on, so that changes don't act destructively on the hook data structure.

```
\prop_new:N \l__hook_work_prop
```

(*End of definition for* \l__hook_work_prop.)

\g__hook_used_prop All hooks that receive code (for use in debugging display).

```
\prop_new:N \g__hook_used_prop
```

(*End of definition for* \g__hook_used_prop.)

\g__hook_hook_curr_name_tl
\g__hook_name_stack_seq
Default label used for hook commands, and a stack to keep track of packages within packages.

```
\tl_new:N \g__hook_hook_curr_name_tl
\seq_new:N \g__hook_name_stack_seq
```

(*End of definition for* \g__hook_hook_curr_name_tl *and* \g__hook_name_stack_seq.)

__hook_tmp:w Temporary macro for generic usage.

```
\cs_new_eq:NN \__hook_tmp:w ?
```

(*End of definition for* __hook_tmp:w.)

\c__hook_empty_tl
\c__hook_nine_parameters_tl

An empty token list, and one containing nine parameters.

```
\tl_const:Nn \c__hook_empty_tl { }
\tl_const:Nn \c__hook_nine_parameters_tl { #1#2#3#4#5#6#7#8#9 }
```

(*End of definition for* `\c__hook_empty_tl` *and* `\c__hook_nine_parameters_tl`.)

\str_count:e

Some variants of expl3 functions.

FMi: should probably be moved to expl3

```
\cs_generate_variant:Nn \str_count:n { e }
```

(*End of definition for* `\str_count:e`.)

\s__hook_mark

Scan mark used for delimited arguments.

```
\scan_new:N \s__hook_mark
```

(*End of definition for* `\s__hook_mark`.)

__hook_use_none_delimit_by_s_mark:w
__hook_use_i_delimit_by_s_mark:nw

Removes tokens until the next `\s__hook_mark`.

```
\cs_new:Npn \__hook_use_none_delimit_by_s_mark:w #1 \s__hook_mark { }
\cs_new:Npn \__hook_use_i_delimit_by_s_mark:nw #1 #2 \s__hook_mark {#1}
```

(*End of definition for* `\__hook_use_none_delimit_by_s_mark:w` *and* `\__hook_use_i_delimit_by_s_mark:nw`.)

__hook_tl_set:cn

Private copies of a few expl3 functions. l3debug will only add debugging to the public names, not to these copies, so we don't have to use `\debug_suspend:` and `\debug_resume:` everywhere.

Functions like `\__hook_tl_set:Nn` have to be redefined, rather than copied because in expl3 they use `\__kernel_tl_(g)set:Nx`, which is also patched by l3debug.

```
\cs_new_protected:Npn \__hook_tl_set:cn #1#2
  { \cs_set_nopar:cpe {#1} { \__kernel_exp_not:w {#2} } }
```

(*End of definition for* `\__hook_tl_set:cn`.)

__hook_tl_gset:Nn
__hook_tl_gset:Ne
__hook_tl_gset:cn
__hook_tl_gset:co
__hook_tl_gset:ce

Same as above.

```
\cs_new_protected:Npn \__hook_tl_gset:Nn #1#2
  { \cs_gset_nopar:Npe #1 { \__kernel_exp_not:w {#2} } }
\cs_new_protected:Npn \__hook_tl_gset:Ne #1#2
  { \cs_gset_nopar:Npe #1 {#2} }
\cs_generate_variant:Nn \__hook_tl_gset:Nn { c, co }
\cs_generate_variant:Nn \__hook_tl_gset:Ne { c }
```

(*End of definition for* `\__hook_tl_gset:Nn`.)

__hook_tl_gput_right:Nn
__hook_tl_gput_right:Ne
__hook_tl_gput_right:cn

Same as above.

```
\cs_new_protected:Npn \__hook_tl_gput_right:Nn #1#2
  { \__hook_tl_gset:Ne #1 { \__kernel_exp_not:w \exp_after:wN { #1 #2 } } }
\cs_generate_variant:Nn \__hook_tl_gput_right:Nn { Ne, cn }
```

(*End of definition for* `\__hook_tl_gput_right:Nn`.)

__hook_tl_gput_left:Nn Same as above.

```
\cs_new_protected:Npn \__hook_tl_gput_left:Nn #1#2
  {
    \__hook_tl_gset:Ne #1
      { \__kernel_exp_not:w {#2} \__kernel_exp_not:w \exp_after:wN {#1} }
  }
```

(*End of definition for* `\__hook_tl_gput_left:Nn`.)

__hook_tl_gset_eq:NN Same as above.

```
\cs_new_eq:NN \__hook_tl_gset_eq:NN \tl_gset_eq:NN
```

(*End of definition for* `\__hook_tl_gset_eq:NN`.)

__hook_tl_gclear:N
__hook_tl_gclear:c Same as above.

```
\cs_new_protected:Npn \__hook_tl_gclear:N #1
  { \__hook_tl_gset_eq:NN #1 \c_empty_tl }
\cs_generate_variant:Nn \__hook_tl_gclear:N { c }
```

(*End of definition for* `\__hook_tl_gclear:N`.)

4.4 Providing new hooks

4.4.1 The data structures of a hook

\g__hook_⟨*hook*⟩_code_prop
__hook␣⟨*hook*⟩
\g__hook_⟨*hook*⟩_reversed_tl
\g__hook_⟨*hook*⟩_declared_tl
\g__hook_⟨*hook*⟩_parameter_tl
__hook_next␣⟨*hook*⟩
__hook_toplevel␣⟨*hook*⟩

Hooks have a name (called ⟨*hook*⟩ in the description below) and for each hook we have to provide a number of data structures. These are

`\g__hook_`⟨*hook*⟩`_code_prop` A property list holding the code for the hook in separate chunks. The keys are by default the package names that add code to the hook, but it is possible for packages to define other keys.

`\g__hook_`⟨*hook*⟩`_rule_`⟨*label1*⟩`|`⟨*label2*⟩`_tl` A token list holding the relation between ⟨*label1*⟩ and ⟨*label2*⟩ in the ⟨*hook*⟩. The ⟨*labels*⟩ are lexically (reverse) sorted to ensure that two labels always point to the same token list. For global rules, the ⟨*hook*⟩ name is `??`.

`\__hook␣`⟨*hook*⟩ The code that is actually executed when the hook is called in the document is stored in this token list. It is constructed from the code chunks applying the information. This token list is named like that so that in case of an error inside the hook, the reported token list in the error is shorter, and to make it simpler to normalize hook names in `\__hook_make_name:n`.

`\g__hook_`⟨*hook*⟩`_reversed_tl` Some hooks are "reversed". This token list stores a `-` for such hook so that it can be identified. The `-` character is used because ⟨*reversed*⟩1 is +1 for normal hooks and −1 for reversed ones.

`\g__hook_`⟨*hook*⟩`_declared_tl` This token list serves as a marker for the hook being officially declared. Its existence is tested to raise an error in case another declaration is attempted.

`\c__hook_`⟨*hook*⟩`_parameter_tl` This token list stores the parameter text for a declared hook (its existence almost completely intersects the token list above), which is used for managing hooks with arguments.

`\__hook_toplevel␣`⟨*hook*⟩ This token list stores the code inserted in the hook from the user's document, in the `top-level` label. This label is special, and doesn't participate in sorting. Instead, all code is appended to it and executed after (or before, if the hook is reversed) the normal hook code, but before the `next` code chunk.

`\__hook_next␣`⟨*hook*⟩ Finally there is extra code (normally empty) that is used on the next invocation of the hook (and then deleted). This can be used to define some special behavior for a single occasion from within the document. This token list follows the same naming scheme than the main `\__hook␣`⟨*hook*⟩ token list. It is called `\__hook_next␣`⟨*hook*⟩ rather than `\__hook␣next_`⟨*hook*⟩ because otherwise a hook whose name is `next_`⟨*hook*⟩ would clash with the next code-token list of the hook called ⟨*hook*⟩.

4.4.2 On the existence of hooks

A hook may be in different states of existence. Here we give an overview of the internal commands to set up hooks and explain how the different states are distinguished. The actual implementation then follows in subsequent sections.

One problem we have to solve is that we need to be able to add code to hooks (e.g., with `\AddToHook`) even if that code has not yet been declared. For example, one package needs to write into a hook of another package, but that package may not get loaded, or is loaded only later. Another problem is that most hooks, but not the generic hooks, require a declaration.

We therefore distinguish the following states for a hook, which are managed by four different tests: structure existence (`\__hook_if_structure_exist:nTF`), creation (`\__hook_if_usable:nTF`), declaration (`\__hook_if_declared:nTF`) and disabled or not (`\__hook_if_disabled:nTF`)

not existing Nothing is known about the hook so far. This state can be detected with `\__hook_if_structure_exist:nTF` (which uses the false branch).

In this state the hook can be declared, disabled, rules can be defined or code could be added to it, but it is not possible to use the hook (with `\UseHook`).

basic data structure set up A hook is this state when its basic data structure has been set up (using `\__hook_init_structure:n`). The data structure setup happens automatically when commands such as `\AddToHook` are used and the hook is at that point in state "not existing".

In this state the four tests give the following results:

`\__hook_if_structure_exist:nTF` returns `true`.
`\__hook_if_usable:nTF` returns `false`.
`\__hook_if_declared:nTF` returns `false`.
`\__hook_if_disabled:nTF` returns `false`.

The allowed actions are the same as in the "not existing" state.

declared A hook is in this state it is not disabled and was explicitly declared (e.g., with `\NewHook`). In this case the four tests give the following results:

`\__hook_if_structure_exist:nTF` returns `true`.

`\__hook_if_usable:nTF` returns `true`.

`\__hook_if_declared:nTF` returns `true`.

`\__hook_if_disabled:nTF` returns `false`.

usable A hook is in this state if it is not disabled, was not explicitly declared but nevertheless is allowed to be used (with `\UseHook` or `\hook_use:n`). This state is only possible for generic hooks as they do not need to be declared. Therefore such hooks move directly from state "not existing" to "usable" the moment a declaration such as `\AddToHook` wants to add to the hook data structure. In this state the tests give the following results:

`\__hook_if_structure_exist:nTF` returns `true`.

`\__hook_if_usable:nTF` returns `true`.

`\__hook_if_declared:nTF` returns `false`.

`\__hook_if_disabled:nTF` returns `false`.

disabled A generic hook in any state is moved to this state when `\DisableGenericHook` is used. This changes the tests to give the following results:

`\__hook_if_structure_exist:nTF` *unchanged.*

`\__hook_if_usable:nTF` returns `false`.

`\__hook_if_declared:nTF` returns `true`.

`\__hook_if_disabled:nTF` returns `true`.

The structure test is unchanged (if the hook was unknown before it is `false`, otherwise `true`). The usable test returns `false` so that any `\UseHook` will bypass the hook from now on. The declared test returns true so that any further `\NewHook` generates an error and the disabled test returns true so that `\AddToHook` can return an error.

FMi: maybe it should do this only after begin document?

4.4.3 Setting hooks up

`\hook_new:n`
`\hook_new_with_args:nn`
`\__hook_new:nn`

The `\hook_new:n` declaration declares a new hook and expects the hook ⟨*name*⟩ as its argument, e.g., `begindocument`.

```
⟨!latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_new_with_args:nn}
⟨!latexrelease⟩                    {Hooks~with~args}
\cs_new_protected:Npn \hook_new:n #1
  { \__hook_normalize_hook_args:Nn \__hook_new:nn {#1} { 0 } }
\cs_new_protected:Npn \hook_new_with_args:nn #1 #2
  { \__hook_normalize_hook_args:Nn \__hook_new:nn {#1} {#2} }

\cs_new_protected:Npn \__hook_new:nn #1 #2
  {
```

We check if the hook was already *explicitly* declared with `\hook_new:n`, and if it already exists we complain, otherwise set the "created" flag for the hook so that it errors next time `\hook_new:n` is used.

```
    \__hook_if_declared:nTF {#1}
      { \msg_error:nnn { hooks } { exists } {#1} }
```

```
      {
        \tl_new:c { g__hook_#1_declared_tl }
        \cs_undefine:c { __hook~#1 }
        \cs_undefine:c { c__hook_#1_parameter_tl }
        \__hook_make_usable:nn {#1} {#2}
```

In case there is already code in a hook, but it's undeclared, run `\__hook_update_hook_code:n` to make it ready to be executed (see test `lthooks-034`).

```
        \__hook_update_hook_code:n {#1}
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_new_with_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new:n #1
⟨latexrelease⟩  { \__hook_normalize_hook_args:Nn \__hook_new:n {#1} }
⟨latexrelease⟩\cs_undefine:N \__hook_new:nn
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_new:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_declared:nTF {#1}
⟨latexrelease⟩      { \msg_error:nnn { hooks } { exists } {#1} }
⟨latexrelease⟩      {
⟨latexrelease⟩        \tl_new:c { g__hook_#1_declared_tl }
⟨latexrelease⟩        \__hook_make_usable:n {#1}
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new_with_args:nn #1 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_new:n`*,* `\hook_new_with_args:nn`*, and* `\__hook_new:nn`*. These functions are documented on page 15.*)

`\__hook_make_usable:nn` This initializes all hook data structures for the hook but if used on its own doesn't mark the hook as declared (as `\hook_new:n` does, so a later `\hook_new:n` on that hook will not result in an error. This command is internally used by `\hook_gput_code:nnn` when adding code to a generic hook.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_make_usable:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_make_usable:nn #1 #2
  {
```

Now we check if the hook's data structure can be safely created without expl3 raising errors, then we add the hook name to the list of all hooks and allocate the necessary data structures for the new hook, otherwise just do nothing.

```
    \__hook_if_usable:nF {#1}
      {
        \seq_gput_right:Nn \g__hook_all_seq {#1}
```

Here we'll define the `\c__hook_⟨hook⟩_parameter_tl` to hold a run of parameters up to the number of arguments of the hook (`#2`).

```
        \__kernel_cs_parm_from_arg_count:nnF
          { \tl_const:cn { c__hook_#1_parameter_tl } } {#2}
          {
            \msg_error:nnnn { hooks } { too-many-args } {#1} {#2}
```

```
            \tl_const:ce { c__hook_#1_parameter_tl }
              { \exp_not:V \c__hook_nine_parameters_tl }
          }
```

After that, use `\__hook_normalize_cs_args:nn` to correct the number of parameters of the macros `\__hook_toplevel␣`⟨*hook*⟩ and `\__hook_next␣`⟨*hook*⟩. We need to be able to add code with arguments to a hook without prior knowledge of the number of arguments of that hook, so lthooks assumes 9 until the hook is properly declared and the number of arguments is known. `\__hook_normalize_cs_args:nn` does the normalization by using the `\c__hook_`⟨*hook*⟩`_parameter_tl` defined just above.

```
        \__hook_normalize_cs_args:nn { _toplevel } {#1}
        \__hook_normalize_cs_args:nn { _next } {#1}
```

This is only used by the actual code of the current hook, so declare it normally:

```
        \__hook_code_gset:nn {#1} { }
```

Now ensure that the base data structure for the hook exists:

```
        \__hook_init_structure:n {#1}
```

The call to `\__hook_normalize_code_pool:n` will correct any improper reference to arguments that don't exist in the hook, raising a low-level TEX error and doubling the offending parameter tokens. It has to be done after `\__hook_init_structure:n` because it operates on `\g__hook_`⟨*hook*⟩`_code_prop`.

```
        \__hook_normalize_code_pool:n {#1}
```

The `\g__hook_`⟨*hook*⟩`_labels_clist` holds the sorted list of labels (once it got sorted). This is used only for debugging. These are defined conditionally, in case `\__hook_make_usable:nn` is being used to redefine a hook.

```
        \clist_if_exist:cF { g__hook_#1_labels_clist }
          {
            \clist_new:c { g__hook_#1_labels_clist }
```

Some hooks should reverse the default order of code chunks. To signal this we have a token list which is empty for normal hooks and contains a `-` for reversed hooks.

```
            \tl_new:c { g__hook_#1_reversed_tl }
          }
```

The above is all in L3 convention, but we also provide an interface to legacy LATEX 2_ε hooks of the form `\@...hook`, e.g., `\@begindocumenthook`. there have been a few of them and they have been added to using `\g@addto@macro`. If there exists such a macro matching the name of the new hook, i.e., `\@`⟨*hook-name*⟩`hook` and it is not empty then we add its contents as a code chunk under the label `legacy`.

Warning: this support will vanish in future releases!

```
        \__hook_include_legacy_code_chunk:n {#1}
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_make_usable:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_make_usable:nn
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_make_usable:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_exist:cF { __hook~#1 }
⟨latexrelease⟩      {
```

```
⟨latexrelease⟩          \seq_gput_right:Nn \g__hook_all_seq {#1}
⟨latexrelease⟩          \tl_new:c { __hook~#1 }
⟨latexrelease⟩          \__hook_init_structure:n {#1}
⟨latexrelease⟩          \clist_new:c { g__hook_#1_labels_clist }
⟨latexrelease⟩          \tl_new:c { g__hook_#1_reversed_tl }
⟨latexrelease⟩          \__hook_include_legacy_code_chunk:n {#1}
⟨latexrelease⟩        }
⟨latexrelease⟩    }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_make_usable:nn`.)

`\__hook_init_structure:n`

This function declares the basic data structures for a hook without explicit declaring the hook itself. This is needed to allow adding to undeclared hooks. Here it is unnecessary to check whether all variables exist, since all three are declared at the same time (either all of them exist, or none).

It creates the hook code pool (`\g__hook_`⟨*hook*⟩`_code_prop`) and the `top-level` and `next` token lists. A hook is initialized with `\__hook_init_structure:n` the first time anything is added to it. Initializing a hook just with `\__hook_init_structure:n` will not make it usable with `\hook_use:n`.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_init_structure:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_init_structure:n #1
  {
    \__hook_if_structure_exist:nF {#1}
      {
        \prop_new:c { g__hook_#1_code_prop }
        \__hook_toplevel_gset:nn {#1} { }
        \__hook_next_gset:nn {#1} { }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_init_structure:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_init_structure:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_structure_exist:nF {#1}
⟨latexrelease⟩      {
⟨latexrelease⟩        \prop_new:c { g__hook_#1_code_prop }
⟨latexrelease⟩        \tl_new:c { __hook_toplevel~#1 }
⟨latexrelease⟩        \tl_new:c { __hook_next~#1 }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_init_structure:n`.)

`\hook_new_reversed:n`
`\hook_new_reversed_with_args:nn`
`\__hook_new_reversed:nn`

Declare a new hook. The default ordering of code chunks is reversed, signaled by setting the token list to a minus sign.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_new_reversed_with_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_new_reversed:n #1
  { \__hook_normalize_hook_args:Nn \__hook_new_reversed:nn {#1} { 0 } }
```

```
\cs_new_protected:Npn \hook_new_reversed_with_args:nn #1 #2
  { \__hook_normalize_hook_args:Nn \__hook_new_reversed:nn {#1} {#2} }
\cs_new_protected:Npn \__hook_new_reversed:nn #1 #2
  {
    \__hook_if_declared:nTF {#1}
      { \msg_error:nnn { hooks } { exists } {#1} }
      {
        \__hook_new:nn {#1} {#2}
        \tl_gset:cn { g__hook_#1_reversed_tl } { - }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_new_reversed_with_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new_reversed:n #1
⟨latexrelease⟩  { \__hook_normalize_hook_args:Nn \__hook_new_reversed:n {#1} }
⟨latexrelease⟩\cs_undefine:N \__hook_new_reversed:nn
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_new_reversed:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_new:n {#1}
⟨latexrelease⟩    \tl_gset:cn { g__hook_#1_reversed_tl } { - }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_undefine:N \__hook_new_reversed:nn
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new_reversed_with_args:nn #1 #2 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_new_reversed:n`, `\hook_new_reversed_with_args:nn`, *and* `\__hook_new_reversed:nn`. *These functions are documented on page* 15.)

`\hook_new_pair:nn`
`\hook_new_pair_with_args:nnn`

A shorthand for declaring a normal and a (matching) reversed hook in one go.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_new_pair_with_args:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_new_pair:nn #1#2
  { \__hook_normalize_hook_args:Nnn \__hook_new_pair:nnn {#1} {#2} { 0 } }
\cs_new_protected:Npn \hook_new_pair_with_args:nnn #1#2#3
  { \__hook_normalize_hook_args:Nnn \__hook_new_pair:nnn {#1} {#2} {#3} }
\cs_new_protected:Npn \__hook_new_pair:nnn #1 #2 #3
  {
    \__hook_if_declared:nTF {#1}
      { \msg_error:nnn { hooks } { exists } {#1} }
      {
        \__hook_if_declared:nTF {#2}
          { \msg_error:nnn { hooks } { exists } {#2} }
          {
            \__hook_new:nn {#1} {#3}
            \__hook_new_reversed:nn {#2} {#3}
          }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_new_pair_with_args:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new_pair:nn #1#2
⟨latexrelease⟩  {
```

```
⟨latexrelease⟩      \hook_new:n {#1}
⟨latexrelease⟩      \hook_new_reversed:n {#2}
⟨latexrelease⟩    }
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new_pair_with_args:nnn #1#2#3
⟨latexrelease⟩    { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_new_pair:nn` *and* `\hook_new_pair_with_args:nnn`. *These functions are documented on page* 15.)

`\__hook_include_legacy_code_chunk:n`

The LaTeX legacy concept for hooks uses with hooks the following naming scheme in the code: `\@...hook`.

If this macro is not empty we add it under the label `legacy` to the current hook and then empty it globally. This way packages or classes directly manipulating commands such as `\@begindocumenthook` still get their hook data added.

Warning: this support will vanish in future releases!

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_include_legacy_code_chunk:n}
⟨latexrelease⟩                    {Hooks~with~args}
\cs_new_protected:Npn \__hook_include_legacy_code_chunk:n #1
  {
```

If the macro doesn't exist (which is the usual case) then nothing needs to be done.

```
    \tl_if_exist:cT { @#1hook }
      {
```

Of course if the legacy hook exists but is empty, there is no need to add anything under `legacy` the legacy label.

```
        \tl_if_empty:cF { @#1hook }
          {
```

Here we set `\__hook_replacing_args_false:` because no legacy code will reference hook arguments.

```
            \__hook_replacing_args_false:
            \use:e
              {
                \__hook_hook_gput_code_do:nnn {#1} { legacy }
                  { \exp_not:v { @#1hook } }
              }
            \__hook_replacing_args_reset:
```

Once added to the hook, we need to clear it otherwise it might get added again later if the hook data gets updated.

```
            \__hook_tl_gclear:c { @#1hook }
          }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_include_legacy_code_chunk:n}
⟨latexrelease⟩                    {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_include_legacy_code_chunk:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_exist:cT { @#1hook }
⟨latexrelease⟩      {
⟨latexrelease⟩        \tl_if_empty:cF { @#1hook }
```

```
⟨latexrelease⟩           {
⟨latexrelease⟩             \exp_args:Nnnv \__hook_hook_gput_code_do:nnn
⟨latexrelease⟩               {#1} { legacy } { @#1hook }
⟨latexrelease⟩             \__hook_tl_gclear:c { @#1hook }
⟨latexrelease⟩           }
⟨latexrelease⟩       }
⟨latexrelease⟩   }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_include_legacy_code_chunk:n`.)

4.4.4 Disabling and providing hooks

`\hook_disable_generic:n`
`\__hook_disable:n`
`\__hook_if_disabled_p:n`
`\__hook_if_disabled:nTF`

Disables a hook by creating its `\g__hook_⟨hook⟩_declared_tl` so that the hook errors when used with `\hook_new:n`, then it undefines `\__hook_⟨hook⟩` so that it may not be executed.

This does not clear any code that may be already stored in the hook's structure, but doesn't allow adding more code. `\__hook_if_disabled:nTF` uses that specific combination to check if the hook is disabled.

```
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\hook_disable_generic:n}
⟨latexrelease⟩                  {Disable~hooks}
\cs_new_protected:Npn \hook_disable_generic:n #1
  { \__hook_normalize_hook_args:Nn \__hook_disable:n {#1} }
\cs_new_protected:Npn \__hook_disable:n #1
  {
    \tl_gclear_new:c { g__hook_#1_declared_tl }
    \cs_undefine:c { __hook~#1 }
  }
\prg_new_conditional:Npnn \__hook_if_disabled:n #1 { p, T, F, TF }
  {
    \bool_lazy_and:nnTF
        { \tl_if_exist_p:c { g__hook_#1_declared_tl } }
        { ! \cs_if_exist_p:c { __hook~#1 } }
      { \prg_return_true: }
      { \prg_return_false: }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_disable_generic:n}
⟨latexrelease⟩                  {Disable~hooks}
⟨latexrelease⟩
⟨latexrelease⟩\cs_new_protected:Npn \hook_disable_generic:n #1 {}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_disable_generic:n`, `\__hook_disable:n`, *and* `\__hook_if_disabled:nTF`. *This function is documented on page* 16.)

`\hook_activate_generic:n`
`\__hook_activate_generic:n`

The `\hook_activate_generic:n` declaration declares a new hook if it wasn't declared already, in which case it only checks that the already existing hook is not a reversed hook.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_activate_generic:n}
⟨latexrelease⟩                  {Providing~hooks}
```

```
\cs_new_protected:Npn \hook_activate_generic:n #1
  { \__hook_normalize_hook_args:Nn \__hook_activate_generic:nn {#1} {   } }
\cs_new_protected:Npn \__hook_activate_generic:nn #1 #2
  {
```

If the hook to be activated was disabled we warn (for now — this may change).

```
    \__hook_if_disabled:nTF {#1}
      { \msg_warning:nnn { hooks } { activate-disabled } {#1} }
```

Otherwise we check if the hook is not declared, and if it isn't, figure out if it's reversed or not, then declare it accordingly.

```
    {
      \__hook_if_declared:nF {#1}
        {
          \tl_new:c { g__hook_#1_declared_tl }
          \__hook_make_usable:nn {#1} { 0 }
          \tl_gset:ce { g__hook_#1_reversed_tl }
            { \__hook_if_generic_reversed:nT {#1} { - } }
```

Reflect that we have activated the generic hook and set its execution code.

```
          \__hook_update_hook_code:n {#1}
        }
    }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\hook_activate_generic:n}
⟨latexrelease⟩                  {Providing~hooks}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_activate_generic:nn #1 #2
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_disabled:nTF {#1}
⟨latexrelease⟩      { \msg_warning:nnn { hooks } { activate-disabled } {#1} }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_if_declared:nF {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_new:c { g__hook_#1_declared_tl }
⟨latexrelease⟩            \__hook_make_usable:n {#1}
⟨latexrelease⟩            \tl_gset:cx { g__hook_#1_reversed_tl }
⟨latexrelease⟩              { \__hook_if_generic_reversed:nT {#1} { - } }
⟨latexrelease⟩            \__hook_update_hook_code:n {#1}
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_activate_generic:n}
⟨latexrelease⟩                  {Providing~hooks}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_activate_generic:n #1 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_activate_generic:n` *and* `\__hook_activate_generic:n`*. This function is documented on page* 16.)

4.5 Parsing a label

__hook_parse_label_default:nN

This macro checks if a label was given (not \c_novalue_tl), and if so, tries to parse the label looking for a leading . to replace by __hook_currname_or_default:. #2 is a boolean representing if #1 is a label name.

```
\cs_new:Npn \__hook_parse_label_default:nN #1#2
  {
    \tl_if_novalue:nTF {#1}
      { \__hook_currname_or_default: }
      { \tl_trim_spaces_apply:nN {#1} \__hook_parse_dot_label:nN #2 }
  }
```

(*End of definition for* __hook_parse_label_default:nN.)

__hook_parse_dot_label:nN
__hook_parse_dot_label:w
__hook_parse_dot_label_cleanup:w
__hook_parse_dot_label_aux:w

Start by checking if the label is empty, which raises an error, and uses the fallback value. If not, split the label at a ./, if any, and check if no tokens are before the ./, or if the only character is a .. If these requirements are fulfilled, the leading . is replaced with __hook_currname_or_default:. Otherwise the label is returned unchanged. #2 is a boolean representing if #1 is a label name.

```
\cs_new:Npn \__hook_parse_dot_label:nN #1#2
  {
    \tl_if_empty:nTF {#1}
      {
        \bool_if:NTF #2
          { \msg_expandable_error:nn { hooks } { empty-label } }
          { \msg_expandable_error:nn { hooks } { empty-hook } }
        \__hook_currname_or_default:
      }
      {
        \str_if_eq:nnTF {#1} { . }
          { \__hook_currname_or_default: }
          { \__hook_parse_dot_label:w #1 ./ \s__hook_mark }
      }
  }
\cs_new:Npn \__hook_parse_dot_label:w #1 ./ #2 \s__hook_mark
  {
    \tl_if_empty:nTF {#1}
      { \__hook_parse_dot_label_aux:w #2 \s__hook_mark }
      {
        \tl_if_empty:nTF {#2}
          { \__hook_make_name:n {#1} }
          { \__hook_parse_dot_label_cleanup:w #1 ./ #2 \s__hook_mark }
      }
  }
\cs_new:Npn \__hook_parse_dot_label_cleanup:w #1 ./ \s__hook_mark {#1}
\cs_new:Npn \__hook_parse_dot_label_aux:w #1 ./ \s__hook_mark
  { \__hook_currname_or_default: / \__hook_make_name:n {#1} }
```

(*End of definition for* __hook_parse_dot_label:nN *and others.*)

__hook_currname_or_default:

This uses \g__hook_hook_curr_name_tl if it is set, otherwise it tries \@currname. If neither is set, it raises an error and uses the fallback value label-missing.

```
\cs_new:Npn \__hook_currname_or_default:
  {
```

```
    \tl_if_empty:NTF \g__hook_hook_curr_name_tl
      {
        \tl_if_empty:NTF \@currname
          {
            \msg_expandable_error:nnn { latex2e } { should-not-happen }
              { Empty~default~label. }
            \__hook_make_name:n { label-missing }
          }
          { \@currname }
      }
      { \g__hook_hook_curr_name_tl }
  }
```

(*End of definition for* `\__hook_currname_or_default:`.)

`\__hook_make_name:n`
`\__hook_make_name:w`

This provides a standard sanitization of a hook's name. It uses `\cs:w` to build a control sequence out of the hook name, then uses `\cs_to_str:N` to get the string representation of that, without the escape character. `\cs:w`-based expansion is used instead of `e`-based because Unicode characters don't behave well inside `\expanded`. The macro adds the `\__hook␣` prefix to the hook name to reuse the hook's code token list to build the csname and avoid leaving "public" control sequences defined (as `\relax`) in TeX's memory.

```
\cs_new:Npn \__hook_make_name:n #1
  {
    \exp_after:wN \exp_after:wN \exp_after:wN \__hook_make_name:w
    \exp_after:wN \token_to_str:N \cs:w __hook~ #1 \cs_end:
  }
\exp_last_unbraced:NNNNo
\cs_new:Npn \__hook_make_name:w #1 \tl_to_str:n { __hook~ } { }
```

(*End of definition for* `\__hook_make_name:n` *and* `\__hook_make_name:w`.)

`\__hook_normalize_hook_args:Nn`
`\__hook_normalize_hook_args:Nnn`
`\__hook_normalize_hook_rule_args:Nnnnn`
`\__hook_normalize_hook_args_aux:Nn`

This is the standard route for normalizing hook and label arguments. The main macro does the entire operation within a group so that csnames made by `\__hook_make_name:n` are wiped off before continuing. This means that this function cannot be used for `\hook_use:n`!

```
\cs_new_protected:Npn \__hook_normalize_hook_args_aux:Nn #1 #2
  {
    \group_begin:
    \use:e
      {
        \group_end:
        \exp_not:N #1 #2
      }
  }
\cs_new_protected:Npn \__hook_normalize_hook_args:Nn #1 #2
  {
    \__hook_normalize_hook_args_aux:Nn #1
      { { \__hook_parse_label_default:nN {#2} \c_false_bool } }
  }
\cs_new_protected:Npn \__hook_normalize_hook_args:Nnn #1 #2 #3
  {
    \__hook_normalize_hook_args_aux:Nn #1
      {
        { \__hook_parse_label_default:nN {#2} \c_false_bool }
```

```
        { \__hook_parse_label_default:nN {#3} \c_true_bool }
      }
  }
\cs_new_protected:Npn \__hook_normalize_hook_rule_args:Nnnnn #1 #2 #3 #4 #5
  {
    \__hook_normalize_hook_args_aux:Nn #1
      {
        { \__hook_parse_label_default:nN {#2} \c_false_bool }
        { \__hook_parse_label_default:nN {#3} \c_true_bool }
        { \tl_trim_spaces:n {#4} }
        { \__hook_parse_label_default:nN {#5} \c_true_bool }
      }
  }
```

(*End of definition for* `\__hook_normalize_hook_args:Nn` *and others.*)

`\__hook_curr_name_push:n`
`\__hook_curr_name_push_aux:n`
`\__hook_curr_name_pop:`
`\__hook_end_document_label_check:`

The token list `\g__hook_hook_curr_name_tl` stores the name of the current package/file to be used as the default label in hooks. Providing a consistent interface is tricky because packages can be loaded within packages, and some packages may not use `\SetDefaultHookLabel` to change the default label (in which case `\@currname` is used).

To pull that one off, we keep a stack that contains the default label for each level of input. The bottom of the stack contains the default label for the `top-level` (this stack should never go empty). If we're building the format, set the default label to be `top-level`:

```
\tl_gset:Nn \g__hook_hook_curr_name_tl { top-level }
```

Then, in case we're in latexrelease we push something on the stack to support roll forward. But in some rare cases, latexrelease may be loaded inside another package (notably platexrelease), so we'll first push the `top-level` entry:

```
⟨latexrelease⟩\seq_if_empty:NT \g__hook_name_stack_seq
⟨latexrelease⟩  { \seq_gput_right:Nn \g__hook_name_stack_seq { top-level } }
```

then we dissect the `\@currnamestack`, adding `\@currname` to the stack:

```
⟨latexrelease⟩\cs_set_protected:Npn \__hook_tmp:w #1 #2 #3
⟨latexrelease⟩  {
⟨latexrelease⟩    \quark_if_recursion_tail_stop:n {#1}
⟨latexrelease⟩    \seq_gput_right:Nn \g__hook_name_stack_seq {#1}
⟨latexrelease⟩    \__hook_tmp:w
⟨latexrelease⟩  }
⟨latexrelease⟩\exp_after:wN \__hook_tmp:w \@currnamestack
⟨latexrelease⟩  \q_recursion_tail \q_recursion_tail
⟨latexrelease⟩  \q_recursion_tail \q_recursion_stop
```

and finally set the default label to be the `\@currname`:

```
⟨latexrelease⟩\tl_gset:Nx \g__hook_hook_curr_name_tl { \@currname }
⟨latexrelease⟩\seq_gpop_right:NN \g__hook_name_stack_seq \l__hook_tmpa_tl
```

Two commands keep track of the stack: when a file is input, `\__hook_curr_name_push:n` pushes the current default label onto the stack and sets the new default label (all in one go):

```
\cs_new_protected:Npn \__hook_curr_name_push:n #1
  {
      \exp_args:Ne \__hook_curr_name_push_aux:n
        {
          \tl_if_blank:nF {#1}
```

```
            { \__hook_parse_dot_label:nN {#1} \c_false_bool }
        }
  }
\cs_new_protected:Npn \__hook_curr_name_push_aux:n #1
  {
    \tl_if_blank:nTF {#1}
      { \msg_error:nn { hooks } { no-default-label } }
      {
        \str_if_eq:nnTF {#1} { top-level }
          {
            \msg_error:nnnnn { hooks } { set-top-level }
              { to } { PushDefaultHookLabel } {#1}
          }
          {
            \seq_gpush:NV \g__hook_name_stack_seq \g__hook_hook_curr_name_tl
            \tl_gset:Nn \g__hook_hook_curr_name_tl {#1}
          }
      }
  }
```

and when an input is over, the topmost item of the stack is popped, since that label will not be used again, and `\g__hook_hook_curr_name_tl` is updated to equal the now topmost item of the stack:

```
\cs_new_protected:Npn \__hook_curr_name_pop:
  {
    \seq_gpop:NNTF \g__hook_name_stack_seq \l__hook_return_tl
      { \tl_gset_eq:NN \g__hook_hook_curr_name_tl \l__hook_return_tl }
      { \msg_error:nn { hooks } { extra-pop-label } }
  }
```

At the end of the document we want to check if there was no `\__hook_curr_name_push:n` without a matching `\__hook_curr_name_pop:` (not a critical error, but it might indicate that something else is not quite right):

```
\tl_gput_right:Nn \@kernel@after@enddocument@afterlastpage
  { \__hook_end_document_label_check: }
\cs_new_protected:Npn \__hook_end_document_label_check:
  {
    \seq_gpop:NNT \g__hook_name_stack_seq \l__hook_return_tl
      {
        \msg_error:nne { hooks } { missing-pop-label }
          { \g__hook_hook_curr_name_tl }
        \tl_gset_eq:NN \g__hook_hook_curr_name_tl \l__hook_return_tl
        \__hook_end_document_label_check:
      }
  }
```

The token list `\g__hook_hook_curr_name_tl` is but a mirror of the top of the stack.

Now define a wrapper that replaces the top of the stack with the argument, and updates `\g__hook_hook_curr_name_tl` accordingly.

```
\cs_new_protected:Npn \__hook_set_default_hook_label:n #1
  {
    \seq_if_empty:NTF \g__hook_name_stack_seq
      {
        \msg_error:nnnnn { hooks } { set-top-level }
```

`\__hook_set_default_hook_label:n`
`\__hook_set_default_label:n`

```
          { for } { SetDefaultHookLabel } {#1}
      }
      {
          \exp_args:Ne \__hook_set_default_label:n
            {
              \tl_if_blank:nF {#1}
                { \__hook_parse_dot_label:nN {#1} \c_false_bool }
            }
      }
  }
\cs_new_protected:Npn \__hook_set_default_label:n #1
  {
    \str_if_eq:nnTF {#1} { top-level }
      {
        \msg_error:nnnnn { hooks } { set-top-level }
          { to } { SetDefaultHookLabel } {#1}
      }
      { \tl_gset:Nn \g__hook_hook_curr_name_tl {#1} }
  }
```

(*End of definition for* `\__hook_curr_name_push:n` *and others.*)

4.6 Adding or removing hook code

`\hook_gput_code:nnn`
`\hook_gput_code_with_args:nnn`
`\__hook_gput_code:nnn`
`\__hook_gput_code_store:nnn`
`\__hook_hook_gput_code_do:nnn`
`\__hook_prop_gput_labeled_cleanup:nnn`
`\__hook_prop_gput_labeled_do:Nnnn`
`\__hook_hash_check:nTF`
`\__hook_hash_check_aux:w`

With `\hook_gput_code:nnn{`⟨*hook*⟩`}{`⟨*label*⟩`}{`⟨*code*⟩`}` a chunk of ⟨*code*⟩ is added to an existing ⟨*hook*⟩ labeled with ⟨*label*⟩.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_gput_code:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_gput_code:nnn #1 #2 #3
  {
    \__hook_replacing_args_false:
    \__hook_normalize_hook_args:Nnn \__hook_gput_code:nnn {#1} {#2} {#3}
    \__hook_replacing_args_reset:
  }
\cs_new_protected:Npn \hook_gput_code_with_args:nnn #1 #2 #3
  {
    \__hook_replacing_args_true:
    \__hook_normalize_hook_args:Nnn \__hook_gput_code:nnn {#1} {#2} {#3}
    \__hook_replacing_args_reset:
  }
```

If `\AddToHookWithArguments` was used, do some sanity checking, and if it's not possible to use arguments at this point, fall back to regular `\AddToHook` by using `\__hook_replacing_args_false:`.

```
\cs_new_protected:Npn \__hook_gput_code:nnn #1 #2 #3
  {
    \__hook_chk_args_allowed:nn {#1} { AddToHook }
```

Then check if the code should be executed immediately, rather than stored:

```
    \__hook_if_execute_immediately:nTF {#1}
      {
```

`\AddToHookWithArguments` can't be used on one-time hooks (that were already used).

```
        \__hook_if_replacing_args:TF
          {
```

```
            \msg_error:nnnn { hooks } { one-time-args }
              {#1} { AddToHook }
          }
          { }
        \use:n
      }
      { \__hook_gput_code_store:nnn {#1} {#2} }
        {#3}
  }
\cs_new_protected:Npn \__hook_gput_code_store:nnn #1 #2 #3
  {
```

Then check if the hook is usable.

```
    \__hook_if_usable:nTF {#1}
```

If so we simply add (or append) the new code to the property list holding different chunks for the hook. At `\begin{document}` this is then sorted into a token list for fast execution.

```
      {
        \__hook_hook_gput_code_do:nnn {#1} {#2} {#3}
```

However, if there is an update within the document we need to alter this execution code which is done by `\__hook_update_hook_code:n`. In the preamble this does nothing.

```
        \__hook_update_hook_code:n {#1}
      }
```

If the hook is not usable, before giving up, check if it's not disabled and otherwise try to declare it as a generic hook, if its name matches one of the valid patterns.

```
      {
        \__hook_if_disabled:nTF {#1}
          { \msg_error:nnn { hooks } { hook-disabled } {#1} }
          { \__hook_try_declaring_generic_hook:nnn {#1} {#2} {#3} }
      }
  }
```

This macro will unconditionally add a chunk of code to the given hook.

```
\cs_new_protected:Npn \__hook_hook_gput_code_do:nnn #1 #2 #3
  {
```

However, first some debugging info if debugging is enabled:

```
    \__hook_debug:n{\iow_term:e{[lthooks]~ Add~ to~
                        \__hook_if_usable:nF {#1} { undeclared~ }
                        hook~ '#1'~ (#2) \on@line
                        ^^J [lthooks] \@spaces
                        <-~ \tl_to_str:n{#3}} }
```

Then try to get the code chunk labeled `#2` from the hook. If there's code already there, then append `#3` to that, otherwise just put `#3`. If the current label is `top-level`, the code is added to a dedicated token list `\__hook_toplevel␣`⟨*hook*⟩ that goes at the end of the hook (or at the beginning, for a reversed hook), just before `\__hook_next␣`⟨*hook*⟩.

```
    \str_if_eq:nnTF {#2} { top-level }
      {
        \str_if_eq:eeTF { top-level } { \__hook_currname_or_default: }
          {
```

If the hook's basic structure does not exist, we need to declare it with `\__hook_init_structure:n`.

```
            \__hook_init_structure:n {#1}
```

Then append to the `_toplevel` container for the hook.

```
            \__hook_cs_gput_right:nnn { _toplevel } {#1} {#3}
          }
          { \msg_error:nnn { hooks } { misused-top-level } {#1} }
      }
      {
```

When adding to the code pool, we have to double hashes if `\AddToHook` was used (`replacing_args` is false), so that later it is turned into a single parameter token, rather than a parameter to the hook macro. We skip this step if there are no hashes at all in the argument: the token-by-token approach otherwise becomes a major performance issue when the contents of the hook are long.

```
        \exp_args:Ne \__hook_prop_gput_labeled_cleanup:nnn
          {
            \__hook_if_replacing_args:TF
              { \exp_not:n }
              {
                \__hook_hash_check:nTF {#3}
                  { \exp_not:n }
                  { \__hook_double_hashes:n }
              }
                {#3}
          }
          {#1} {#2}
      }
  }
\cs_new:Npe \__hook_hash_check:nTF #1
  {
    \exp_not:N \exp_after:wN \exp_not:N \__hook_hash_check_aux:w
      \exp_not:N \tl_to_str:n {#1} \c_hash_str \c_hash_str
      \exp_not:N \q_stop
  }
\use:e
  {
    \cs_new:Npn \exp_not:N \__hook_hash_check_aux:w
      #1 \c_hash_str #2 \c_hash_str #3 \exp_not:N \q_stop
  }
  { \tl_if_empty:nTF {#3} }
```

Adds code to a hook's code pool.

```
\cs_new_protected:Npn \__hook_prop_gput_labeled_cleanup:nnn #1 #2 #3
  {
    \tl_set:Nn \l__hook_return_tl {#1}
    \__hook_if_replacing_args:TF
      {
        \__hook_if_usable:nT {#2}
          {
            \__hook_set_normalize_fn:nn {#2}
              { Invalid~code~added~\msg_line_context: }
            \__hook_normalize_fn:nn {#3} {#1}
            \prop_get:NnN \l__hook_work_prop {#3} \l__hook_return_tl
```

```
          }
      }
      { }
    \exp_args:NcV \__hook_prop_gput_labeled_do:Nnn
      { g__hook_#2_code_prop } \l__hook_return_tl {#3}
  }
\cs_new_protected:Npn \__hook_prop_gput_labeled_do:Nnn #1 #2 #3
  {
    \prop_get:NnNTF #1 {#3} \l__hook_return_tl
      { \prop_gput:Nno #1 {#3} { \l__hook_return_tl #2 } }
      { \prop_gput:Nnn #1 {#3} {#2} }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_gput_code:nnn}
⟨latexrelease⟩                  {Providing~hooks}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_gput_code:nnn #1 #2
⟨latexrelease⟩  { \__hook_normalize_hook_args:Nnn
⟨latexrelease⟩        \__hook_gput_code:nnn {#1} {#2} }
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_gput_code:nnn #1 #2 #3
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_execute_immediately:nTF {#1}
⟨latexrelease⟩      {#3}
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_if_usable:nTF {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_hook_gput_code_do:nnn {#1} {#2} {#3}
⟨latexrelease⟩            \__hook_update_hook_code:n {#1}
⟨latexrelease⟩          }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_if_disabled:nTF {#1}
⟨latexrelease⟩              { \msg_error:nnn { hooks } { hook-disabled } {#1} }
⟨latexrelease⟩              { \__hook_try_declaring_generic_hook:nnn
⟨latexrelease⟩                  {#1} {#2} {#3} }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_hook_gput_code_do:nnn #1 #2 #3
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_debug:n{\iow_term:x{****~ Add~ to~
⟨latexrelease⟩                      \__hook_if_usable:nF {#1} { undeclared~ }
⟨latexrelease⟩                      hook~ #1~ (#2)
⟨latexrelease⟩                      \on@line\space <-~ \tl_to_str:n{#3}} }
⟨latexrelease⟩    \str_if_eq:nnTF {#2} { top-level }
⟨latexrelease⟩      {
⟨latexrelease⟩        \str_if_eq:eeTF { top-level }
⟨latexrelease⟩                        { \__hook_currname_or_default: }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_init_structure:n {#1}
⟨latexrelease⟩            \__hook_tl_gput_right:cn { __hook_toplevel~#1 } {#3}
⟨latexrelease⟩          }
⟨latexrelease⟩          { \msg_error:nnn { hooks } { misused-top-level } {#1} }
⟨latexrelease⟩      }
⟨latexrelease⟩      {
```

```
⟨latexrelease⟩          \prop_get:cnNTF
⟨latexrelease⟩            { g__hook_#1_code_prop } {#2} \l__hook_return_tl
⟨latexrelease⟩            {
⟨latexrelease⟩              \prop_gput:cno { g__hook_#1_code_prop } {#2}
⟨latexrelease⟩                { \l__hook_return_tl #3 }
⟨latexrelease⟩            }
⟨latexrelease⟩            { \prop_gput:cnn { g__hook_#1_code_prop } {#2} {#3} }
⟨latexrelease⟩        }
⟨latexrelease⟩    }
⟨latexrelease⟩\cs_gset_protected:Npn \hook_gput_code_with_args:nnn #1#2#3 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_gput_code:nnn` *and others. These functions are documented on page 17.*)

`\__hook_chk_args_allowed:nn`

This macro checks if it is possible to add code with references to a hook's arguments for hook `#1`. It only does something if the function being run is `replacing_args`. This macro will error if the hook is declared and takes no arguments, then it will set `\__hook_replacing_args_false:` so that the macro which called it will add the code normally.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_chk_args_allowed:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_chk_args_allowed:nn #1 #2
  {
    \__hook_if_replacing_args:TF
      {
        \__hook_if_declared:nT {#1}
          { \tl_if_empty:cT { c__hook_#1_parameter_tl } { \use_ii:nn } }
        \use_none:n
          {
            \msg_error:nnnn { hooks } { without-args } {#1} {#2}
            \__hook_replacing_args_false:
          }
      }
      { }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_chk_args_allowed:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_chk_args_allowed:nn
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_chk_args_allowed:nn`.)

`\__hook_gput_undeclared_hook:nnn`

Often it may happen that a package A defines a hook `foo`, but package B, that adds code to that hook, is loaded before A. In such case we need to add code to the hook before it is declared. An implicitly declared hook doesn't have arguments (in principle), so use `\c_false_bool` here.

```
\cs_new_protected:Npn \__hook_gput_undeclared_hook:nnn #1 #2 #3
  {
    \__hook_init_structure:n {#1}
    \__hook_hook_gput_code_do:nnn {#1} {#2} {#3}
  }
```

(*End of definition for* `\__hook_gput_undeclared_hook:nnn`.)

__hook_try_declaring_generic_hook:nnn
__hook_try_declaring_generic_next_hook:nn

These entry-level macros just pass the arguments along to the common `\__hook_try_declaring_generic_hook:nNNnn` with the right functions to execute when some action is to be taken.

The wrapper `\__hook_try_declaring_generic_hook:nnn` then defers `\hook_gput_code:nnn` if the generic hook was declared, or to `\__hook_gput_undeclared_hook:nnn` otherwise (the hook was tested for existence before, so at this point if it isn't generic, it doesn't exist).

The wrapper `\__hook_try_declaring_generic_next_hook:nn` for next-execution hooks does the same: it defers the code to `\hook_gput_next_code:nn` if the generic hook was declared, or to `\__hook_gput_next_do:nn` otherwise.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_try_declaring_generic_hook:nnn #1
  {
    \__hook_try_declaring_generic_hook:wnTF #1 / / / \scan_stop: {#1}
      \__hook_gput_code:nnn
      \__hook_gput_undeclared_hook:nnn
        {#1}
  }
\cs_new_protected:Npn \__hook_try_declaring_generic_next_hook:nn #1
  {
    \__hook_try_declaring_generic_hook:wnTF #1 / / / \scan_stop: {#1}
      \__hook_gput_next_code:nn
      \__hook_gput_next_do:nn
        {#1}
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/11/15}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:nnn}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_try_declaring_generic_hook:nnn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wnTF #1 / / / \scan_stop:
⟨latexrelease⟩      {#1}
⟨latexrelease⟩      \hook_gput_code:nnn
⟨latexrelease⟩      \__hook_gput_undeclared_hook:nnn
⟨latexrelease⟩        {#1}
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset_protected:Npn
⟨latexrelease⟩  \__hook_try_declaring_generic_next_hook:nn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wnTF #1 / / / \scan_stop:
⟨latexrelease⟩      {#1}
⟨latexrelease⟩      \hook_gput_next_code:nn
⟨latexrelease⟩      \__hook_gput_next_do:nn
⟨latexrelease⟩        {#1}
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:nnn}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new_protected:Npn
```

```
⟨latexrelease⟩  \__hook_try_declaring_generic_hook:nnn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:nNNnn {#1}
⟨latexrelease⟩      \hook_gput_code:nnn \__hook_gput_undeclared_hook:nnn
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new_protected:Npn
⟨latexrelease⟩  \__hook_try_declaring_generic_next_hook:nn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:nNNnn {#1}
⟨latexrelease⟩      \hook_gput_next_code:nn \__hook_gput_next_do:nn
⟨latexrelease⟩  }
```

(*End of definition for* `\__hook_try_declaring_generic_hook:nnn` *and* `\__hook_try_declaring_generic_next_hook:nn`.)

`\__hook_try_declaring_generic_hook:nNNnn`
`hook_try_declaring_generic_hook_split:nNNnn`

`\__hook_try_declaring_generic_hook:nNNnn` now splits the hook name at the first `/` (if any) and first checks if it is a file-specific hook (they require some normalization) using `\__hook_if_file_hook:wTF`. If not then check it is one of a predefined set for generic names. We also split off the second component to see if we have to make a reversed hook. In either case the function returns ⟨`true`⟩ for a generic hook and ⟨`false`⟩ in other cases.

```
⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_declaring_generic_hook:nNNnn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_file_hook:wTF #1 / / \s__hook_mark
⟨latexrelease⟩      {
⟨latexrelease⟩        \exp_args:Ne
⟨latexrelease⟩        \__hook_try_declaring_generic_hook_split:nNNnn
⟨latexrelease⟩          { \exp_args:Ne \__hook_file_hook_normalize:n {#1} }
⟨latexrelease⟩      }
⟨latexrelease⟩      { \__hook_try_declaring_generic_hook_split:nNNnn {#1} }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new_protected:Npn
⟨latexrelease⟩    \__hook_try_declaring_generic_hook_split:nNNnn #1 #2 #3
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wnTF #1 / / / \scan_stop:
⟨latexrelease⟩      {#1}
⟨latexrelease⟩      { #2 }
⟨latexrelease⟩      { #3 } {#1}
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_try_declaring_generic_hook:nNNnn` *and* `\__hook_try_declaring_generic_hook_split:nNNnn`.)

`\__hook_try_declaring_generic_hook:wnTF`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}
⟨latexrelease⟩                 {\__hook_try_declaring_generic_hook:wn}
⟨latexrelease⟩                 {Hooks~with~args}
\prg_new_protected_conditional:Npnn
    \__hook_try_declaring_generic_hook:wn
    #1 / #2 / #3 / #4 \scan_stop: #5 { TF }
  {
    \__hook_if_generic:nTF {#5}
      {
        \__hook_if_usable:nF {#5}
          {
```

If the hook doesn't exist yet we check if it is a `cmd` hook and if so we attempt patching the command in addition to declaring the hook.

For some commands this will not be possible, in which case `\__hook_patch_cmd_or_delay:Nnn` (defined in `ltcmdhooks`) will generate an appropriate error message.

```
            \str_if_eq:nnT {#1} { cmd }
              {
                \__hook_try_put_cmd_hook:n {#5}
                \__hook_make_usable:nn {#5} { 9 }
                \use_none:nnn
              }
```

Declare the hook always even if it can't really be used (error message generated elsewhere).

Here we use `\__hook_make_usable:nn`, so that a `\hook_new:n` is still possible later. Generic hooks (except `cmd` hooks) take no arguments, so use zero as the second argument.

```
            \__hook_make_usable:nn {#5} { 0 }
          }
        \__hook_if_generic_reversed:nT {#5}
          { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
        \prg_return_true:
      }
      {
```

Generic hooks are all named ⟨*type*⟩/⟨*name*⟩/⟨*place*⟩, where ⟨*type*⟩ and ⟨*place*⟩ are predefined (`\c__hook_generic_`⟨*type*⟩`/./`⟨*place*⟩`_tl`), and ⟨*name*⟩ is the variable component. Older releases had some hooks with the ⟨*name*⟩ in the third part, so the code below supports that syntax for a while, with a warning.

The `\exp_after:wN ... \exp:w` trick is there to remove the conditional structure inserted by `\__hook_try_declaring_generic_hook:wnTF` and thus allow access to the tokens that follow it, as is needed to keep things going.

When the deprecation cycle ends, the lines below should all be replaced by `\prg_return_false:`.

```
        \__hook_if_deprecated_generic:nTF {#5}
          {
            \__hook_deprecated_generic_warn:n {#5}
            \exp_after:wN \__hook_declare_deprecated_generic:NNn
            \exp:w % \exp_end:
          }
          { \prg_return_false: }
      }
  }
```

`\__hook_deprecated_generic_warn:n` will issue a deprecation warning for a given hook, and mark that hook such that the warning will not be issued again (multiple warnings can be issued, but only once per hook).

__hook_deprecated_generic_warn:Nn
__hook_deprecated_generic_warn:Nw

```
\cs_new_protected:Npn \__hook_deprecated_generic_warn:n #1
  { \__hook_deprecated_generic_warn:w #1 \s__hook_mark }
\cs_new_protected:Npn \__hook_deprecated_generic_warn:w
    #1 / #2 / #3 \s__hook_mark
  {
    \if_cs_exist:w __hook~#1/#2/#3 \cs_end: \else:
      \msg_warning:nnnnn { hooks } { generic-deprecated } {#1} {#2} {#3}
    \fi:
```

```
    \cs_gset_eq:cN { __hook~#1/#2/#3 } \scan_stop:
  }
```

Now that the user has been told about the deprecation, we proceed by swapping ⟨*name*⟩ and ⟨*place*⟩ and adding the code to the correct hook.

__hook_do_deprecated_generic:Nn
__hook_do_deprecated_generic:Nw
__hook_declare_deprecated_generic:NNw
__hook_declare_deprecated_generic:NNw

```
\cs_new_protected:Npn \__hook_do_deprecated_generic:Nn #1 #2
  { \__hook_do_deprecated_generic:Nw #1 #2 \s__hook_mark }
\cs_new_protected:Npn \__hook_do_deprecated_generic:Nw #1
         #2 / #3 / #4 \s__hook_mark
  { #1 { #2 / #4 / #3 } }
\cs_new_protected:Npn \__hook_declare_deprecated_generic:NNn #1 #2 #3
  { \__hook_declare_deprecated_generic:NNw #1 #2 #3 \s__hook_mark }
\cs_new_protected:Npn \__hook_declare_deprecated_generic:NNw #1 #2
    #3 / #4 / #5 \s__hook_mark
  {
    \__hook_try_declaring_generic_hook:wnTF #3 / #5 / #4 / \scan_stop:
        { #3 / #5 / #4 }
      #1 #2 { #3 / #5 / #4 }
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/11/15}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:wn}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\prg_new_protected_conditional:Npnn
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wn
⟨latexrelease⟩    #1 / #2 / #3 / #4 \scan_stop: #5 { TF }
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_generic:nTF {#5}
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_if_usable:nF {#5}
⟨latexrelease⟩          {
⟨latexrelease⟩            \str_if_eq:nnT {#1} { cmd }
⟨latexrelease⟩              { \__hook_try_put_cmd_hook:n {#5} }
⟨latexrelease⟩            \__hook_make_usable:n {#5}
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_if_generic_reversed:nT {#5}
⟨latexrelease⟩          { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
⟨latexrelease⟩        \prg_return_true:
⟨latexrelease⟩      }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_if_deprecated_generic:nTF {#5}
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_deprecated_generic_warn:n {#5}
⟨latexrelease⟩            \exp_after:wN \__hook_declare_deprecated_generic:NNn
⟨latexrelease⟩            \exp:w % \exp_end:
⟨latexrelease⟩          }
⟨latexrelease⟩          { \prg_return_false: }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/06/01}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:wn}
⟨latexrelease⟩                  {Support~cmd~hooks}
```

```
⟨latexrelease⟩\prg_new_protected_conditional:Npnn
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wn
⟨latexrelease⟩    #1 / #2 / #3 / #4 \scan_stop: #5 { TF }
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_empty:nTF {#2}
⟨latexrelease⟩      { \prg_return_false: }
⟨latexrelease⟩      {
⟨latexrelease⟩        \prop_if_in:NnTF \c__hook_generics_prop {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_if_usable:nF {#5}
⟨latexrelease⟩              {
⟨latexrelease⟩                \str_if_eq:nnT {#1} { cmd }
⟨latexrelease⟩                  { \__hook_try_put_cmd_hook:n {#5} }
⟨latexrelease⟩                \__hook_make_usable:n {#5}
⟨latexrelease⟩              }
⟨latexrelease⟩            \prop_if_in:NnTF
⟨latexrelease⟩              \c__hook_generics_reversed_ii_prop {#2}
⟨latexrelease⟩              { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
⟨latexrelease⟩              {
⟨latexrelease⟩                \prop_if_in:NnT
⟨latexrelease⟩                  \c__hook_generics_reversed_iii_prop {#3}
⟨latexrelease⟩                  { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
⟨latexrelease⟩              }
⟨latexrelease⟩            \prg_return_true:
⟨latexrelease⟩          }
⟨latexrelease⟩          { \prg_return_false: }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}
⟨latexrelease⟩                 {\__hook_try_declaring_generic_hook:wn}
⟨latexrelease⟩                 {Support~cmd~hooks}
⟨latexrelease⟩\prg_new_protected_conditional:Npnn
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wn
⟨latexrelease⟩    #1 / #2 / #3 / #4 \scan_stop: #5 { TF }
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_empty:nTF {#2}
⟨latexrelease⟩      { \prg_return_false: }
⟨latexrelease⟩      {
⟨latexrelease⟩        \prop_if_in:NnTF \c__hook_generics_prop {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_if_declared:nF {#5} { \hook_new:n {#5} }
⟨latexrelease⟩            \prop_if_in:NnTF
⟨latexrelease⟩              \c__hook_generics_reversed_ii_prop {#2}
⟨latexrelease⟩              { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
⟨latexrelease⟩              {
⟨latexrelease⟩                \prop_if_in:NnT
⟨latexrelease⟩                  \c__hook_generics_reversed_iii_prop {#3}
⟨latexrelease⟩                  { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
⟨latexrelease⟩              }
⟨latexrelease⟩            \prg_return_true:
⟨latexrelease⟩          }
⟨latexrelease⟩          { \prg_return_false: }
⟨latexrelease⟩      }
```

```
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_try_declaring_generic_hook:wnTF` *and others.*)

`\__hook_if_file_hook_p:w`
`\__hook_if_file_hook:wTF`

`\__hook_if_file_hook:wTF` checks if the argument is a valid file-specific hook (not, for example, `file/before`, but `file/foo.tex/before`). If it is a file-specific hook, then it executes the ⟨*true*⟩ branch, otherwise ⟨*false*⟩.

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__hook_if_file_hook:w}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_if_file_hook:w}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\prg_new_conditional:Npnn \__hook_if_file_hook:w
⟨latexrelease⟩    #1 / #2 / #3 \s__hook_mark { TF }
⟨latexrelease⟩  {
⟨latexrelease⟩    \str_if_eq:nnTF {#1} { file }
⟨latexrelease⟩      {
⟨latexrelease⟩        \bool_lazy_or:nnTF
⟨latexrelease⟩            { \tl_if_empty_p:n {#3} }
⟨latexrelease⟩            { \str_if_eq_p:nn {#3} { / } }
⟨latexrelease⟩          { \prg_return_false: }
⟨latexrelease⟩          {
⟨latexrelease⟩            \prop_if_in:NnTF \c__hook_generics_file_prop {#2}
⟨latexrelease⟩              { \prg_return_true: }
⟨latexrelease⟩              { \prg_return_false: }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩      { \prg_return_false: }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_if_file_hook:wTF`.)

`\__hook_file_hook_normalize:n`
`\__hook_strip_double_slash:n`
`\__hook_strip_double_slash:w`

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__hook_file_hook_normalize:n}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\EndIncludeInRelease
```

When a file-specific hook is found, before being declared it is lightly normalized by `\__hook_file_hook_normalize:n`. The current implementation just replaces two consecutive slashes (`//`) by a single one, to cope with simple cases where the user did something like `\def\input@path{{./mypath/}}`, in which case a hook would have to be `\AddToHook{file/./mypath//file.tex/after}`.

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_file_hook_normalize:n}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new:Npn \__hook_file_hook_normalize:n #1
⟨latexrelease⟩  { \__hook_strip_double_slash:n {#1} }
⟨latexrelease⟩\cs_new:Npn \__hook_strip_double_slash:n #1
⟨latexrelease⟩  { \__hook_strip_double_slash:w #1 // \s__hook_mark }
```

This function is always called after testing if the argument is a file hook with `\__hook_if_file_hook:wTF`, so we can assume it has three parts (it is either `file/.../before` or `file/.../after`), so we use `#1/#2/#3 //` instead of just `#1 //` to prevent losing a slash if the file name is empty.

```
⟨latexrelease⟩\cs_new:Npn \__hook_strip_double_slash:w #1/#2/#3//#4\s__hook_mark
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_empty:nTF {#4}
⟨latexrelease⟩      { #1/#2/#3 }
⟨latexrelease⟩      { \__hook_strip_double_slash:w #1/#2/#3 /#4\s__hook_mark }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_file_hook_normalize:n`, `\__hook_strip_double_slash:n`, *and* `\__hook_strip_double_slash:w`.)

`\c__hook_generic_cmd/./before_tl`
`\c__hook_generic_cmd/./after_tl`
`\c__hook_generic_env/./before_tl`
`\c__hook_generic_env/./after_tl`
`\c__hook_generic_file/./before_tl`
`\c__hook_generic_file/./after_tl`
`\c__hook_generic_package/./before_tl`
`\c__hook_generic_package/./after_tl`
`\c__hook_generic_class/./before_tl`
`\c__hook_generic_class/./after_tl`
`\c__hook_generic_include/./before_tl`
`\c__hook_generic_include/./after_tl`
`\c__hook_generic_env/./begin_tl`
`\c__hook_generic_env/./end_tl`
`\c__hook_generic_include/./end_tl`

Token lists defining the possible generic hooks. We don't provide any user interface to this as this is meant to be static.

`cmd` The generic hooks used for commands.

`env` The generic hooks used in `\begin` and `\end`.

`file`, `package`, `class`, `include` The generic hooks used when loading a file

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\c__hook_generics_prop}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
\clist_map_inline:nn { cmd , env , file , package , class , include }
  {
    \tl_const:cn { c__hook_generic_#1/./before_tl } { + }
    \tl_const:cn { c__hook_generic_#1/./after_tl  } { - }
  }
\tl_const:cn { c__hook_generic_env/./begin_tl } { + }
\tl_const:cn { c__hook_generic_env/./end_tl   } { + }

\tl_const:cn { c__hook_generic_include/./end_tl } { - }
\tl_const:cn { c__hook_generic_include/./excluded_tl } { + }
```

Deprecated generic hooks:

```
\clist_map_inline:nn { file , package , class , include }
  {
    \tl_const:cn { c__hook_deprecated_#1/./before_tl } { }
    \tl_const:cn { c__hook_deprecated_#1/./after_tl  } { }
  }
\tl_const:cn { c__hook_deprecated_include/./end_tl } { }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\c__hook_generics_prop}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\prop_const_from_keyval:Nn \c__hook_generics_prop
⟨latexrelease⟩     {cmd=,env=,file=,package=,class=,include=}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\c__hook_generic_cmd/./before_tl` *and others.*)

`\c__hook_generics_reversed_ii_prop`
`\c__hook_generics_reversed_iii_prop`
`\c__hook_generics_file_prop`

The following generic hooks are supposed to use reverse ordering (the `ii` and `iii` names are kept for the deprecation cycle):

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\c__hook_generics_reversed_ii_prop}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\c__hook_generics_reversed_ii_prop}
⟨latexrelease⟩                    {Standardize~generic~hook~names}
⟨latexrelease⟩\prop_const_from_keyval:Nn
⟨latexrelease⟩     \c__hook_generics_reversed_ii_prop {after=,end=}
⟨latexrelease⟩\prop_const_from_keyval:Nn
⟨latexrelease⟩     \c__hook_generics_reversed_iii_prop {after=}
⟨latexrelease⟩\prop_const_from_keyval:Nn
⟨latexrelease⟩     \c__hook_generics_file_prop {before=,after=}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\c__hook_generics_reversed_ii_prop`, `\c__hook_generics_reversed_iii_prop`, *and* `\c__hook_generics_file_prop`.)

`\c__hook_parameter_cmd/./before_tl`
`\c__hook_parameter_cmd/./after_tl`

Token lists defining the number of arguments for a given type of generic hook.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\c__hook_parameter_cmd/./before_tl}
⟨latexrelease⟩                    {Hooks~with~args}
```

`cmd` hooks are declared with 9 arguments because they have a variable number of arguments (depending on the command they are attached to), so we use the maximum here.

```
\tl_const:cn { c__hook_parameter_cmd/./before_tl } { #1#2#3#4#5#6#7#8#9 }
\tl_const:cn { c__hook_parameter_cmd/./after_tl }  { #1#2#3#4#5#6#7#8#9 }

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\c__hook_parameter_cmd/./before_tl}
⟨latexrelease⟩                    {Hooks~with~args}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\c__hook_parameter_cmd/./before_tl` *and* `\c__hook_parameter_cmd/./after_tl`.)

`\hook_gremove_code:nn`
`\__hook_gremove_code:nn`

With `\hook_gremove_code:nn{`⟨*hook*⟩`}{`⟨*label*⟩`}` any code for ⟨*hook*⟩ stored under ⟨*label*⟩ is removed.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_gremove_code:nn}
⟨latexrelease⟩                    {Hooks~with~args}
\cs_new_protected:Npn \hook_gremove_code:nn #1 #2
  { \__hook_normalize_hook_args:Nnn \__hook_gremove_code:nn {#1} {#2} }
\cs_new_protected:Npn \__hook_gremove_code:nn #1 #2
  {
```

First check that the hook code pool exists. `\__hook_if_usable:nTF` isn't used here because it should be possible to remove code from a hook before its defined (see section 2.1.8).

```
    \__hook_if_structure_exist:nTF {#1}
      {
```

Then remove the chunk and run `\__hook_update_hook_code:n` so that the execution token list reflects the change if we are after `\begin{document}`.

If all code is to be removed, clear the code pool `\g__hook_`⟨*hook*⟩`_code_prop`, the top-level code `\__hook_toplevel_`␣⟨*hook*⟩, and the next-execution code `\__hook_next_`␣⟨*hook*⟩.

```
        \str_if_eq:nnTF {#2} {*}
          {
            \prop_gclear:c { g__hook_#1_code_prop }
            \__hook_toplevel_gset:nn {#1} { }
            \__hook_next_gset:nn {#1} { }
```

```
          }
          {
```

If the label is `top-level` then clear the token list, as all code there is under the same label.

```
            \str_if_eq:nnTF {#2} { top-level }
              { \__hook_toplevel_gset:nn {#1} { } }
              {
                \prop_gpop:cnNF { g__hook_#1_code_prop }
                  {#2} \l__hook_return_tl
                  { \msg_warning:nnnn { hooks } { cannot-remove } {#1} {#2} }
              }
          }
```

Finally update the code, if the hook exists.

```
        \__hook_if_usable:nT {#1}
          { \__hook_update_hook_code:n {#1} }
      }
```

If the code pool for this hook doesn't exist, show a warning:

```
      {
        \__hook_if_deprecated_generic:nTF {#1}
          {
            \__hook_deprecated_generic_warn:n {#1}
            \__hook_do_deprecated_generic:Nn
                \__hook_gremove_code:nn {#1} {#2}
          }
          { \msg_warning:nnnn { hooks } { cannot-remove } {#1} {#2} }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_gremove_code:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_gremove_code:nn #1 #2
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_structure_exist:nTF {#1}
⟨latexrelease⟩      {
⟨latexrelease⟩        \str_if_eq:nnTF {#2} {*}
⟨latexrelease⟩          {
⟨latexrelease⟩            \prop_gclear:c { g__hook_#1_code_prop }
⟨latexrelease⟩            \__hook_tl_gclear:c { __hook_toplevel~#1 }
⟨latexrelease⟩            \__hook_tl_gclear:c { __hook_next~#1 }
⟨latexrelease⟩          }
⟨latexrelease⟩          {
⟨latexrelease⟩            \str_if_eq:nnTF {#2} { top-level }
⟨latexrelease⟩              { \__hook_tl_gclear:c { __hook_toplevel~#1 } }
⟨latexrelease⟩              {
⟨latexrelease⟩                \prop_gpop:cnNF { g__hook_#1_code_prop }
⟨latexrelease⟩                  {#2} \l__hook_return_tl
⟨latexrelease⟩                  { \msg_warning:nnnn { hooks } { cannot-remove }
⟨latexrelease⟩                                      {#1} {#2} }
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_if_usable:nT {#1}
⟨latexrelease⟩          { \__hook_update_hook_code:n {#1} }
```

```
⟨latexrelease⟩      }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_if_deprecated_generic:nTF {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_deprecated_generic_warn:n {#1}
⟨latexrelease⟩            \__hook_do_deprecated_generic:Nn
⟨latexrelease⟩                \__hook_gremove_code:nn {#1} {#2}
⟨latexrelease⟩          }
⟨latexrelease⟩          { \msg_warning:nnnn { hooks } { cannot-remove }
⟨latexrelease⟩                              {#1} {#2} }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_gremove_code:nn` *and* `\__hook_gremove_code:nn`*. This function is documented on page* 17.)

`\__hook_cs_gput_right:nnn`
`\__hook_cs_gput_right_fast:nnn`
`\__hook_cs_gput_right_slow:nnn`
`\__hook_code_gset_auxi:nnnn`
`\__hook_code_gset_auxi:eeen`

This macro is used to append code to the `toplevel` and `next` token lists, treating them correctly depending on their number of arguments, and depending on whether the code being added should have parameter tokens understood as parameters, or doubled to be stored as parameter tokens.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_cs_gput_right:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
```

Check if the current hook is declared and takes no arguments. In this case, we short-circuit and use the simpler and much faster approach that doesn't require hash-doubling.

```
\cs_new_protected:Npn \__hook_cs_gput_right:nnn #1 #2
  {
    \if:w T
        \__hook_if_declared:nF {#2} { F }
        \tl_if_empty:cF { c__hook_#2_parameter_tl } { F }
          T
      \exp_after:wN \__hook_cs_gput_right_fast:nnn
    \else:
      \exp_after:wN \__hook_cs_gput_right_slow:nnn
    \fi:
        {#1} {#2}
  }
\cs_new_protected:Npn \__hook_cs_gput_right_fast:nnn #1 #2 #3
  { \cs_gset:cpe { __hook#1~#2 }
      { \exp_not:v { __hook#1~#2 } \exp_not:n {#3} } }
\cs_new_protected:Npn \__hook_cs_gput_right_slow:nnn #1 #2 #3
  {
```

The auxiliary `\__hook_code_gset_auxi:eeen` just does the assignment at the end. Its first argument is the parameter text of the macro, which is chosen here depending if `\c__hook_`⟨*hook*⟩`_parameter_tl` exists, if the hook is declared, and if it's a generic hook.

```
    \cs_if_exist:cF { __hook#1~#2 }
      { \__hook_code_gset_aux:nnn {#1} {#2} { } }
    \__hook_code_gset_auxi:eeen
      {
        \__hook_if_declared:nTF {#2}
          { \tl_use:c { c__hook_#2_parameter_tl } }
          {
            \__hook_if_generic:nTF {#2}
```

```
            { \__hook_generic_parameter:n {#2} }
            { \c__hook_nine_parameters_tl }
        }
    }
```

Here we take the existing code in the macro, expand it with as many arguments as it takes, then double the hashes so the code can be reused.

PhO: Maybe can be improved. The case of adding to an empty cs can be optimized by quickly checking `\cs_replacement_spec`.

```
    {
      \exp_args:NNo \exp_args:No \__hook_double_hashes:n
        {
          \cs:w __hook#1~#2 \exp_last_unbraced:Ne \cs_end:
            { \__hook_braced_cs_parameter:n { __hook#1~#2 } }
        }
    }
```

Now the new code: if we are replacing arguments, then hashes are left untouched, otherwise they are doubled.

```
    {
      \__hook_if_replacing_args:TF
        { \exp_not:n }
        { \__hook_double_hashes:n }
          {#3}
    }
```

And finally, the csname which we'll define with all the above.

```
    { __hook#1~#2 }
}
```

And as promised, the auxiliary that does the definition.

```
\cs_new_protected:Npn \__hook_code_gset_auxi:nnnn #1 #2 #3 #4
  { \cs_gset:cpn {#4} #1 { #2 #3 } }
\cs_generate_variant:Nn \__hook_code_gset_auxi:nnnn { eeen }

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_cs_gput_right:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_cs_gput_right:nnn
⟨latexrelease⟩\cs_undefine:N \__hook_cs_gput_right_fast:nnn
⟨latexrelease⟩\cs_undefine:N \__hook_cs_gput_right_slow:nnn
⟨latexrelease⟩\cs_undefine:N \__hook_code_gset_auxi:nnnn
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_cs_gput_right:nnn` *and others.*)

`\__hook_code_gset:nn`
`\__hook_code_gset:ne`
`\__hook_toplevel_gset:nn`
`\__hook_next_gset:nn`
`\__hook_code_gset_aux:nnn`

These macros define `\__hook`⟨*type*⟩`_`⟨*hook*⟩ (with ⟨*type*⟩ being `_next`, `_toplevel`, or empty) with the given code and the parameters stored in `\c__hook_`⟨*hook*⟩`_parameter_tl` (or none, if that doesn't exist).

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_code_gset:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_code_gset:nn
  { \__hook_code_gset_aux:nnn { } }
\cs_new_protected:Npn \__hook_toplevel_gset:nn
  { \__hook_code_gset_aux:nnn { _toplevel } }
\cs_new_protected:Npn \__hook_next_gset:nn
  { \__hook_code_gset_aux:nnn { _next } }
\cs_new_protected:Npn \__hook_code_gset_aux:nnn #1 #2 #3
```

```
  {
    \cs_gset:cpn { __hook#1~#2 \exp_last_unbraced:Ne }
      { \__hook_parameter:n {#2} }
      {#3}
  }
\cs_generate_variant:Nn \__hook_code_gset:nn { ne }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_code_gset:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_code_gset:nn
⟨latexrelease⟩\cs_undefine:N \__hook_toplevel_gset:nn
⟨latexrelease⟩\cs_undefine:N \__hook_next_gset:nn
⟨latexrelease⟩\cs_undefine:N \__hook_code_gset_aux:nnn
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_code_gset:nn` *and others.*)

`\__hook_normalize_cs_args:nn`

This macro normalizes the parameters of the macros `\__hook`⟨*type*⟩`_`⟨*hook*⟩ to take the right number of arguments after a hook is declared. At this point we know `\c__hook_`⟨*hook*⟩`_parameter_tl` exists, so use that to count the arguments and use that as ⟨*parameter text*⟩ for the newly (re)defined macro.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_normalize_cs_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_normalize_cs_args:nn #1 #2
  {
    \cs_if_exist:cT { __hook#1~#2 }
      {
        \__hook_code_gset_auxi:eeen
          { \tl_use:c { c__hook_#2_parameter_tl } }
          {
            \exp_args:NNo \exp_args:No \__hook_double_hashes:n
              {
                \cs:w __hook#1~#2 \exp_last_unbraced:Ne \cs_end:
                  { \__hook_braced_cs_parameter:n { __hook#1~#2 } }
              }
          }
          { }
          { __hook#1~#2 }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_normalize_cs_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_normalize_cs_args:nn
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_normalize_cs_args:nn`.)

`\__hook_normalize_code_pool:n`
`\__hook_set_normalize_fn:nn`

This one's a bit of a hack. It takes a hook, and iterates over its code pool (`\g__hook_`⟨*hook*⟩`_code_prop`), redefining each code label to use only valid arguments. This is used when, for example, a code is added referencing arguments `#1` and `#2`, but the hook has only `#1`. In this example, every reference to `#2` is changed to `##2`. This is done because

otherwise TEX will throw a low-level error every time some change happens to the hook (code is added, a rule is set, etc), which can get quite repetitive for no good reason.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_normalize_code_pool:n}
⟨latexrelease⟩                    {Hooks~with~args}
\cs_new_protected:Npn \__hook_normalize_code_pool:n #1
  {
```

First, call `\__hook_set_normalize_fn:nn` with the hook name to set everything up, then we'll loop over the hook's code pool applying the normalization above. After that's done, copy the temporary property list back to the hook's.

```
    \__hook_set_normalize_fn:nn {#1} { Offending~label:~'##1' }
    \prop_clear:N \l__hook_work_prop
    \prop_map_function:cN { g__hook_#1_code_prop } \__hook_normalize_fn:nn
    \prop_gset_eq:cN { g__hook_#1_code_prop } \l__hook_work_prop
  }
```

The sole purpose of this function is to define `\__hook_normalize_fn:nn`, which will then do the correcting of the code being added to the hook.

```
\cs_new_protected:Npn \__hook_set_normalize_fn:nn #1 #2
  {
```

To start, we define two auxiliary token lists. `\l__hook_tmpb_tl` contains:

```
{\c__hook_hashes_tl 1}
{\c__hook_hashes_tl 2}
...
{\c__hook_hashes_tl 9}
```

```
    \cs_set:Npn \__hook_tmp:w ##1##2##3##4##5##6##7##8##9 { }
    \tl_set:Ne \l__hook_tmpb_tl
      { \__hook_braced_cs_parameter:n { __hook_tmp:w } }
    \group_begin:
      \__hook_tl_set:cn { c__hook_hash_tl } { \exp_not:N \c__hook_hashes_tl }
      \use:e
        {
    \group_end:
    \tl_set:Nn \exp_not:N \l__hook_tmpb_tl { \l__hook_tmpb_tl }
        }
```

And `\l__hook_tmpa_tl` contains:

```
{\c__hook_hash_tl 1}
{\c__hook_hash_tl 2}
...
{\c__hook_hash_tl <n>}
```

with ⟨*n*⟩ being the number of arguments declared for the hook.

```
    \exp_last_unbraced:NNf
    \cs_set:Npn \__hook_tmp:w { \__hook_parameter:n {#1} } { }
    \tl_set:Ne \l__hook_tmpa_tl
      { \__hook_braced_cs_parameter:n { __hook_tmp:w } }
```

Now this function does the fun part. It is meant to be used with `\prop_map_function:NN`, taking a label name in `##1` and the code stored in that label in `##2`.

```
    \cs_gset_protected:Npe \__hook_normalize_fn:nn ##1 ##2
      {
```

Here we'll define two auxiliary macros: the first one throws an error when it detects an invalid argument reference. It piggybacks on TEX's low-level "Illegal parameter number" error, but it defines a weirdly-named control sequence so that the error comes out nicely formatted. For example, if the label "badpkg" adds some code that references argument `#3` in the hook "foo", which takes only two arguments, the error will be:

```
! Illegal parameter number in definition of hook 'foo'.
(hooks)              Offending label: 'badpkg'.
<to be read again>
                   3
```

At the point of this definition, the error is raised if the code happens to reference an invalid argument. If it was possible to detect that this definition raised no error, the next step would be unnecessary. We'll do all this in a group so this weird definition doesn't leak out, and set `\tex_escapechar:D` to -1 so this hack shows up extra nice in the case of an error.

```
\group_begin:
  \int_set:Nn \tex_escapechar:D { -1 }
  \cs_set:cpn
      {
        hook~'#1'. ^^J
        (hooks) \prg_replicate:nn { 13 } { ~ }
        #2 % more message text
      }
      \exp_not:v { c__hook_#1_parameter_tl }
    {##2}
\group_end:
```

This next macro, with a much less fabulous name, takes always nine arguments, and it just transfers the code `##2` under the label `##1` to the temporary property list. The first $\langle n \rangle$ arguments are taken from `\l__hook_tmpa_tl`, and the other $9-\langle n \rangle$ taken from `\l__hook_tmpb_tl` (which contains twice as many `#` tokens as the former). Then, `\__hook_double_hashes:n` is used to double non-argument hashes, and expand the `\c__hook_hash_tl` and `\c__hook_hashes_tl` to the actual parameter tokens.

```
\cs_set:Npn \exp_not:N \__hook_tmp:w
    \exp_not:V \c__hook_nine_parameters_tl
  {
    \prop_put:Nne \exp_not:N \l__hook_work_prop
      {##1} { \exp_not:N \__hook_double_hashes:n {##2} }
  }
```

This next macro, with a much less fabulous name, takes always nine arguments, and it just transfers the code `##2` under the label `##1` to the temporary property list. The first $\langle n \rangle$ arguments are taken from `\l__hook_tmpa_tl`, and the other $9-\langle n \rangle$ taken from `\l__hook_tmpb_tl` (which contains twice as many `#` tokens as the former). Then, `\__hook_double_hashes:n` is used to double non-argument hashes, and expand the `\c__hook_hash_tl` and `\c__hook_hashes_tl` to the actual parameter tokens.

```
\exp_not:N \__hook_tmp:w
  \exp_not:V \l__hook_tmpa_tl
  \exp_args:No \exp_not:o
    { \exp_after:wN \__hook_tmp:w \l__hook_tmpb_tl }
  }
}
```

```
\cs_new_eq:NN \__hook_normalize_fn:nn ?
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_normalize_code_pool:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_normalize_code_pool:n
⟨latexrelease⟩\EndIncludeInRelease
```

Check if the expansion of a control sequence is empty by looking at its replacement text.

__hook_cs_if_empty_p:c
__hook_cs_if_empty:cTF

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_cs_if_empty:c}
⟨latexrelease⟩                  {Hooks~with~args}
\prg_new_conditional:Npnn \__hook_cs_if_empty:c #1 { p, T, F, TF }
  {
    \if:w \scan_stop: \__hook_replacement_spec:c {#1} \scan_stop:
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
\cs_new:Npn \__hook_replacement_spec:c #1
  {
    \exp_args:Nc \token_if_macro:NT {#1}
      { \cs_replacement_spec:c {#1} }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_cs_if_empty:c}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_cs_if_empty:c
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_normalize_code_pool:n`, `\__hook_set_normalize_fn:nn`, *and* `\__hook_cs_if_empty:cTF`.)

__hook_braced_cs_parameter:n
__hook_braced_hidden_loop:w
__hook_cs_parameter_count:N
__hook_cs_parameter_count:w
__hook_cs_end:w

Looks at the ⟨*parameter text*⟩ of a control sequence, and returns a run of "hidden" braced parameters for that macro. This works as long as the macros take a simple run of zero to nine arguments. The parameters are "hidden" because the parameter tokens are returned inside `\c__hook_hash_tl` instead of explicitly, so that `\__hook_double_hashes:n` won't touch these.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_braced_cs_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new:Npn \__hook_braced_cs_parameter:n #1
  {
    \exp_last_unbraced:Ne \__hook_braced_hidden_loop:w
      { \exp_args:Nc \__hook_cs_parameter_count:N {#1} } ? \s__hook_mark
  }
\cs_new:Npn \__hook_braced_hidden_loop:w #1
  {
    \if:w ? #1
      \__hook_use_i_delimit_by_s_mark:nw
    \fi:
    { \exp_not:N \c__hook_hash_tl #1 }
    \__hook_braced_hidden_loop:w
  }
```

```
\cs_new:Npn \__hook_cs_parameter_count:N #1
  {
    \exp_last_unbraced:Nf \__hook_cs_parameter_count:w
      { \token_if_macro:NT #1 { \cs_parameter_spec:N #1 } }
      ? \__hook_cs_end:w ? \__hook_cs_end:w ? \__hook_cs_end:w
      ? \__hook_cs_end:w ? \__hook_cs_end:w ? \__hook_cs_end:w
      ? \__hook_cs_end:w ? \__hook_cs_end:w ? \__hook_cs_end:w
      \s__hook_mark
  }
\cs_new:Npn \__hook_cs_parameter_count:w #1#2 #3#4 #5#6 #7#8
  { #2 #4 #6 #8 \__hook_cs_parameter_count:w }
\cs_new:Npn \__hook_cs_end:w #1 \s__hook_mark { }
⟨latexrelease⟩\EndIncludeInRelease
```

This function can't be undefined when rolling back because it's used at the end of this module to adequate the hook data structures to previous versions.

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_braced_cs_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_braced_cs_parameter:n` *and others.*)

`\__hook_braced_parameter:n`
`\__hook_braced_real_loop:w`

This one is used in simpler cases, where no special handling of hashes is required. This is used only inside `\__hook_initialize_hook_code:n`, so it assumes `\c__hook_⟨hook⟩_parameter_tl` is defined, but should work otherwise.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_braced_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new:Npn \__hook_braced_parameter:n #1
  {
    \if_case:w
      \int_eval:n
        { \exp_args:Nv \str_count:n { c__hook_#1_parameter_tl } / 3 }
      \exp_stop_f:
    \or: {##1}
    \or: {##1} {##2}
    \or: {##1} {##2} {##3}
    \or: {##1} {##2} {##3} {##4}
    \or: {##1} {##2} {##3} {##4} {##5}
    \or: {##1} {##2} {##3} {##4} {##5} {##6}
    \or: {##1} {##2} {##3} {##4} {##5} {##6} {##7}
    \or: {##1} {##2} {##3} {##4} {##5} {##6} {##7} {##8}
    \or: {##1} {##2} {##3} {##4} {##5} {##6} {##7} {##8} {##9}
    \else:
      \msg_expandable_error:nnn { latex2e } { should-not-happen }
        { Invalid~parameter~spec. }
    \fi:
  }
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_braced_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_braced_parameter:n
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_braced_parameter:n` *and* `\__hook_braced_real_loop:w`.)

__hook_parameter:n This is just a shortcut to e- or f-expand to the ⟨*parameter text*⟩ of the hook.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new:Npn \__hook_parameter:n #1
  {
    \cs:w c__hook_
    \tl_if_exist:cTF { c__hook_#1_parameter_tl }
      { #1_parameter } { empty }
    _tl \cs_end:
  }
\cs_new:Npn \__hook_generic_parameter:n #1
  { \__hook_generic_parameter:w #1 / / / \s__hook_mark }
\cs_new:Npn \__hook_generic_parameter:w #1 / #2 / #3 / #4 \s__hook_mark
  {
    \cs_if_exist_use:cF { c__hook_parameter_#1/./#3_tl }
      { \c__hook_empty_tl }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_parameter:n
⟨latexrelease⟩\cs_undefine:N \__hook_generic_parameter:n
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_parameter:n`.)

4.7 Setting rules for hooks code

\g__hook_??_code_prop
__hook~??
\g__hook_??_reversed_tl
\c__hook_??_parameter_tl

Initially these variables simply used an empty "label" name (not two question marks). This was a bit unfortunate, because then `l3doc` complains about `__` in the middle of a command name when trying to typeset the documentation. However using a "normal" name such as `default` has the disadvantage of that being not really distinguishable from a real hook name. I now have settled for `??` which needs some gymnastics to get it into the csname, but since this is used a lot, the code should be fast, so this is not done with `c` expansion in the code later on.

`\__hook␣??` isn't used, but it has to be defined to trick the code into thinking that `??` is actually a hook.

```
\prop_new:c { g__hook_??_code_prop }
\prop_new:c { __hook~?? }
```

Default rules are always given in normal ordering (never in reversed ordering). If such a rule is applied to a reversed hook it behaves as if the rule is reversed (e.g., `after` becomes `before`) because those rules are applied first and then the order is reversed.

```
\tl_new:c { g__hook_??_reversed_tl }
```

The parameter text for the "default" hook is empty.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\c__hook_??_parameter_tl}
⟨latexrelease⟩                  {Hooks~with~args}
\tl_const:cn { c__hook_??_parameter_tl } { }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\c__hook_??_parameter_tl}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:c { c__hook_??_parameter_tl }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\g__hook_??_code_prop` *and others.*)

`\hook_gset_rule:nnnn`
`\__hook_gset_rule:nnnn`

With `\hook_gset_rule:nnnn{`⟨*hook*⟩`}{`⟨*label1*⟩`}{`⟨*relation*⟩`}{`⟨*label2*⟩`}` a relation is defined between the two code labels for the given ⟨***hook***⟩. The special hook `??` stands for *any* hook, which sets a default rule (to be used if no other relation between the two hooks exist).

```
\cs_new_protected:Npn \hook_gset_rule:nnnn #1#2#3#4
  {
    \__hook_normalize_hook_rule_args:Nnnnn \__hook_gset_rule:nnnn
      {#1} {#2} {#3} {#4}
  }
⟨latexrelease⟩\IncludeInRelease{2022/06/01}{\__hook_gset_rule:nnnn}
⟨latexrelease⟩                  {Refuse~setting~rule~for~one-time~hooks}
\cs_new_protected:Npn \__hook_gset_rule:nnnn #1#2#3#4
  {
    \__hook_if_deprecated_generic:nT {#1}
      {
        \__hook_deprecated_generic_warn:n {#1}
        \__hook_do_deprecated_generic:Nn \__hook_gset_rule:nnnn {#1}
          {#2} {#3} {#4}
        \__hook_use_none_delimit_by_s_mark:w
      }
    \__hook_if_execute_immediately:nT {#1}
      {
        \msg_error:nnnnnn { hooks } { rule-too-late }
          {#1} {#2} {#3} {#4}
        \__hook_use_none_delimit_by_s_mark:w
      }
```

First we ensure the basic data structure of the hook exists:

```
    \__hook_init_structure:n {#1}
```

Then we clear any previous relationship between both labels.

```
    \__hook_rule_gclear:nnn {#1} {#2} {#4}
```

Then we call the function to handle the given rule. Throw an error if the rule is invalid.

```
    \cs_if_exist_use:cTF { __hook_rule_#3_gset:nnn }
      {
          {#1} {#2} {#4}
        \__hook_update_hook_code:n {#1}
      }
      {
        \msg_error:nnnnnn { hooks } { unknown-rule }
          {#1} {#2} {#3} {#4}
      }
    \s__hook_mark
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_gset_rule:nnnn}
⟨latexrelease⟩                  {Refuse~setting~rule~for~one-time~hooks}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_gset_rule:nnnn #1#2#3#4
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_deprecated_generic:nT {#1}
```

```
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_deprecated_generic_warn:n {#1}
⟨latexrelease⟩        \__hook_do_deprecated_generic:Nn \__hook_gset_rule:nnnn
⟨latexrelease⟩          {#1} {#2} {#3} {#4}
⟨latexrelease⟩        \exp_after:wN \use_none:nnnnnnnnn \use_none:n
⟨latexrelease⟩      }
⟨latexrelease⟩    \__hook_init_structure:n {#1}
⟨latexrelease⟩    \__hook_rule_gclear:nnn {#1} {#2} {#4}
⟨latexrelease⟩    \cs_if_exist_use:cTF { __hook_rule_#3_gset:nnn }
⟨latexrelease⟩      {
⟨latexrelease⟩          {#1} {#2} {#4}
⟨latexrelease⟩        \__hook_update_hook_code:n {#1}
⟨latexrelease⟩      }
⟨latexrelease⟩      {
⟨latexrelease⟩        \msg_error:nnnnnn { hooks } { unknown-rule }
⟨latexrelease⟩          {#1} {#2} {#3} {#4}
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_gset_rule:nnnn` *and* `\__hook_gset_rule:nnnn`*. This function is documented on page 18.*)

`\__hook_rule_before_gset:nnn`
`\__hook_rule_after_gset:nnn`
`\__hook_rule_<_gset:nnn`
`\__hook_rule_>_gset:nnn`

Then we add the new rule. We need to normalize the rules here to allow for faster processing later. Given a pair of labels l_A and l_B, the rule $l_A > l_B$ is the same as $l_B < l_A$ only presented differently. But by normalizing the forms of the rule to a single representation, say, $l_B < l_A$, reduces the time spent looking for the rules later considerably.

Here we do that normalization by using `\(pdf)strcmp` to lexically sort labels l_A and l_B to a fixed order. This order is then enforced every time these two labels are used together.

Here we use `\__hook_label_pair:nn {`⟨*hook*⟩`} {`⟨l_A⟩`} {`⟨l_B⟩`}` to build a string $l_B|l_A$ with a fixed order, and use `\__hook_label_ordered:nnTF` to apply the correct rule to the pair of labels, depending if it was sorted or not.

```
\cs_new_protected:Npn \__hook_rule_before_gset:nnn #1#2#3
  {
    \__hook_tl_gset:ce
      { g__hook_#1_rule_ \__hook_label_pair:nn {#2} {#3} _tl }
      { \__hook_label_ordered:nnTF {#2} {#3} { < } { > } }
  }
\cs_new_eq:cN { __hook_rule_<_gset:nnn } \__hook_rule_before_gset:nnn
\cs_new_protected:Npn \__hook_rule_after_gset:nnn #1#2#3
  {
    \__hook_tl_gset:ce
      { g__hook_#1_rule_ \__hook_label_pair:nn {#3} {#2} _tl }
      { \__hook_label_ordered:nnTF {#3} {#2} { < } { > } }
  }
\cs_new_eq:cN { __hook_rule_>_gset:nnn } \__hook_rule_after_gset:nnn
```

(*End of definition for* `\__hook_rule_before_gset:nnn` *and others.*)

`\__hook_rule_voids_gset:nnn`

This rule removes (clears, actually) the code from label `#3` if label `#2` is in the hook `#1`.

```
\cs_new_protected:Npn \__hook_rule_voids_gset:nnn #1#2#3
  {
```

```
    \__hook_tl_gset:ce
      { g__hook_#1_rule_ \__hook_label_pair:nn {#2} {#3} _tl }
      { \__hook_label_ordered:nnTF {#2} {#3} { -> } { <- } }
  }
```

(*End of definition for* `\__hook_rule_voids_gset:nnn`.)

`\__hook_rule_incompatible-error_gset:nnn`
`\__hook_rule_incompatible-warning_gset:nnn`

These relations make an error/warning if labels `#2` and `#3` appear together in hook `#1`.

```
\cs_new_protected:cpn { __hook_rule_incompatible-error_gset:nnn } #1#2#3
  { \__hook_tl_gset:cn
      { g__hook_#1_rule_ \__hook_label_pair:nn {#2} {#3} _tl }
      { xE }
  }
\cs_new_protected:cpn { __hook_rule_incompatible-warning_gset:nnn } #1#2#3
  { \__hook_tl_gset:cn
      { g__hook_#1_rule_ \__hook_label_pair:nn {#2} {#3} _tl }
      { xW }
  }
```

(*End of definition for* `\__hook_rule_incompatible-error_gset:nnn` *and* `\__hook_rule_incompatible-warning_gset:nnn`.)

`\__hook_rule_unrelated_gset:nnn`
`\__hook_rule_gclear:nnn`

Undo a setting. `\__hook_rule_unrelated_gset:nnn` doesn't need to do anything, since we use `\__hook_rule_gclear:nnn` before setting any rule.

```
\cs_new_protected:Npn \__hook_rule_unrelated_gset:nnn #1#2#3 { }
\cs_new_protected:Npn \__hook_rule_gclear:nnn #1#2#3
  { \cs_undefine:c { g__hook_#1_rule_ \__hook_label_pair:nn {#2} {#3} _tl } }
```

(*End of definition for* `\__hook_rule_unrelated_gset:nnn` *and* `\__hook_rule_gclear:nnn`.)

`\__hook_label_pair:nn`

Ensure that the lexically greater label comes first.

```
\cs_new:Npn \__hook_label_pair:nn #1#2
  {
    \if_case:w \__hook_str_compare:nn {#1} {#2} \exp_stop_f:
          #1 | #1 %  0
    \or:   #1 | #2 % +1
    \else: #2 | #1 % -1
    \fi:
  }
```

(*End of definition for* `\__hook_label_pair:nn`.)

`\__hook_label_ordered_p:nn`
`\__hook_label_ordered:nnTF`

Check that labels `#1` and `#2` are in the correct order (as returned by `\__hook_label_pair:nn`) and if so return true, else return false.

```
\prg_new_conditional:Npnn \__hook_label_ordered:nn #1#2 { TF }
  {
    \if_int_compare:w \__hook_str_compare:nn {#1} {#2} > 0 \exp_stop_f:
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
```

(*End of definition for* `\__hook_label_ordered:nnTF`.)

__hook_if_label_case:nnnnn To avoid doing the string comparison twice in __hook_initialize_single:NNn (once with \str_if_eq:nn and again with __hook_label_ordered:nn), we use a three-way branching macro that will compare #1 and #2 and expand to \use_i:nnn if they are equal, \use_ii:nn if #1 is lexically greater, and \use_iii:nn otherwise.

```
\cs_new:Npn \__hook_if_label_case:nnnnn #1#2
  {
    \cs:w use_
      \if_case:w \__hook_str_compare:nn {#1} {#2}
         i \or: ii \else: iii \fi: :nnn
    \cs_end:
  }
```

(*End of definition for* __hook_if_label_case:nnnnn.)

__hook_update_hook_code:n Before \begin{document} this does nothing, in the body it reinitializes the hook code using the altered data.

```
\cs_new_eq:NN \__hook_update_hook_code:n \use_none:n
```

(*End of definition for* __hook_update_hook_code:n.)

__hook_initialize_all: Initialize all known hooks (at \begin{document}), i.e., update the fast execution token lists to hold the necessary code in the right order.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_initialize_all:}
⟨latexrelease⟩                    {Hooks~with~args}
\cs_new_protected:Npn \__hook_initialize_all:
  {
```

First we change __hook_update_hook_code:n which so far was a no-op to now initialize one hook. This way any later updates to the hook will run that code and also update the execution token list.

```
    \cs_gset_eq:NN \__hook_update_hook_code:n \__hook_initialize_hook_code:n
```

Now we loop over all hooks that have been defined and update each of them. Here we have to determine if the hook has arguments so that auxiliaries know what to do with hashes. We look at \c__hook_⟨*hook*⟩_parameter_tl, if it has any parameters, and set replacing_args accordingly.

```
    \__hook_debug:n { \prop_gclear:N \g__hook_used_prop }
    \seq_map_inline:Nn \g__hook_all_seq
      {
        \tl_if_empty:cTF { c__hook_##1_parameter_tl }
          { \__hook_replacing_args_false: }
          { \__hook_replacing_args_true: }
        \__hook_update_hook_code:n {##1}
        \__hook_replacing_args_reset:
      }
```

If we are debugging we show results hook by hook for all hooks that have data.

```
    \__hook_debug:n
      {
        \iow_term:e { ^^J[lthooks]~ All~initialized~(non-empty)~hooks: }
        \prop_map_inline:Nn \g__hook_used_prop
          {
            \iow_term:e
              { ^^J ~ ##1 ~ -> ~ \cs_replacement_spec:c { __hook~##1 } ~ }
          }
      }
```

After all hooks are initialized we change the "use" to just call the hook code and not initialize it (as this was already done in the preamble.

```
    \__hook_post_initialization_defs:
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_initialize_all:}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_initialize_all:
⟨latexrelease⟩  {
⟨latexrelease⟩    \cs_gset_eq:NN \__hook_update_hook_code:n
⟨latexrelease⟩                   \__hook_initialize_hook_code:n
⟨latexrelease⟩    \__hook_debug:n { \prop_gclear:N \g__hook_used_prop }
⟨latexrelease⟩    \seq_map_inline:Nn \g__hook_all_seq
⟨latexrelease⟩      { \__hook_update_hook_code:n {##1} }
⟨latexrelease⟩    \__hook_debug:n
⟨latexrelease⟩      {
⟨latexrelease⟩        \iow_term:x{^^JAll~ initialized~ (non-empty)~ hooks:}
⟨latexrelease⟩        \prop_map_inline:Nn \g__hook_used_prop
⟨latexrelease⟩          {
⟨latexrelease⟩            \iow_term:x
⟨latexrelease⟩              { ^^J ~ ##1 ~ -> ~
⟨latexrelease⟩                \cs_replacement_spec:c { __hook~##1 } ~ }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩    \cs_gset_eq:NN \hook_use:n \__hook_use_initialized:n
⟨latexrelease⟩    \cs_gset_eq:NN \__hook_preamble_hook:n \use_none:n
⟨latexrelease⟩  }
⟨@@=⟩
⟨latexrelease⟩\cs_gset_eq:NN \@expl@@@initialize@all@@
⟨latexrelease⟩               \__hook_initialize_all:
⟨@@=hook⟩
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_initialize_all:`.)

`\__hook_initialize_hook_code:n` Initializing or reinitializing the fast execution hook code. In the preamble this is selectively done in case a hook gets used and at `\begin{document}` this is done for all hooks and afterwards only if the hook code changes.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_initialize_hook_code:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_initialize_hook_code:n #1
  {
    \__hook_debug:n
      { \iow_term:e { ^^J[lthooks]~ Update~code~for~hook~'#1' \on@line :^^J } }
```

This does the sorting and the updates. First thing we do is to check if a legacy hook macro exists and if so we add it to the hook under the label `legacy`. This might make the hook non-empty so we have to do this before the then following test.

```
    \__hook_include_legacy_code_chunk:n {#1}
```

If there aren't any code chunks for the current hook, there is no point in even starting the sorting routine so we make a quick test for that and in that case just update `\__hook␣`⟨*hook*⟩ to hold the `top-level` and `next` code chunks. If there are code chunks we call `\__hook_initialize_single:NNn` and pass to it ready made csnames as they are

needed several times inside. This way we save a bit on processing time if we do that up front.

```
    \__hook_if_usable:nT {#1}
      {
        \prop_if_empty:cTF { g__hook_#1_code_prop }
          {
            \__hook_code_gset:ne {#1}
              {
```

The hook may take arguments, so we add a run of braced parameters after the `_next` and `_toplevel` macros, so that the arguments passed to the hook are forwarded to them.

```
                \exp_not:c { __hook_toplevel~#1 }
                \__hook_braced_parameter:n {#1}
                \exp_not:c { __hook_next~#1 }
                \__hook_braced_parameter:n {#1}
              }
          }
          {
```

By default the algorithm sorts the code chunks and then saves the result in a token list for fast execution; this is done by adding the code chunks one after another, using `\tl_gput_right:NV`. When we sort code for a reversed hook, all we have to do is to add the code chunks in the opposite order into the token list. So all we have to do in preparation is to change two definitions that are used later on.

```
            \__hook_if_reversed:nTF {#1}
              { \cs_set_eq:NN \__hook_tl_gput:Nn    \__hook_tl_gput_left:Nn
                \cs_set_eq:NN \__hook_clist_gput:NV \clist_gput_left:NV  }
              { \cs_set_eq:NN \__hook_tl_gput:Nn    \__hook_tl_gput_right:Nn
                \cs_set_eq:NN \__hook_clist_gput:NV \clist_gput_right:NV }
```

When sorting, some relations (namely `voids`) need to act destructively on the code property lists to remove code that shouldn't appear in the sorted hook token list, so we make a copy of the code property list that we can safely work on without changing the main one.

```
            \prop_set_eq:Nc \l__hook_work_prop { g__hook_#1_code_prop }
            \__hook_initialize_single:ccn
              { __hook~#1 } { g__hook_#1_labels_clist } {#1}
```

For debug display we want to keep track of those hooks that actually got code added to them, so we record that in plist. We use a plist to ensure that we record each hook name only once, i.e., we are only interested in storing the keys and the value is arbitrary.

```
            \__hook_debug:n
              { \exp_args:NNe \prop_gput:Nnn \g__hook_used_prop {#1} { } }
          }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_initialize_hook_code:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_initialize_hook_code:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_debug:n
⟨latexrelease⟩      { \iow_term:x { ^^J Update~code~for~hook~'#1'
⟨latexrelease⟩                      \on@line :^^J } }
```

```
⟨latexrelease⟩    \__hook_include_legacy_code_chunk:n {#1}
⟨latexrelease⟩    \__hook_if_usable:nT {#1}
⟨latexrelease⟩      {
⟨latexrelease⟩        \prop_if_empty:cTF { g__hook_#1_code_prop }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_tl_gset:co { __hook~#1 }
⟨latexrelease⟩              {
⟨latexrelease⟩                \cs:w __hook_toplevel~#1 \exp_after:wN \cs_end:
⟨latexrelease⟩                \cs:w __hook_next~#1 \cs_end:
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_if_reversed:nTF {#1}
⟨latexrelease⟩              { \cs_set_eq:NN \__hook_tl_gput:Nn
⟨latexrelease⟩                              \__hook_tl_gput_left:Nn
⟨latexrelease⟩                \cs_set_eq:NN \__hook_clist_gput:NV
⟨latexrelease⟩                              \clist_gput_left:NV  }
⟨latexrelease⟩              { \cs_set_eq:NN \__hook_tl_gput:Nn
⟨latexrelease⟩                              \__hook_tl_gput_right:Nn
⟨latexrelease⟩                \cs_set_eq:NN \__hook_clist_gput:NV
⟨latexrelease⟩                              \clist_gput_right:NV }
⟨latexrelease⟩            \prop_set_eq:Nc \l__hook_work_prop
⟨latexrelease⟩              { g__hook_#1_code_prop }
⟨latexrelease⟩            \__hook_initialize_single:ccn
⟨latexrelease⟩              { __hook~#1 } { g__hook_#1_labels_clist } {#1}
⟨latexrelease⟩            \__hook_debug:n
⟨latexrelease⟩              { \exp_args:NNx \prop_gput:Nnn \g__hook_used_prop
⟨latexrelease⟩                    {#1} { } }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_initialize_hook_code:n`.)

`\__hook_tl_csname:n`
`\__hook_seq_csname:n`

It is faster to pass a single token and expand it when necessary than to pass a bunch of character tokens around.

FMi: note to myself: verify

```
\cs_new:Npn \__hook_tl_csname:n #1 { l__hook_label_#1_tl }
\cs_new:Npn \__hook_seq_csname:n #1 { l__hook_label_#1_seq }
```

(*End of definition for* `\__hook_tl_csname:n` *and* `\__hook_seq_csname:n`.)

`\l__hook_labels_seq`
`\l__hook_labels_int`
`\l__hook_front_tl`
`\l__hook_rear_tl`
`\l__hook_label_0_tl`

For the sorting I am basically implementing Knuth's algorithm for topological sorting as given in TAOCP volume 1 pages 263–266. For this algorithm we need a number of local variables:

- List of labels used in the current hook to label code chunks:

```
\seq_new:N \l__hook_labels_seq
```

- Number of labels used in the current hook. In Knuth's algorithm this is called N:

```
\int_new:N \l__hook_labels_int
```

- The sorted code list to be build is managed using two pointers one to the front of the queue and one to the rear. We model this using token list pointers. Knuth calls them F and R:

```
\tl_new:N \l__hook_front_tl
\tl_new:N \l__hook_rear_tl
```

- The data for the start of the queue is kept in this token list, it corresponds to what Don calls `QLINK[0]` but since we aren't manipulating individual words in memory it is slightly differently done:

```
\tl_new:c { \__hook_tl_csname:n { 0 } }
```

(*End of definition for* `\l__hook_labels_seq` *and others.*)

`\__hook_initialize_single:NNn`
`\__hook_initialize_single:ccn`

`\__hook_initialize_single:NNn` implements the sorting of the code chunks for a hook and saves the result in the token list for fast execution (`#4`). The arguments are ⟨*hook-code-plist*⟩, ⟨*hook-code-tl*⟩, ⟨*hook-top-level-code-tl*⟩, ⟨*hook-next-code-tl*⟩, ⟨*hook-ordered-labels-clist*⟩ and ⟨*hook-name*⟩ (the latter is only used for debugging—the ⟨*hook-rule-plist*⟩ is accessed using the ⟨*hook-name*⟩).

The additional complexity compared to Don's algorithm is that we do not use simple positive integers but have arbitrary alphanumeric labels. As usual Don's data structures are chosen in a way that one can omit a lot of tests and I have mimicked that as far as possible. The result is a restriction I do not test for at the moment: a label can't be equal to the number 0!

FMi: Needs checking for, just in case … maybe

```
⟨!latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_initialize_single:NNn}
⟨!latexrelease⟩                      {Hooks~with~args}
\cs_new_protected:Npn \__hook_initialize_single:NNn #1#2#3
  {
```

Step T1: Initialize the data structure …

```
    \seq_clear:N \l__hook_labels_seq
    \int_zero:N  \l__hook_labels_int
```

Store the name of the hook:

```
    \tl_set:Nn \l__hook_cur_hook_tl {#3}
```

We loop over the property list holding the code and record all the labels listed there. Only the rules for those labels are of interest to us. While we are at it we count them (which gives us the N in Knuth's algorithm). The prefix `label_` is added to the variables to ensure that labels named `front`, `rear`, `labels`, or `return` don't interact with our code.

```
    \prop_map_inline:Nn \l__hook_work_prop
      {
        \int_incr:N \l__hook_labels_int
        \seq_put_right:Nn \l__hook_labels_seq {##1}
        \__hook_tl_set:cn { \__hook_tl_csname:n {##1} } { 0 }
        \seq_clear_new:c { \__hook_seq_csname:n {##1} }
      }
```

Steps T2 and T3: Here we sort the relevant rules into the data structure...

This loop constitutes a square matrix of the labels in `\l__hook_work_prop` in the vertical and the horizontal directions. However, since the rule $l_A\langle\texttt{rel}\rangle l_B$ is the same as $l_B\langle\texttt{rel}\rangle^{-1} l_A$ we can cut the loop short at the diagonal of the matrix (*i.e.*, when both labels are equal), saving a good amount of time. The way the rules were set up (see the implementation of `\__hook_rule_before_gset:nnn` above) ensures that we have no rule in the ignored side of the matrix, and all rules are seen. The rules are applied in `\__hook_apply_label_pair:nnn`, which takes the properly-ordered pair of labels as argument.

```
\prop_map_inline:Nn \l__hook_work_prop
  {
    \prop_map_inline:Nn \l__hook_work_prop
      {
        \__hook_if_label_case:nnnnn {##1} {####1}
          { \prop_map_break: }
          { \__hook_apply_label_pair:nnn {##1} {####1} }
          { \__hook_apply_label_pair:nnn {####1} {##1} }
              {#3}
      }
  }
```

Now take a breath, and look at the data structures that have been set up:

```
\__hook_debug:n { \__hook_debug_label_data:N \l__hook_work_prop }
```

Step T4:

```
\tl_set:Nn \l__hook_rear_tl { 0 }
\tl_set:cn { \__hook_tl_csname:n { 0 } } { 0 }
\seq_map_inline:Nn \l__hook_labels_seq
  {
    \int_compare:nNnT { \cs:w \__hook_tl_csname:n {##1} \cs_end: } = 0
        {
          \tl_set:cn { \__hook_tl_csname:n { \l__hook_rear_tl } }{##1}
          \tl_set:Nn \l__hook_rear_tl {##1}
        }
  }
\tl_set_eq:Nc \l__hook_front_tl { \__hook_tl_csname:n { 0 } }
\__hook_tl_gclear:N #1
\clist_gclear:N #2
```

The whole loop gets combined in steps T5–T7:

```
\bool_while_do:nn { ! \str_if_eq_p:Vn \l__hook_front_tl { 0 } }
  {
```

This part is step T5:

```
    \int_decr:N \l__hook_labels_int
    \prop_get:NVN \l__hook_work_prop \l__hook_front_tl \l__hook_return_tl
    \exp_args:NNV \__hook_tl_gput:Nn #1 \l__hook_return_tl
    \__hook_clist_gput:NV #2 \l__hook_front_tl
    \__hook_debug:n{ \iow_term:e{[lthooks]~ Handled~ code~ for~ \l__hook_front_tl} }
```

This is step T6, except that we don't use a pointer P to move through the successors, but instead use `##1` of the mapping function.

```
    \seq_map_inline:cn { \__hook_seq_csname:n { \l__hook_front_tl } }
```

```
      {
        \tl_set:ce { \__hook_tl_csname:n {##1} }
                   { \int_eval:n
                       { \cs:w \__hook_tl_csname:n {##1} \cs_end: - 1 }
                   }
        \int_compare:nNnT
            { \cs:w \__hook_tl_csname:n {##1} \cs_end: } = 0
            {
              \tl_set:cn
                { \__hook_tl_csname:n { \l__hook_rear_tl } } {##1}
              \tl_set:Nn \l__hook_rear_tl            {##1}
            }
      }
```

and here is step T7:

```
      \tl_set_eq:Nc \l__hook_front_tl
                    { \__hook_tl_csname:n { \l__hook_front_tl } }
```

This is step T8: If we haven't moved the code for all labels (i.e., if `\l__hook_labels_int` is still greater than zero) we have a loop and our partial order can't be flattened out.

```
    }
  \int_compare:nNnF \l__hook_labels_int = 0
    {
      \iow_term:e{====================}
      \iow_term:e{Error:~ label~ rules~ are~ incompatible:}
```

This is not really the information one needs in the error case but it will do for now ...

FMi: improve output on a rainy day

```
      \__hook_debug_label_data:N \l__hook_work_prop
      \iow_term:e{====================}
    }
```

After we have added all hook code to `#1`, we finish it off by adding extra code for the `top-level` (`#2`) and for one time execution (`#3`). These should normally be empty. The `top-level` code is added with `\__hook_tl_gput:Nn` as that might change for a reversed hook (then `top-level` is the very first code chunk added). The `next` code is always added last (to the right). The hook may take arguments, so we add a run of braced parameters after the `_next` and `_toplevel` macros, so that the arguments passed to the hook are forwarded to them.

```
  \exp_args:NNe \__hook_tl_gput:Nn #1
    { \exp_not:c { __hook_toplevel~#3 } \__hook_braced_parameter:n {#3} }
  \__hook_tl_gput_right:Ne #1
    { \exp_not:c { __hook_next~#3 } \__hook_braced_parameter:n {#3} }
  \use:e
    {
      \cs_gset:cpn { __hook~#3 } \use:c { c__hook_#3_parameter_tl }
        { \exp_not:V #1 }
    }
}
\cs_generate_variant:Nn \__hook_initialize_single:NNn { cc }
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_initialize_single:NNn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_initialize_single:NNn #1#2#3
⟨latexrelease⟩  {
⟨latexrelease⟩    \seq_clear:N \l__hook_labels_seq
⟨latexrelease⟩    \int_zero:N  \l__hook_labels_int
⟨latexrelease⟩    \tl_set:Nn \l__hook_cur_hook_tl {#3}
⟨latexrelease⟩    \prop_map_inline:Nn \l__hook_work_prop
⟨latexrelease⟩       {
⟨latexrelease⟩         \int_incr:N \l__hook_labels_int
⟨latexrelease⟩         \seq_put_right:Nn \l__hook_labels_seq {##1}
⟨latexrelease⟩         \__hook_tl_set:cn { \__hook_tl_csname:n {##1} } { 0 }
⟨latexrelease⟩         \seq_clear_new:c { \__hook_seq_csname:n {##1} }
⟨latexrelease⟩       }
⟨latexrelease⟩    \prop_map_inline:Nn \l__hook_work_prop
⟨latexrelease⟩      {
⟨latexrelease⟩        \prop_map_inline:Nn \l__hook_work_prop
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_if_label_case:nnnnn {##1} {####1}
⟨latexrelease⟩              { \prop_map_break: }
⟨latexrelease⟩              { \__hook_apply_label_pair:nnn {##1} {####1} }
⟨latexrelease⟩              { \__hook_apply_label_pair:nnn {####1} {##1} }
⟨latexrelease⟩                  {#3}
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩    \__hook_debug:n
⟨latexrelease⟩      { \__hook_debug_label_data:N \l__hook_work_prop }
⟨latexrelease⟩    \tl_set:Nn \l__hook_rear_tl { 0 }
⟨latexrelease⟩    \tl_set:cn { \__hook_tl_csname:n { 0 } } { 0 }
⟨latexrelease⟩    \seq_map_inline:Nn \l__hook_labels_seq
⟨latexrelease⟩      {
⟨latexrelease⟩        \int_compare:nNnT
⟨latexrelease⟩          { \cs:w \__hook_tl_csname:n {##1} \cs_end: } = 0
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_set:cn { \__hook_tl_csname:n
⟨latexrelease⟩                           { \l__hook_rear_tl } } {##1}
⟨latexrelease⟩            \tl_set:Nn \l__hook_rear_tl           {##1}
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩    \tl_set_eq:Nc \l__hook_front_tl { \__hook_tl_csname:n { 0 } }
⟨latexrelease⟩    \__hook_tl_gclear:N #1
⟨latexrelease⟩    \clist_gclear:N #2
⟨latexrelease⟩    \bool_while_do:nn
⟨latexrelease⟩      { ! \str_if_eq_p:Vn \l__hook_front_tl { 0 } }
⟨latexrelease⟩      {
⟨latexrelease⟩        \int_decr:N \l__hook_labels_int
⟨latexrelease⟩        \prop_get:NVN \l__hook_work_prop
⟨latexrelease⟩            \l__hook_front_tl \l__hook_return_tl
⟨latexrelease⟩        \exp_args:NNV \__hook_tl_gput:Nn #1 \l__hook_return_tl
⟨latexrelease⟩        \__hook_clist_gput:NV #2 \l__hook_front_tl
⟨latexrelease⟩        \__hook_debug:n{ \iow_term:x
⟨latexrelease⟩              {Handled~ code~ for~ \l__hook_front_tl} }
⟨latexrelease⟩        \seq_map_inline:cn
⟨latexrelease⟩              { \__hook_seq_csname:n { \l__hook_front_tl } }
```

```
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_set:cx { \__hook_tl_csname:n {##1} }
⟨latexrelease⟩              { \int_eval:n
⟨latexrelease⟩                { \cs:w \__hook_tl_csname:n {##1} \cs_end: - 1 }
⟨latexrelease⟩              }
⟨latexrelease⟩            \int_compare:nNnT
⟨latexrelease⟩              { \cs:w \__hook_tl_csname:n {##1} \cs_end: } = 0
⟨latexrelease⟩              {
⟨latexrelease⟩                \tl_set:cn { \__hook_tl_csname:n
⟨latexrelease⟩                               { \l__hook_rear_tl } } {##1}
⟨latexrelease⟩                \tl_set:Nn \l__hook_rear_tl           {##1}
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩        \tl_set_eq:Nc \l__hook_front_tl
⟨latexrelease⟩          { \__hook_tl_csname:n { \l__hook_front_tl } }
⟨latexrelease⟩      }
⟨latexrelease⟩    \int_compare:nNnF \l__hook_labels_int = 0
⟨latexrelease⟩      {
⟨latexrelease⟩        \iow_term:x{====================}
⟨latexrelease⟩        \iow_term:x{Error:~ label~ rules~ are~ incompatible:}
⟨latexrelease⟩        \__hook_debug_label_data:N \l__hook_work_prop
⟨latexrelease⟩        \iow_term:x{====================}
⟨latexrelease⟩      }
⟨latexrelease⟩    \exp_args:NNo \__hook_tl_gput:Nn #1
⟨latexrelease⟩                  { \cs:w __hook_toplevel~#3 \cs_end: }
⟨latexrelease⟩    \__hook_tl_gput_right:No #1 { \cs:w __hook_next~#3 \cs_end: }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_generate_variant:Nn \__hook_tl_gput_right:Nn { No }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_initialize_single:NNn`.)

`\__hook_tl_gput:Nn`
`\__hook_clist_gput:NV`

These append either on the right (normal hook) or on the left (reversed hook). This is setup up in `\__hook_initialize_hook_code:n`, elsewhere their behavior is undefined.

```
\cs_new:Npn \__hook_tl_gput:Nn    { \ERROR }
\cs_new:Npn \__hook_clist_gput:NV { \ERROR }
```

(*End of definition for* `\__hook_tl_gput:Nn` *and* `\__hook_clist_gput:NV`.)

`\__hook_apply_label_pair:nnn`
`\__hook_label_if_exist_apply:nnnF`

This is the payload of steps T2 and T3 executed in the loop described above. This macro assumes `#1` and `#2` are ordered, which means that any rule pertaining the pair `#1` and `#2` is `\g__hook_⟨hook⟩_rule_#1|#2_tl`, and not `\g__hook_⟨hook⟩_rule_#2|#1_tl`. This also saves a great deal of time since we only need to check the order of the labels once.

The arguments here are ⟨*label1*⟩, ⟨*label2*⟩, ⟨*hook*⟩, and ⟨*hook-code-plist*⟩. We are about to apply the next rule and enter it into the data structure. `\__hook_apply_label_pair:nnn` will just call `\__hook_label_if_exist_apply:nnnF` for the ⟨*hook*⟩, and if no rule is found, also try the ⟨*hook*⟩ name `??` denoting a default hook rule.

`\__hook_label_if_exist_apply:nnnF` will check if the rule exists for the given hook, and if so call `\__hook_apply_rule:nnn`.

```
\cs_new_protected:Npn \__hook_apply_label_pair:nnn #1#2#3
  {
```

Extra complication: as we use default rules and local hook specific rules we first have to check if there is a local rule and if that exist use it. Otherwise check if there is a default rule and use that.

```
    \__hook_label_if_exist_apply:nnnF {#1} {#2} {#3}
      {
```

If there is no hook-specific rule we check for a default one and use that if it exists.

```
        \__hook_label_if_exist_apply:nnnF {#1} {#2} { ?? } { }
      }
  }
\cs_new_protected:Npn \__hook_label_if_exist_apply:nnnF #1#2#3
  {
    \if_cs_exist:w g__hook_ #3 _rule_ #1 | #2 _tl \cs_end:
```

What to do precisely depends on the type of rule we have encountered. If it is a `before` rule it will be handled by the algorithm but other types need to be managed differently. All this is done in `\__hook_apply_rule:nnnN`.

```
      \__hook_apply_rule:nnn {#1} {#2} {#3}
      \exp_after:wN \use_none:n
    \else:
      \use:nn
    \fi:
  }
```

(*End of definition for* `\__hook_apply_label_pair:nnn` *and* `\__hook_label_if_exist_apply:nnnF`.)

`\__hook_apply_rule:nnn` This is the code executed in steps T2 and T3 while looping through the matrix This is part of step T3. We are about to apply the next rule and enter it into the data structure. The arguments are ⟨*label1*⟩, ⟨*label2*⟩, ⟨*hook-name*⟩, and ⟨*hook-code-plist*⟩.

```
\cs_new_protected:Npn \__hook_apply_rule:nnn #1#2#3
  {
    \cs:w __hook_apply_
      \cs:w g__hook_#3_reversed_tl \cs_end: rule_
        \cs:w g__hook_ #3 _rule_ #1 | #2 _tl \cs_end: :nnn \cs_end:
      {#1} {#2} {#3}
  }
```

(*End of definition for* `\__hook_apply_rule:nnn`.)

`\__hook_apply_rule_<:nnn` `\__hook_apply_rule_>:nnn` The most common cases are `<` and `>` so we handle that first. They are relations $\prec$ and $\succ$ in TAOCP, and they dictate sorting.

```
\cs_new_protected:cpn { __hook_apply_rule_<:nnn } #1#2#3
  {
    \__hook_debug:n { \__hook_msg_pair_found:nnn {#1} {#2} {#3} }
    \tl_set:ce { \__hook_tl_csname:n {#2} }
       { \int_eval:n{ \cs:w \__hook_tl_csname:n {#2} \cs_end: + 1 } }
    \seq_put_right:cn{ \__hook_seq_csname:n {#1} }{#2}
  }
\cs_new_protected:cpn { __hook_apply_rule_>:nnn } #1#2#3
  {
    \__hook_debug:n { \__hook_msg_pair_found:nnn {#1} {#2} {#3} }
    \tl_set:ce { \__hook_tl_csname:n {#1} }
       { \int_eval:n{ \cs:w \__hook_tl_csname:n {#1} \cs_end: + 1 } }
    \seq_put_right:cn{ \__hook_seq_csname:n {#2} }{#1}
  }
```

(*End of definition for* `\__hook_apply_rule_<:nnn` *and* `\__hook_apply_rule_>:nnn`.)

`\__hook_apply_rule_xE:nnn`
`\__hook_apply_rule_xW:nnn`

These relations make two labels incompatible within a hook. `xE` makes raises an error if the labels are found in the same hook, and `xW` makes it a warning.

```
\cs_new_protected:cpn { __hook_apply_rule_xE:nnn } #1#2#3
  {
    \__hook_debug:n { \__hook_msg_pair_found:nnn {#1} {#2} {#3} }
    \msg_error:nnnnnn { hooks } { labels-incompatible }
      {#1} {#2} {#3} { 1 }
    \use:c { __hook_apply_rule_->:nnn } {#1} {#2} {#3}
    \use:c { __hook_apply_rule_<-:nnn } {#1} {#2} {#3}
  }
\cs_new_protected:cpn { __hook_apply_rule_xW:nnn } #1#2#3
  {
    \__hook_debug:n { \__hook_msg_pair_found:nnn {#1} {#2} {#3} }
    \msg_warning:nnnnnn { hooks } { labels-incompatible }
      {#1} {#2} {#3} { 0 }
  }
```

(*End of definition for* `\__hook_apply_rule_xE:nnn` *and* `\__hook_apply_rule_xW:nnn`.)

`\__hook_apply_rule_->:nnn`
`\__hook_apply_rule_<-:nnn`

If we see `->` we have to drop code for label `#3` and carry on. We could do a little better and drop everything for that label since it doesn't matter where we put such empty code. However that would complicate the algorithm a lot with little gain.[10] So we still unnecessarily try to sort it in and depending on the rules that might result in a loop that is otherwise resolved. If that turns out to be a real issue, we can improve the code.

Here the code is removed from `\l__hook_cur_hook_tl` rather than `#3` because the latter may be `??`, and the default hook doesn't store any code. Removing it instead from `\l__hook_cur_hook_tl` makes the default rules `->` and `<-` work properly.

```
\cs_new_protected:cpn { __hook_apply_rule_->:nnn } #1#2#3
  {
    \__hook_debug:n
      {
        \__hook_msg_pair_found:nnn {#1} {#2} {#3}
        \iow_term:e{--->~ Drop~ '#2'~ code~ from~
          \iow_char:N \\ g__hook_ \l__hook_cur_hook_tl _code_prop ~
          because~ of~ '#1' }
      }
    \prop_put:Nnn \l__hook_work_prop {#2} { }
  }
\cs_new_protected:cpn { __hook_apply_rule_<-:nnn } #1#2#3
  {
    \__hook_debug:n
      {
        \__hook_msg_pair_found:nnn {#1} {#2} {#3}
        \iow_term:e{--->~ Drop~ '#1'~ code~ from~
          \iow_char:N \\ g__hook_ \l__hook_cur_hook_tl _code_prop ~
          because~ of~ '#2' }
      }
    \prop_put:Nnn \l__hook_work_prop {#1} { }
  }
```

[10] This also has the advantage that the result of the sorting doesn't change, as it might otherwise do (for unrelated chunks) if we aren't careful.

(*End of definition for* `\__hook_apply_rule_->:nnn` *and* `\__hook_apply_rule_<-:nnn`.)

`\__hook_apply_-rule_<:nnn`
`\__hook_apply_-rule_>:nnn`
`\__hook_apply_-rule_<-:nnn`
`\__hook_apply_-rule_->:nnn`
`\__hook_apply_-rule_xW:nnn`
`\__hook_apply_-rule_xE:nnn`

Reversed rules.

```
\cs_new_eq:cc { __hook_apply_-rule_<:nnn } { __hook_apply_rule_>:nnn }
\cs_new_eq:cc { __hook_apply_-rule_>:nnn } { __hook_apply_rule_<:nnn }
\cs_new_eq:cc { __hook_apply_-rule_<-:nnn } { __hook_apply_rule_<-:nnn }
\cs_new_eq:cc { __hook_apply_-rule_->:nnn } { __hook_apply_rule_->:nnn }
\cs_new_eq:cc { __hook_apply_-rule_xE:nnn } { __hook_apply_rule_xE:nnn }
\cs_new_eq:cc { __hook_apply_-rule_xW:nnn } { __hook_apply_rule_xW:nnn }
```

(*End of definition for* `\__hook_apply_-rule_<:nnn` *and others.*)

`\__hook_msg_pair_found:nnn`

A macro to avoid moving this many tokens around.

```
\cs_new_protected:Npn \__hook_msg_pair_found:nnn #1#2#3
  {
    \iow_term:e{~ \str_if_eq:nnTF {#3} {??} {default} {~normal} ~
        rule~ \__hook_label_pair:nn {#1} {#2}:~
        \use:c { g__hook_#3_rule_ \__hook_label_pair:nn {#1} {#2} _tl } ~
        found}
  }
```

(*End of definition for* `\__hook_msg_pair_found:nnn`.)

`\__hook_debug_label_data:N`

```
\cs_new_protected:Npn \__hook_debug_label_data:N #1 {
  \iow_term:e{Code~ labels~ for~ sorting:}
  \iow_term:e{~ \seq_use:Nnnn\l__hook_labels_seq {~and~}{,~}{~and~} }
  \iow_term:e{^^J Data~ structure~ for~ label~ rules:}
  \prop_map_inline:Nn #1
       {
         \iow_term:e{~ ##1~ =~ \tl_use:c{ \__hook_tl_csname:n {##1} }~ ->~
           \seq_use:cnnn{ \__hook_seq_csname:n {##1} }{~->~}{~->~}{~->~}
         }
       }
  \iow_term:e{}
}
```

(*End of definition for* `\__hook_debug_label_data:N`.)

`\hook_show:n`
`\hook_log:n`
`\__hook_log_line:e`
`\__hook_log_line_indent:e`
`\__hook_log:nN`

This writes out information about the hook given in its argument onto the `.log` file and the terminal, if `\show_hook:n` is used. Internally both share the same structure, except that at the end, `\hook_show:n` triggers TEX's prompt.

```
\cs_new_protected:Npn \hook_log:n #1
  {
    \cs_set_eq:NN \__hook_log_cmd:e \iow_log:e
    \__hook_normalize_hook_args:Nn \__hook_log:nN {#1} \tl_log:e
  }
\cs_if_exist:NTF \iow_show:e
  {
    \cs_new_protected:Npn \hook_show:n #1
      {
        \cs_set_eq:NN \__hook_log_cmd:e \iow_show:e
        \__hook_normalize_hook_args:Nn \__hook_log:nN {#1} \tl_show:e
      }
```

```
  }
  {
    \cs_new_protected:Npn \hook_show:n #1
      {
        \cs_set_eq:NN \__hook_log_cmd:e \iow_term:e
        \__hook_normalize_hook_args:Nn \__hook_log:nN {#1} \tl_show:e
      }
  }
\cs_new_protected:Npn \__hook_log_line:e #1
  { \__hook_log_cmd:e { >~#1 } }
\cs_new_protected:Npn \__hook_log_line_indent:e #1
  { \__hook_log_cmd:e { >~\@spaces #1 } }

⟨!latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_log:nN}
⟨!latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_log:nN #1 #2
  {
    \__hook_if_deprecated_generic:nT {#1}
      {
        \__hook_deprecated_generic_warn:n {#1}
        \__hook_do_deprecated_generic:Nn \__hook_log:nN {#1} #2
        \exp_after:wN \use_none:nnnnnnnnn \use_none:nnnnn
      }
    \__hook_preamble_hook:n {#1}
    \__hook_log_cmd:e
      {
        ^^J ->~The~
        \__hook_if_generic:nT {#1} { generic~ }
        hook~'#1'
        \__hook_if_disabled:nF {#1}
          {
            \exp_args:Nne \__hook_print_args:nn {#1}
              {
                \int_eval:n
                  { \str_count:e { \__hook_parameter:n {#1} } / 3 }
              }
          }
        :
      }

    \__hook_if_usable:nF {#1}
      { \__hook_log_line:e { The~hook~is~not~declared. } }
    \__hook_if_disabled:nT {#1}
      { \__hook_log_line:e { The~hook~is~disabled. } }
    \hook_if_empty:nTF {#1}
      { #2 { The~hook~is~empty } }
      {
        \__hook_log_line:e { Code~chunks: }
        \bool_lazy_or:nnTF
          { ! \prop_if_exist_p:c { g__hook_#1_code_prop } }
          { \prop_if_empty_p:c { g__hook_#1_code_prop } }
          { \__hook_log_line_indent:e { --- } }
          {
            \prop_map_inline:cn { g__hook_#1_code_prop }
              {
```

```
      \exp_after:wN \cs_set:Npn \exp_after:wN \__hook_tmp:w
        \c__hook_nine_parameters_tl {##2}
      \__hook_log_line_indent:e
        { ##1~->~\cs_replacement_spec:N \__hook_tmp:w }
    }
  }
```

If there is code in the `top-level` token list, print it:

```
\__hook_log_line:e
  {
    Document-level~(top-level)~code
    \__hook_if_usable:nT {#1}
      { ~(executed~\__hook_if_reversed:nTF {#1} {first} {last} ) } :
  }
\__hook_log_line_indent:e
  {
    \__hook_cs_if_empty:cTF { __hook_toplevel~#1 }
      { --- }
      { -> ~ \cs_replacement_spec:c { __hook_toplevel~#1 } }
  }
\__hook_log_line:e { Extra~code~for~next~invocation: }
\__hook_log_line_indent:e
  {
    \__hook_cs_if_empty:cTF { __hook_next~#1 }
      { --- }
```

If the token list is not empty we want to display it but without the first tokens (the code to clear itself) so we call a helper command to get rid of them.

```
      {
        -> ~ \exp_last_unbraced:Nf \__hook_log_next_code:w
          { \cs_replacement_spec:c { __hook_next~#1 } }
      }
  }
```

Loop through the rules in a hook and for every rule found, print it. If no rule is there, print `---`. The boolean `\l__hook_tmpa_bool` here indicates if the hook has no rules.

```
\__hook_log_line:e { Rules: }
\bool_set_true:N \l__hook_tmpa_bool
\__hook_list_rules:nn {#1}
  {
    \bool_set_false:N \l__hook_tmpa_bool
    \__hook_log_line_indent:e
      {
        ##2~ with~
        \str_if_eq:nnT {##3} {??} { default~ }
        relation~ ##1
      }
  }
\bool_if:NT \l__hook_tmpa_bool
  { \__hook_log_line_indent:e { --- } }
```

When the hook is declared (that is, the sorting algorithm is applied to that hook) and not empty

```
      \bool_lazy_and:nnTF
          { \__hook_if_usable_p:n {#1} }
          { ! \hook_if_empty_p:n {#1} }
        {
          \__hook_log_line:e
            {
              Execution~order
              \bool_if:NTF \l__hook_tmpa_bool
                { \__hook_if_reversed:nT {#1} { ~(after~reversal) } }
                { ~(after~
                  \__hook_if_reversed:nT {#1} { reversal~and~ }
                  applying~rules)
                } :
            }
          #2 % \tl_show:n
            {
              \@spaces
              \clist_if_empty:cTF { g__hook_#1_labels_clist }
                { --- }
                { \clist_use:cn { g__hook_#1_labels_clist } { ,~ } }
            }
        }
        {
          \__hook_log_line:e { Execution~order: }
          #2
            {
              \@spaces Not~set~because~the~hook~ \__hook_if_usable:nTF {#1}
                { code~pool~is~empty }
                { is~\__hook_if_disabled:nTF {#1} {disabled} {undeclared} }
            }
        }
    }
  }
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_log:nN}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_log:nN #1 #2
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_deprecated_generic:nT {#1}
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_deprecated_generic_warn:n {#1}
⟨latexrelease⟩        \__hook_do_deprecated_generic:Nn \__hook_log:nN {#1} #2
⟨latexrelease⟩        \exp_after:wN \use_none:nnnnnnnnn \use_none:nnnnn
⟨latexrelease⟩      }
⟨latexrelease⟩    \__hook_preamble_hook:n {#1}
⟨latexrelease⟩    \__hook_log_cmd:e
⟨latexrelease⟩      { ^^J ->~The~ \__hook_if_generic:nT
⟨latexrelease⟩                      {#1} { generic~ } hook~'#1': }
⟨latexrelease⟩    \__hook_if_usable:nF {#1}
⟨latexrelease⟩      { \__hook_log_line:e { The~hook~is~not~declared. } }
⟨latexrelease⟩    \__hook_if_disabled:nT {#1}
⟨latexrelease⟩      { \__hook_log_line:e { The~hook~is~disabled. } }
⟨latexrelease⟩    \hook_if_empty:nTF {#1}
```

```
⟨latexrelease⟩      { #2 { The~hook~is~empty } }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_log_line:e { Code~chunks: }
⟨latexrelease⟩        \prop_if_empty:cTF { g__hook_#1_code_prop }
⟨latexrelease⟩          { \__hook_log_line_indent:e { --- } }
⟨latexrelease⟩          {
⟨latexrelease⟩            \prop_map_inline:cn { g__hook_#1_code_prop }
⟨latexrelease⟩              { \__hook_log_line_indent:e
⟨latexrelease⟩                  { ##1~->~\tl_to_str:n {##2} } }
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_log_line:e
⟨latexrelease⟩          {
⟨latexrelease⟩            Document-level~(top-level)~code
⟨latexrelease⟩            \__hook_if_usable:nT {#1}
⟨latexrelease⟩              { ~(executed~
⟨latexrelease⟩                \__hook_if_reversed:nTF {#1} {first} {last} ) } :
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_log_line_indent:e
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_if_empty:cTF { __hook_toplevel~#1 }
⟨latexrelease⟩              { --- }
⟨latexrelease⟩              { -> ~ \exp_args:Nv \tl_to_str:n
⟨latexrelease⟩                                  { __hook_toplevel~#1 } }
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_log_line:e { Extra~code~for~next~invocation: }
⟨latexrelease⟩        \__hook_log_line_indent:e
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_if_empty:cTF { __hook_next~#1 }
⟨latexrelease⟩              { --- }
⟨latexrelease⟩              { ->~ \exp_args:Nv \__hook_log_next_code:n
⟨latexrelease⟩                                  { __hook_next~#1 } }
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_log_line:e { Rules: }
⟨latexrelease⟩        \bool_set_true:N \l__hook_tmpa_bool
⟨latexrelease⟩        \__hook_list_rules:nn {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \bool_set_false:N \l__hook_tmpa_bool
⟨latexrelease⟩            \__hook_log_line_indent:e
⟨latexrelease⟩              {
⟨latexrelease⟩                ##2~ with~
⟨latexrelease⟩                \str_if_eq:nnT {##3} {??} { default~ }
⟨latexrelease⟩                relation~ ##1
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩        \bool_if:NT \l__hook_tmpa_bool
⟨latexrelease⟩          { \__hook_log_line_indent:e { --- } }
⟨latexrelease⟩        \bool_lazy_and:nnTF
⟨latexrelease⟩            { \__hook_if_usable_p:n {#1} }
⟨latexrelease⟩            { ! \hook_if_empty_p:n {#1} }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_log_line:e
⟨latexrelease⟩              {
⟨latexrelease⟩                Execution~order
⟨latexrelease⟩                \bool_if:NTF \l__hook_tmpa_bool
```

```
⟨latexrelease⟩                  { \__hook_if_reversed:nT
⟨latexrelease⟩                      {#1}{ ~(after~reversal) } }
⟨latexrelease⟩                  { ~(after~
⟨latexrelease⟩                    \__hook_if_reversed:nT {#1} { reversal~and~ }
⟨latexrelease⟩                    applying~rules)
⟨latexrelease⟩                  } :
⟨latexrelease⟩              }
⟨latexrelease⟩            #2 % \tl_show:n
⟨latexrelease⟩              {
⟨latexrelease⟩                \@spaces
⟨latexrelease⟩                \clist_if_empty:cTF { g__hook_#1_labels_clist }
⟨latexrelease⟩                  { --- }
⟨latexrelease⟩                  { \clist_use:cn
⟨latexrelease⟩                      { g__hook_#1_labels_clist } { ,~ } }
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_log_line:e { Execution~order: }
⟨latexrelease⟩            #2
⟨latexrelease⟩              {
⟨latexrelease⟩                \@spaces Not~set~because~the~hook~
⟨latexrelease⟩                    \__hook_if_usable:nTF {#1}
⟨latexrelease⟩                  { code~pool~is~empty }
⟨latexrelease⟩                  { is~\__hook_if_disabled:nTF
⟨latexrelease⟩                      {#1} {disabled} {undeclared} }
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

To display the code for next invocation only (i.e., from `\AddToHookNext` we have to remove the string `\__hook_clear_next:n{`⟨*hook*⟩`}`, so the simplest is to use a macro delimited by a $\}_1 2$.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_log_next_code:n}
⟨latexrelease⟩                  {Hooks~with~args}
\__hook_log_next_code:n \exp_last_unbraced:NNNNo
\cs_new:Npn \__hook_log_next_code:w #1 \c_right_brace_str { }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_log_next_code:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset:Npn \__hook_log_next_code:n #1
⟨latexrelease⟩  { \exp_args:No \tl_to_str:n { \use_none:nn #1 } }
⟨latexrelease⟩\EndIncludeInRelease
```

Pretty-prints the number of arguments of a hook.

```
\cs_new:Npn \__hook_print_args:nn #1 #2
  {
    \int_compare:nNnT {#2} > { 0 }
      {
\__hook_print_args:n    \__hook_if_declared:nT {#1} { \use_none:nnn }
        \__hook_if_cmd_hook:nT {#1}
          { \use_i:nnn { ~ (unknown ~ } }
        \use:n { ~ (#2 ~ }
```

```
        argument \int_compare:nNnT {#2} > { 1 } { s } )
    }
  }
```

(*End of definition for* `\hook_show:n` *and others. These functions are documented on page* 18.)

`\__hook_list_rules:nn`
`\__hook_list_one_rule:nnn`
`\__hook_list_if_rule_exists:nnnF`

This macro takes a ⟨*hook*⟩ and an ⟨*inline function*⟩ and loops through each pair of ⟨*labels*⟩ in the ⟨*hook*⟩, and if there is a relation between this pair of ⟨*labels*⟩, the ⟨*inline function*⟩ is executed with `#1` = ⟨*relation*⟩, `#2` = ⟨$label_1$⟩|⟨$label_2$⟩, and `#3` = ⟨*hook*⟩ (the latter may be the argument `#1` to `\__hook_list_rules:nn`, or `??` if it is a default rule).

```
\cs_new_protected:Npn \__hook_list_rules:nn #1 #2
  {
    \prop_if_exist:cT { g__hook_#1_code_prop }
      {
        \cs_set_protected:Npn \__hook_tmp:w ##1 ##2 ##3 {#2}
        \prop_map_inline:cn { g__hook_#1_code_prop }
          {
            \prop_map_inline:cn { g__hook_#1_code_prop }
              {
                \__hook_if_label_case:nnnnn {##1} {####1}
                  { \prop_map_break: }
                  { \__hook_list_one_rule:nnn {##1} {####1} }
                  { \__hook_list_one_rule:nnn {####1} {##1} }
                      {#1}
              }
          }
      }
  }
```

These two are quite similar to `\__hook_apply_label_pair:nnn` and `\__hook_label_if_exist_apply:nnnF`, respectively, but rather than applying the rule, they pass it to the ⟨*inline function*⟩.

```
\cs_new_protected:Npn \__hook_list_one_rule:nnn #1#2#3
  {
    \__hook_list_if_rule_exists:nnnF {#1} {#2} {#3}
      { \__hook_list_if_rule_exists:nnnF {#1} {#2} { ?? } { } }
  }
\cs_new_protected:Npn \__hook_list_if_rule_exists:nnnF #1#2#3
  {
    \if_cs_exist:w g__hook_ #3 _rule_ #1 | #2 _tl \cs_end:
      \exp_args:Nv \__hook_tmp:w
        { g__hook_ #3 _rule_ #1 | #2 _tl } { #1 | #2 } {#3}
      \exp_after:wN \use_none:nn
    \fi:
    \use:n
  }
```

(*End of definition for* `\__hook_list_rules:nn`, `\__hook_list_one_rule:nnn`, *and* `\__hook_list_if_rule_exists:nnnF`.)

`\__hook_debug_print_rules:n`

A shorthand for debugging that prints similar to `\prop_show:N`.

```
\cs_new_protected:Npn \__hook_debug_print_rules:n #1
  {
```

```
    \iow_term:n { The~hook~#1~contains~the~rules: }
    \cs_set_protected:Npn \__hook_tmp:w ##1
      {
        \__hook_list_rules:nn {#1}
          {
            \iow_term:e
              {
                > ##1 {####2} ##1 => ##1 {####1}
                \str_if_eq:nnT {####3} {??} { ~(default) }
              }
          }
      }
    \exp_args:No \__hook_tmp:w { \use:nn { ~ } { ~ } }
  }
```

(*End of definition for* `\__hook_debug_print_rules:n`.)

4.8 Specifying code for next invocation

`\hook_gput_next_code:nn`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_gput_next_code:nn}
⟨latexrelease⟩                    {Hooks~with~args}
\cs_new_protected:Npn \hook_gput_next_code:nn #1 #2
  {
    \__hook_replacing_args_false:
    \__hook_normalize_hook_args:Nn \__hook_gput_next_code:nn {#1} {#2}

    \__hook_debug:n{\iow_term:e{[lthooks]~ Add~ to~
                      \__hook_if_usable:nF {#1} { undeclared~ }
                      hook~ '#1'~ ( next~ invocation~ only )
                      \on@line
                      ^^J [lthooks] \@spaces
                      <-- \tl_to_str:n{#2}
      }
    }

    \__hook_replacing_args_reset:
  }
\cs_new_protected:Npn \hook_gput_next_code_with_args:nn #1 #2
  {
    \__hook_replacing_args_true:
    \__hook_normalize_hook_args:Nn \__hook_gput_next_code:nn {#1} {#2}

    \__hook_debug:n{\iow_term:e{[lthooks]~ Add~ to~
                      \__hook_if_usable:nF {#1} { undeclared~ }
                      hook~ '#1'~ ( next~ invocation~ only )
                      \on@line
                      ^^J [lthooks] \@spaces
                      <-- \tl_to_str:n{#2}
      }
    }

    \__hook_replacing_args_reset:
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_gput_next_code:nn}
```

```
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_gput_next_code:nn #1
⟨latexrelease⟩  { \__hook_normalize_hook_args:Nn
⟨latexrelease⟩          \__hook_gput_next_code:nn {#1} }
⟨latexrelease⟩\cs_gset_protected:Npn \hook_gput_next_code_with_args:nn #1 #2 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_gput_next_code:nn`*. This function is documented on page 17.*)

`\__hook_gput_next_code:nn`

```
\cs_new_protected:Npn \__hook_gput_next_code:nn #1 #2
  {
    \__hook_if_disabled:nTF {#1}
      { \msg_error:nnn { hooks } { hook-disabled } {#1} }
      {
        \__hook_if_structure_exist:nTF {#1}
          { \__hook_gput_next_do:nn }
          { \__hook_try_declaring_generic_next_hook:nn }
              {#1} {#2}
      }
  }
```

(*End of definition for* `\__hook_gput_next_code:nn`*.*)

`\__hook_gput_next_do:nn` Start by sanity-checking with `\__hook_chk_args_allowed:nn`. Then check if the "next code" token list is empty: if so we need to add a `\tl_gclear:c` to clear it, so the code lasts for one usage only. The token list is cleared early so that nested usages don't get lost. `\tl_gclear:c` is used instead of `\tl_gclear:N` in case the hook is used in an expansion-only context, so the token list doesn't expand before `\tl_gclear:N`: that would make an infinite loop. Also in case the main code token list is empty, the hook code has to be updated to add the next execution token list.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_gput_next_do:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_gput_next_do:nn #1
  {
    \__hook_init_structure:n {#1}
    \__hook_chk_args_allowed:nn {#1} { AddToHookNext }
    \__hook_cs_if_empty:cT { __hook~#1 }
      { \__hook_update_hook_code:n {#1} }
    \__hook_cs_if_empty:cT { __hook_next~#1 }
      { \__hook_next_gset:nn {#1} { \__hook_clear_next:n {#1} } }
    \__hook_cs_gput_right:nnn { _next } {#1}
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_gput_next_do:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_gput_next_do:nn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \exp_args:Nc \__hook_gput_next_do:Nnn
⟨latexrelease⟩      { __hook_next~#1 } {#1}
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_gput_next_do:Nnn #1 #2
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_empty:cT { __hook~#2 }
```

```
⟨latexrelease⟩      { \__hook_update_hook_code:n {#2} }
⟨latexrelease⟩    \tl_if_empty:NT #1
⟨latexrelease⟩      { \__hook_tl_gset:Nn #1 { \__hook_clear_next:n {#2} } }
⟨latexrelease⟩    \__hook_tl_gput_right:Nn #1
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_gput_next_do:nn`.)

`\hook_gclear_next_code:n` Discard anything set up for next invocation of the hook.

```
\cs_new_protected:Npn \hook_gclear_next_code:n #1
  { \__hook_normalize_hook_args:Nn \__hook_clear_next:n {#1} }
```

(*End of definition for* `\hook_gclear_next_code:n`. *This function is documented on page* 17.)

`\__hook_clear_next:n`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_clear_next:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_clear_next:n #1
  { \__hook_next_gset:nn {#1} { } }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_clear_next:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_clear_next:n #1
⟨latexrelease⟩  { \cs_gset_eq:cN { __hook_next~#1 } \c_empty_tl }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_clear_next:n`.)

4.9 Using the hook

`\hook_use:n` `\__hook_use_initialized:n` `\__hook_preamble_hook:n`

`\hook_use:n` as defined here is used in the preamble, where hooks aren't initialized by default. `\__hook_use_initialized:n` is also defined, which is the non-`\protected` version for use within the document. Their definition is identical, except for the `\__hook_preamble_hook:n` (which wouldn't hurt in the expandable version, but it would be an unnecessary extra expansion).

`\__hook_use_initialized:n` holds the expandable definition while in the preamble. `\__hook_preamble_hook:n` initializes the hook in the preamble, and is redefined to `\use_none:n` at `\begin{document}`.

Both versions do the same thing internally: they check that the hook exists as given, and if so they use it as quickly as possible.

At `\begin{document}`, all hooks are initialized, and any change in them causes an update, so `\hook_use:n` can be made expandable. This one is better not protected so that it can expand into nothing if containing no code. Also important in case of generic hooks that we do not generate a `\relax` as a side effect of checking for a csname. In contrast to the TEX low-level `\csname ...\endcsname` construct `\tl_if_exist:c` is careful to avoid this.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_use:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_use:n #1
  {
    \__hook_preamble_hook:n {#1}
    \__hook_use_initialized:n {#1}
```

```
  }
\cs_new:Npn \__hook_use_initialized:n #1
  {
    \if_cs_exist:w __hook~#1 \cs_end:
      \cs:w __hook~#1 \use_i:nn
    \fi:
    \use_none:n
    \cs_end:
  }
\cs_new_protected:Npn \__hook_preamble_hook:n #1
  {
    \if_cs_exist:w __hook~#1 \cs_end:
      \__hook_initialize_hook_code:n {#1}
    \fi:
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\hook_use:n}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new_protected:Npn \hook_use:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_exist:cT { __hook~#1 }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_preamble_hook:n {#1}
⟨latexrelease⟩        \cs:w __hook~#1 \cs_end:
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new:Npn \__hook_use_initialized:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \if_cs_exist:w __hook~#1 \cs_end:
⟨latexrelease⟩      \cs:w __hook~#1 \exp_after:wN \cs_end:
⟨latexrelease⟩    \fi:
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new_protected:Npn \__hook_preamble_hook:n #1
⟨latexrelease⟩  { \__hook_initialize_hook_code:n {#1} }
⟨latexrelease⟩\cs_new:Npn \hook_use:nnw #1 { }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_use:n}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new_protected:Npn \hook_use:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_exist:cTF { __hook~#1 }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_preamble_hook:n {#1}
⟨latexrelease⟩        \cs:w __hook~#1 \cs_end:
⟨latexrelease⟩      }
⟨latexrelease⟩      { \__hook_use:wn #1 / \s__hook_mark {#1} }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new:Npn \__hook_use_initialized:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \if_cs_exist:w __hook~#1 \cs_end:
⟨latexrelease⟩    \else:
⟨latexrelease⟩      \__hook_use_undefined:w
⟨latexrelease⟩    \fi:
```

```
⟨latexrelease⟩    \cs:w __hook~#1 \__hook_use_end:
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new:Npn \__hook_use_undefined:w
⟨latexrelease⟩    #1 #2 __hook~#3 \__hook_use_end:
⟨latexrelease⟩  {
⟨latexrelease⟩    #1 % fi
⟨latexrelease⟩    \__hook_use:wn #3 / \s__hook_mark {#3}
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new_protected:Npn \__hook_preamble_hook:n #1
⟨latexrelease⟩  { \__hook_initialize_hook_code:n {#1} }
⟨latexrelease⟩\cs_new_eq:NN \__hook_use_end: \cs_end:
⟨latexrelease⟩\cs_new:Npn \hook_use:nnw #1 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_use:n`, `\__hook_use_initialized:n`, *and* `\__hook_preamble_hook:n`. *This function is documented on page* 16.)

`\hook_use:nnw`
`\__hook_use_initialized:nnw`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_use:nnw}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_use:nnw #1
  {
    \__hook_preamble_hook:n {#1}
    \__hook_use_initialized:nnw {#1}
  }
\cs_new:Npn \__hook_use_initialized:nnw #1 #2
  {
    \cs:w
      \if_cs_exist:w __hook~#1 \cs_end:
        __hook~#1
      \else:
        use_none: \prg_replicate:nn {#2} { n }
      \fi:
    \cs_end:
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_use:nnw}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset:Npn \hook_use:nnw #1 #2
⟨latexrelease⟩  { \use:c { use_none: \prg_replicate:nn {#2} { n } } }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_use:nnw` *and* `\__hook_use_initialized:nnw`. *This function is documented on page* 16.)

`\__hook_post_initialization_defs:`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_post_initialization_defs:}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_post_initialization_defs:
  {
    \cs_gset_eq:NN \hook_use:n \__hook_use_initialized:n
    \cs_gset_eq:NN \hook_use:nnw \__hook_use_initialized:nnw
    \cs_gset_eq:NN \__hook_preamble_hook:n \use_none:n
    \cs_gset_eq:NN \__hook_post_initialization_defs: \prg_do_nothing:
  }
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_post_initialization_defs:}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_post_initialization_defs:
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_post_initialization_defs:`.)

`\__hook_use:wn`
`\__hook_try_file_hook:n`
`\__hook_if_usable_use:n`

`\__hook_use:wn` does a quick check to test if the current hook is a file hook: those need a special treatment. If it is not, the hook does not exist. If it is, then `\__hook_try_file_hook:n` is called, and checks that the current hook is a file-specific hook using `\__hook_if_file_hook:wTF`. If it's not, then it's a generic `file/` hook and is used if it exist.

If it is a file-specific hook, it passes through the same normalization as during declaration, and then it is used if defined. `\__hook_if_usable_use:n` checks if the hook exist, and calls `\__hook_preamble_hook:n` if so, then uses the hook.

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__hook_use:wn}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_use:wn}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new:Npn \__hook_use:wn #1 / #2 \s__hook_mark #3
⟨latexrelease⟩  {
⟨latexrelease⟩    \str_if_eq:nnTF {#1} { file }
⟨latexrelease⟩      { \__hook_try_file_hook:n {#3} }
⟨latexrelease⟩      { } % Hook doesn't exist
⟨latexrelease⟩  }

⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_file_hook:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_file_hook:wTF #1 / / \s__hook_mark
⟨latexrelease⟩      {
⟨latexrelease⟩        \exp_args:Ne \__hook_if_usable_use:n
⟨latexrelease⟩          { \exp_args:Ne \__hook_file_hook_normalize:n {#1} }
⟨latexrelease⟩      }
⟨latexrelease⟩      { \__hook_if_usable_use:n {#1} }
⟨latexrelease⟩            % file/ generic hook (e.g. file/before)
⟨latexrelease⟩  }

⟨latexrelease⟩\cs_new_protected:Npn \__hook_if_usable_use:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_exist:cT { __hook~#1 }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_preamble_hook:n {#1}
⟨latexrelease⟩        \cs:w __hook~#1 \cs_end:
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_use:wn`, `\__hook_try_file_hook:n`, *and* `\__hook_if_usable_use:n`.)

`\hook_use_once:n`
`\hook_use_once:nnw`

For hooks that can and should be used only once we have a special use command that further inhibits the hook from getting more code added to it. This has the effect that any further code added to the hook is executed immediately rather than stored in the hook.

The code needs some gymnastics to prevent space trimming from the hook name, since `\hook_use:n` and `\hook_use_once:n` are documented to not trim spaces.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_use_once:nnw}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_use_once:n #1
  {
    \__hook_if_execute_immediately:nF {#1}
      { \__hook_normalize_hook_args:Nn \__hook_use_once:nn
          { \use:n {#1} } { 0 } }
  }
\cs_new_protected:Npn \hook_use_once:nnw #1 #2
  {
    \__hook_if_execute_immediately:nF {#1}
      { \__hook_normalize_hook_args:Nn \__hook_use_once:nn
          { \use:n {#1} } {#2} }
  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_use_once:n` *and* `\hook_use_once:nnw`. *These functions are documented on page 16.*)

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_use_once:nnw}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_use_once:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_execute_immediately:nF {#1}
⟨latexrelease⟩      { \__hook_normalize_hook_args:Nn \__hook_use_once:n
⟨latexrelease⟩          { \use:n {#1} } }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset:Npn \hook_use_once:nnw #1 #2
⟨latexrelease⟩  { \use:c { use_none: \prg_replicate:nn {#2} { n } } }
⟨latexrelease⟩\EndIncludeInRelease
```

`\__hook_use_once:nn`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_use_once:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_use_once:nn #1 #2
  {
    \__hook_preamble_hook:n {#1}
    \__hook_use_once_set:n {#1}
```

When a hook has arguments, the call to `\__hook_use_initialized:n`, should be the very last thing to happen, otherwise the arguments grabbed will be wrong. So, to clean up after the hook we need to cheat a bit and sneak the cleanup code at the end of the hook, along with the next execution code.

```
    \__hook_replacing_args_false:
    \__hook_cs_gput_right:nnn { _next } {#1}
      { \__hook_use_once_clear:n {#1} }
    \__hook_replacing_args_reset:
    \__hook_if_usable:nTF {#1}
      { \__hook_use_initialized:n {#1} }
      {
        \int_compare:nNnT {#2} > { 0 }
          { \use:c { use_none: \prg_replicate:nn {#2} { n } } }
      }
```

```
  }
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_use_once:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_use_once:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_preamble_hook:n {#1}
⟨latexrelease⟩    \__hook_use_once_set:n {#1}
⟨latexrelease⟩    \__hook_use_initialized:n {#1}
⟨latexrelease⟩    \__hook_use_once_clear:n {#1}
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_undefine:N \__hook_use_once:nn
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_use_once:nn`.)

`\__hook_use_once_set:n`
`\__hook_use_once_clear:n`

`\__hook_use_once_set:n` is used before the actual hook code is executed so that any usage of `\AddToHook` inside the hook causes the code to execute immediately. Setting `\g__hook_⟨hook⟩_reversed_tl` to `I` prevents further code from being added to the hook. `\__hook_use_once_clear:n` then clears the hook so that any further call to `\hook_use:n` or `\hook_use_once:n` will expand to nothing.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_use_once_clear:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_use_once_set:n #1
  { \__hook_tl_gset:cn { g__hook_#1_reversed_tl } { I } }
\cs_new_protected:Npn \__hook_use_once_clear:n #1
  {
    \__hook_code_gset:nn {#1} { }
    \__hook_next_gset:nn {#1} { }
    \__hook_toplevel_gset:nn {#1} { }
    \prop_gclear_new:c { g__hook_#1_code_prop }
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_use_once_clear:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_use_once_clear:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_tl_gclear:c { __hook~#1 }
⟨latexrelease⟩    \__hook_tl_gclear:c { __hook_next~#1 }
⟨latexrelease⟩    \__hook_tl_gclear:c { __hook_toplevel~#1 }
⟨latexrelease⟩    \prop_gclear_new:c { g__hook_#1_code_prop }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_use_once_set:n` *and* `\__hook_use_once_clear:n`.)

`\__hook_if_execute_immediately_p:n`
`\__hook_if_execute_immediately:nTF`

To check whether the code being added should be executed immediately (that is, if the hook is a one-time hook), we check if `\g__hook_⟨hook⟩_reversed_tl` is `I`. The gymnastics around `\if:w` is there to allow the `reversed` token list to be empty.

```
\prg_new_conditional:Npnn \__hook_if_execute_immediately:n #1 { T, F, TF }
  {
    \exp_after:wN \__hook_use_none_delimit_by_s_mark:w
    \if:w I
```

```
        \if_cs_exist:w g__hook_#1_reversed_tl \cs_end:
          \cs:w g__hook_#1_reversed_tl \exp_after:wN \cs_end:
        \fi:
        X
      \s__hook_mark \prg_return_true:
    \else:
      \s__hook_mark \prg_return_false:
    \fi:
  }
```

(*End of definition for* `\__hook_if_execute_immediately:nTF`.)

4.10 Querying a hook

Simpler data types, like token lists, have three possible states; they can exist and be empty, exist and be non-empty, and they may not exist, in which case emptiness doesn't apply (though `\tl_if_empty:N` returns false in this case).

Hooks are a bit more complicated: they have several other states as discussed in 4.4.2. A hook may exist or not, and either way it may or may not be empty (even a hook that doesn't exist may be non-empty) or may be disabled.

A hook is said to be empty when no code was added to it, either to its permanent code pool, or to its "next" token list. The hook doesn't need to be declared to have code added to its code pool (it may happen that a package A defines a hook `foo`, but it's loaded after package B, which adds some code to that hook. In this case it is important that the code added by package B is remembered until package A is loaded).

All other states can only be queried with internal tests as the different states are irrelevant for package code.

`\hook_if_empty_p:n`
`\hook_if_empty:nTF`

Test if a hook is empty (that is, no code was added to that hook). A ⟨*hook*⟩ being empty means that all three of its `\g__hook_`⟨*hook*⟩`_code_prop`, its `\__hook_toplevel␣`⟨*hook*⟩ and its `\__hook_next␣`⟨*hook*⟩ are empty.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_if_empty:n}
⟨latexrelease⟩                    {Hooks~with~args}
\prg_new_conditional:Npnn \hook_if_empty:n #1 { p , T , F , TF }
  {
    \if:w
        T
        \prop_if_exist:cT { g__hook_#1_code_prop }
          { \prop_if_empty:cF { g__hook_#1_code_prop } { F } }
        \__hook_cs_if_empty:cF { __hook_toplevel~#1 } { F }
        \__hook_cs_if_empty:cF { __hook_next~#1 } { F }
        T
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_if_empty:n}
⟨latexrelease⟩                    {Hooks~with~args}
⟨latexrelease⟩\prg_new_conditional:Npnn \hook_if_empty:n #1 { p , T , F , TF }
⟨latexrelease⟩  {
```

```
⟨latexrelease⟩    \__hook_if_structure_exist:nTF {#1}
⟨latexrelease⟩      {
⟨latexrelease⟩        \bool_lazy_and:nnTF
⟨latexrelease⟩            { \prop_if_empty_p:c { g__hook_#1_code_prop } }
⟨latexrelease⟩            {
⟨latexrelease⟩              \bool_lazy_and_p:nn
⟨latexrelease⟩                { \tl_if_empty_p:c { __hook_toplevel~#1 } }
⟨latexrelease⟩                { \tl_if_empty_p:c { __hook_next~#1 } }
⟨latexrelease⟩            }
⟨latexrelease⟩          { \prg_return_true: }
⟨latexrelease⟩          { \prg_return_false: }
⟨latexrelease⟩      }
⟨latexrelease⟩      { \prg_return_true: }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_if_empty:nTF`. *This function is documented on page* 18.)

`\__hook_if_usable_p:n`
`\__hook_if_usable:nTF`

A hook is usable if the token list that stores the sorted code for that hook, `\__hook␣⟨hook⟩`, exists. The property list `\g__hook_⟨hook⟩_code_prop` cannot be used here because often it is necessary to add code to a hook without knowing if such hook was already declared, or even if it will ever be (for example, in case the package that defines it isn't loaded).

```
\prg_new_conditional:Npnn \__hook_if_usable:n #1 { p , T , F , TF }
  {
    \cs_if_exist:cTF { __hook~#1 }
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_if_usable:nTF`.)

`\__hook_if_structure_exist_p:n`
`\__hook_if_structure_exist:nTF`

An internal check if the hook has already its basic internal structure set up with `\__hook_init_structure:n`. This means that the hook was already used somehow (a code chunk or rule was added to it), but it still wasn't declared with `\hook_new:n`.

```
\prg_new_conditional:Npnn \__hook_if_structure_exist:n #1 { p , T , F , TF }
  {
    \prop_if_exist:cTF { g__hook_#1_code_prop }
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_if_structure_exist:nTF`.)

`\__hook_if_declared_p:n`
`\__hook_if_declared:nTF`

Internal test to check if the hook was officially declared with `\hook_new:n` or a variant.

```
\prg_new_conditional:Npnn \__hook_if_declared:n #1 { p, T, F, TF }
  {
    \tl_if_exist:cTF { g__hook_#1_declared_tl }
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_if_declared:nTF`.)

__hook_if_reversed_p:n
__hook_if_reversed:nTF

An internal conditional that checks if a hook is reversed.

```
\prg_new_conditional:Npnn \__hook_if_reversed:n #1 { p , T , F , TF }
  {
    \exp_after:wN \__hook_use_none_delimit_by_s_mark:w
    \if:w - \cs:w g__hook_#1_reversed_tl \cs_end:
      \s__hook_mark \prg_return_true:
    \else:
      \s__hook_mark \prg_return_false:
    \fi:
  }
```

(*End of definition for* `\__hook_if_reversed:nTF`.)

__hook_if_generic_p:n
__hook_if_generic:nTF
__hook_if_deprecated_generic_p:n
__hook_if_deprecated_generic:nTF

An internal conditional that checks if a name belongs to a generic hook. The deprecated version needs to check if `#3` is empty to avoid returning true on `file/before`, for example.

```
\prg_new_conditional:Npnn \__hook_if_generic:n #1 { T, TF }
  { \__hook_if_generic:w #1 / / / \s__hook_mark }
\cs_new:Npn \__hook_if_generic:w #1 / #2 / #3 / #4 \s__hook_mark
  {
    \cs_if_exist:cTF { c__hook_generic_#1/./#3_tl }
      { \prg_return_true: }
      { \prg_return_false: }
  }
\prg_new_conditional:Npnn \__hook_if_deprecated_generic:n #1 { T, TF }
  { \__hook_if_deprecated_generic:w #1 / / / \s__hook_mark }
\cs_new:Npn \__hook_if_deprecated_generic:w #1 / #2 / #3 / #4 \s__hook_mark
  {
    \cs_if_exist:cTF { c__hook_deprecated_#1/./#2_tl }
      {
        \tl_if_empty:nTF {#3}
          { \prg_return_false: }
          { \prg_return_true: }
      }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_if_generic:nTF` *and* `\__hook_if_deprecated_generic:nTF`.)

__hook_if_cmd_hook_p:n
__hook_if_cmd_hook:nTF
__hook_if_cmd_hook_p:w
__hook_if_cmd_hook:wTF

An internal conditional that checks if a given hook is a valid generic `cmd` hook.

```
⟨!latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_if_cmd_hook:n}
⟨!latexrelease⟩                    {Hooks~with~args}
\prg_new_conditional:Npnn \__hook_if_cmd_hook:n #1 { T }
  { \__hook_if_cmd_hook:w #1 / / / \s__hook_mark }
\cs_new:Npn \__hook_if_cmd_hook:w #1 / #2 / #3 / #4 \s__hook_mark
  {
    \if:w Y
          \str_if_eq:nnF {#1} { cmd } { N }
          \tl_if_exist:cF { c__hook_generic_#1/./#3_tl } { N }
          Y
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_if_cmd_hook:n}
⟨latexrelease⟩                    {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_if_cmd_hook:nT
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_if_cmd_hook:nTF` *and* `\__hook_if_cmd_hook:wTF`.)

`\__hook_if_generic_reversed_p:n`
`\__hook_if_generic_reversed:nTF`

An internal conditional that checks if a name belongs to a generic reversed hook.

```
\prg_new_conditional:Npnn \__hook_if_generic_reversed:n #1 { T }
  { \__hook_if_generic_reversed:w #1 / / / \scan_stop: }
\cs_new:Npn \__hook_if_generic_reversed:w #1 / #2 / #3 / #4 \scan_stop:
  {
    \if_charcode:w - \cs:w c__hook_generic_#1/./#3_tl \cs_end:
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
```

(*End of definition for* `\__hook_if_generic_reversed:nTF`.)

`\__hook_if_replacing_args:TF`
`\__hook_misused_if_replacing_args:nn`
`\__hook_replacing_args_true:`
`\__hook_replacing_args_false:`
`\__hook_replacing_args_reset:`
`\g__hook_replacing_stack_seq`

An internal conditional that checks if the code being added to the hook contains arguments.

```
\seq_new:N \g__hook_replacing_stack_seq
\cs_new:Npn \__hook_misused_if_replacing_args:nn #1 #2
  {
    \msg_expandable_error:nnn { latex2e } { should-not-happen }
      { Misused~\__hook_if_replacing_args:. }
  }
\cs_new:Npn \__hook_if_replacing_args:TF
  { \__hook_misused_if_replacing_args:nn }
\cs_new_protected:Npn \__hook_replacing_args_true:
  {
    \seq_gpush:No \g__hook_replacing_stack_seq
      { \__hook_if_replacing_args:TF }
    \cs_set:Npn \__hook_if_replacing_args:TF { \use_i:nn }
  }
\cs_new_protected:Npn \__hook_replacing_args_false:
  {
    \seq_gpush:No \g__hook_replacing_stack_seq
      { \__hook_if_replacing_args:TF }
    \cs_set:Npn \__hook_if_replacing_args:TF { \use_ii:nn }
  }
\cs_new_protected:Npn \__hook_replacing_args_reset:
  {
    \seq_gpop:NN \g__hook_replacing_stack_seq \l__hook_return_tl
    \cs_gset_eq:NN \__hook_if_replacing_args:TF \l__hook_return_tl
  }
```

(*End of definition for* `\__hook_if_replacing_args:TF` *and others.*)

4.11 Messages

Hook errors are LaTeX kernel errors:

```
\prop_gput:Nnn \g_msg_module_type_prop { hooks } { LaTeX }
```

And so are kernel errors (this should move elsewhere eventually).

```
\prop_gput:Nnn \g_msg_module_type_prop { latex2e } { LaTeX }
\prop_gput:Nnn \g_msg_module_name_prop { latex2e } { kernel }

\msg_new:nnnn { hooks } { labels-incompatible }
  {
    Labels~'#1'~and~'#2'~are~incompatible
    \str_if_eq:nnF {#3} {??} { ~in~hook~'#3' } .~
    \int_compare:nNnTF {#4} = { 1 }
      { The~ code~ for~ both~ labels~ will~ be~ dropped. }
      { You~ may~ see~ errors~ later. }
  }
  { LaTeX~found~two~incompatible~labels~in~the~same~hook.~
    This~indicates~an~incompatibility~between~packages.  }

\msg_new:nnnn { hooks } { exists }
    { Hook~'#1'~ has~ already~ been~ declared. }
    { There~ already~ exists~ a~ hook~ declaration~ with~ this~
      name.\\
      Please~ use~ a~ different~ name~ for~ your~ hook.}

⟨latexrelease⟩\IncludeInRelease{2023/06/01}{too-many-args}
⟨latexrelease⟩                    {Hooks~with~args}

\msg_new:nnnn { hooks } { too-many-args }
  { Too~many~arguments~for~hook~'#1'. }
  {
    You~tried~to~declare~a~hook~with~#2~arguments,~but~a~
    hook~can~only~have~up~to~nine.~LaTeX~will~define~this~
    hook~with~nine~arguments.
  }

\msg_new:nnnn { hooks } { without-args }
  { Hook~'#1'~has~no~arguments. }
  {
    You~tried~to~use~\iow_char:N\\#2WithArguments~
    on~a~hook~that~takes~no~arguments.\\
    Check~the~usage~of~the~hook~or~use~\iow_char:N\\#2~instead.\\
    \\
    LaTeX~will~use~\iow_char:N\\#2.
  }

\msg_new:nnnn { hooks } { one-time-args }
  { You~can't~have~arguments~in~used~one-time~hook~'#1'. }
  {
    You~tried~to~use~\iow_char:N\\#2WithArguments~
    on~a~one-time~hook~that~has~already~been~used.~
    You~have~to~add~the~code~before~the~hook~is~used,~
    or~add~the~code~without~arguments~using~\iow_char:N\\#2~instead.\\
    \\
    LaTeX~will~use~\iow_char:N\\#2.
  }
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{too-many-args}
⟨latexrelease⟩                    {Hooks~with~args}
⟨latexrelease⟩\EndIncludeInRelease
\msg_new:nnnn { hooks } { hook-disabled }
  { Cannot~add~code~to~disabled~hook~'#1'. }
  {
    The~hook~'#1'~you~tried~to~add~code~to~was~previously~disabled~
    with~\iow_char:N\\hook_disable_generic:n~or~
    \iow_char:N\\DisableGenericHook,~so~
    it~cannot~have~code~added~to~it.
  }
\msg_new:nnn { hooks } { empty-label }
  {
    Empty~code~label~\msg_line_context:.~
    Using~'\__hook_currname_or_default:'~instead.
  }
\msg_new:nnn { hooks } { empty-hook }
  {
    Empty~hook~name~\msg_line_context:.
  }
\msg_new:nnn { hooks } { no-default-label }
  {
    Missing~(empty)~default~label~\msg_line_context:. \\
    This~command~was~ignored.
  }
\msg_new:nnnn { hooks } { unknown-rule }
  {
    Unknown~ relationship~ '#3'~
    between~ labels~ '#2'~ and~ '#4'~
    \str_if_eq:nnF {#1} {??} { ~in~hook~'#1' }. ~
    Perhaps~ a~ misspelling?
  }
  {
    The~ relation~ used~ not~ known~ to~ the~ system.~ Allowed~ values~ are~
    'before'~ or~ '<',~
    'after'~ or~ '>',~
    'incompatible-warning',~
    'incompatible-error',~
    'voids'~ or~
    'unrelated'.
  }
\msg_new:nnnn { hooks } { rule-too-late }
  {
    Sorting~rule~for~'#1'~hook~applied~too~late.\\
    Try~setting~this~rule~earlier.
  }
  {
    You~tried~to~set~the~ordering~of~hook~'#1'~using\\
    \ \ \iow_char:N\\DeclareHookRule{#1}{#2}{#3}{#4}\\
    but~hook~'#1'~was~already~used~as~a~one-time~hook,~
```

```
    thus~sorting~is\\
    no~longer~possible.~Declare~the~rule~
    before~the~hook~is~used.
  }
\msg_new:nnnn { hooks } { misused-top-level }
  {
    Illegal~use~of~\iow_char:N \\AddToHook{#1}[top-level]{...}.\\
    'top-level'~is~reserved~for~the~user's~document.
  }
  {
    The~'top-level'~label~is~meant~for~user~code~only,~and~should~only~
    be~used~(sparingly)~in~the~main~document.~Use~the~default~label~
    '\__hook_currname_or_default:'~for~this~\@cls@pkg,~or~another~
    suitable~label.
  }
\msg_new:nnn { hooks } { set-top-level }
  {
    You~cannot~change~the~default~label~#1~'top-level'.~Illegal \\
    \use:nn { ~ } { ~ } \iow_char:N \\#2{#3} \\
    \msg_line_context:.
  }
\msg_new:nnn { hooks } { extra-pop-label }
  {
    Extra~\iow_char:N \\PopDefaultHookLabel. \\
    This~command~will~be~ignored.
  }
\msg_new:nnn { hooks } { missing-pop-label }
  {
    Missing~\iow_char:N \\PopDefaultHookLabel. \\
    The~label~'#1'~was~pushed~but~never~popped.~Something~is~wrong.
  }
\msg_new:nnn { latex2e } { should-not-happen }
  {
    This~should~not~happen.~#1 \\
    Please~report~at~https://github.com/latex3/latex2e.
  }
\msg_new:nnn { hooks } { activate-disabled }
  {
    Cannot~ activate~ hook~ '#1'~ because~ it~ is~ disabled!
  }
\msg_new:nnn { hooks } { cannot-remove }
  {
    Cannot~remove~chunk~'#2'~from~hook~'#1'~because~
    \__hook_if_structure_exist:nTF {#1}
      { it~does~not~exist~in~that~hook. }
      { the~hook~does~not~exist. }
  }
\msg_new:nnn { hooks } { generic-deprecated }
  {
    Generic~hook~'#1/#2/#3'~is~deprecated. \\
    Use~hook~'#1/#3/#2'~instead.
  }
```

4.12 LaTeX 2ε package interface commands

\NewHook
\NewReversedHook
\NewMirroredHookPair

Declaring new hooks ...

```
\NewDocumentCommand \NewHook             { m }
  { \hook_new:n {#1} }
\NewDocumentCommand \NewReversedHook     { m }
  { \hook_new_reversed:n {#1} }
\NewDocumentCommand \NewMirroredHookPair { mm }
  { \hook_new_pair:nn {#1}{#2} }
```

(*End of definition for* `\NewHook`, `\NewReversedHook`, *and* `\NewMirroredHookPair`. *These functions are documented on page 3.*)

\NewHookWithArguments
\NewReversedHookWithArguments
\NewMirroredHookPairWithArguments

Declaring new hooks with arguments...

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\NewHookWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
\NewDocumentCommand \NewHookWithArguments             { mm }
  { \hook_new_with_args:nn {#1} {#2} }
\NewDocumentCommand \NewReversedHookWithArguments     { mm }
  { \hook_new_reversed_with_args:nn {#1} {#2} }
\NewDocumentCommand \NewMirroredHookPairWithArguments { mmm }
  { \hook_new_pair_with_args:nnn {#1} {#2} {#3} }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\NewHookWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \NewHookWithArguments #1 #2 { }
⟨latexrelease⟩\cs_new_protected:Npn \NewReversedHookWithArguments #1 #2 { }
⟨latexrelease⟩\cs_new_protected:Npn \NewMirroredHookPairWithArguments #1 #2 #3{}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\NewHookWithArguments`, `\NewReversedHookWithArguments`, *and* `\NewMirroredHookPairWithArgument` *These functions are documented on page 3.*)

```
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\ActivateGenericHook}
⟨latexrelease⟩                  {Providing~hooks}
```

\ActivateGenericHook

Providing new hooks ...

```
\NewDocumentCommand \ActivateGenericHook { m }
  { \hook_activate_generic:n {#1} }
```

(*End of definition for* `\ActivateGenericHook`. *This function is documented on page 4.*)

\DisableGenericHook

Disabling a generic hook.

```
\NewDocumentCommand \DisableGenericHook { m }
  { \hook_disable_generic:n {#1} }
```

(*End of definition for* `\DisableGenericHook`. *This function is documented on page 4.*)

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\ActivateGenericHook}
⟨latexrelease⟩                  {Providing~hooks}
⟨latexrelease⟩\def \ActivateGenericHook #1 { }
⟨latexrelease⟩\def \DisableGenericHook #1 { }
⟨latexrelease⟩\EndIncludeInRelease
```

\AddToHook
\AddToHookWithArguments

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\AddToHookWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
\NewDocumentCommand \AddToHook { m o +m }
  { \hook_gput_code:nnn {#1} {#2} {#3} }
\NewDocumentCommand \AddToHookWithArguments { m o +m }
  { \hook_gput_code_with_args:nnn {#1} {#2} {#3} }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\AddToHookWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \AddToHookWithArguments #1 #2 #3 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\AddToHook` *and* `\AddToHookWithArguments`*. These functions are documented on page 6.*)

\AddToHookNext
\AddToHookNextWithArguments

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\AddToHookNextWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
\NewDocumentCommand \AddToHookNext { m +m }
  { \hook_gput_next_code:nn {#1} {#2} }
\NewDocumentCommand \AddToHookNextWithArguments { m +m }
  { \hook_gput_next_code_with_args:nn {#1} {#2} }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\AddToHookNextWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \AddToHookNextWithArguments #1 #2 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\AddToHookNext` *and* `\AddToHookNextWithArguments`*. These functions are documented on page 7.*)

\ClearHookNext

```
\NewDocumentCommand \ClearHookNext { m }
  { \hook_gclear_next_code:n {#1} }
```

(*End of definition for* `\ClearHookNext`*. This function is documented on page 8.*)

\RemoveFromHook

```
\NewDocumentCommand \RemoveFromHook { m o }
  { \hook_gremove_code:nn {#1} {#2} }
```

(*End of definition for* `\RemoveFromHook`*. This function is documented on page 6.*)

\SetDefaultHookLabel
\PushDefaultHookLabel
\PopDefaultHookLabel

Now define a wrapper that replaces the top of the stack with the argument, and updates `\g__hook_hook_curr_name_tl` accordingly.

```
\NewDocumentCommand \SetDefaultHookLabel { m }
  { \__hook_set_default_hook_label:n {#1} }
```

The label is only automatically updated with `\@onefilewithoptions` (`\usepackage` and `\documentclass`), but some packages, like Ti*k*Z, define package-like interfaces, like `\usetikzlibrary` that are wrappers around `\input`, so they inherit the default label currently in force (usually `top-level`, but it may change if loaded in another package). To provide a package-like behavior also for hooks in these files, we provide high-level access to the default label stack.

```
\NewDocumentCommand \PushDefaultHookLabel { m }
  { \__hook_curr_name_push:n {#1} }
\NewDocumentCommand \PopDefaultHookLabel { }
  { \__hook_curr_name_pop: }
```

The current label stack holds the labels for all files but the current one (more or less like `\@currnamestack`), and the current label token list, `\g__hook_hook_curr_name_tl`, holds the label for the current file. However `\@pushfilename` happens before `\@currname` is set, so we need to look ahead to get the `\@currname` for the label. expl3 also requires the current file in `\@pushfilename`, so here we abuse `\@expl@push@filename@aux@@` to do `\__hook_curr_name_push:n`.

```
\cs_gset_protected:Npn \@expl@push@filename@aux@@ #1#2#3
  {
    \__hook_curr_name_push:n {#3}
    \str_gset:Ne \g_file_curr_name_str {#3}
    #1 #2 {#3}
  }
```

(*End of definition for* `\SetDefaultHookLabel`, `\PushDefaultHookLabel`, *and* `\PopDefaultHookLabel`. *These functions are documented on page* 10.)

`\UseHook`
`\UseOneTimeHook`
`\UseHookWithArguments`
`\UseOneTimeHookWithArguments`

Avoid the overhead of xparse and its protection that we don't want here (since the hook should vanish without trace if empty)!

```
⟨!latexrelease⟩\IncludeInRelease{2023/06/01}{\UseHookWithArguments}
⟨!latexrelease⟩                  {Hooks~with~args}
\cs_new:Npn \UseHook        { \hook_use:n }
\cs_new:Npn \UseOneTimeHook { \hook_use_once:n }
\cs_new:Npn \UseHookWithArguments        { \hook_use:nnw }
\cs_new:Npn \UseOneTimeHookWithArguments { \hook_use_once:nnw }
⟨!latexrelease⟩\EndIncludeInRelease
⟨!latexrelease⟩\IncludeInRelease{2020/10/01}{\UseHookWithArguments}
⟨!latexrelease⟩                  {Hooks~with~args}
⟨!latexrelease⟩\cs_new:Npn \UseHookWithArguments #1 #2 { }
⟨!latexrelease⟩\cs_new:Npn \UseOneTimeHookWithArguments #1 #2 { }
⟨!latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\UseHook` *and others. These functions are documented on page* 4.)

`\ShowHook`
`\LogHook`

```
\cs_new_protected:Npn \ShowHook { \hook_show:n }
\cs_new_protected:Npn \LogHook { \hook_log:n }
```

(*End of definition for* `\ShowHook` *and* `\LogHook`. *These functions are documented on page* 14.)

`\DebugHooksOn`
`\DebugHooksOff`

```
\cs_new_protected:Npn \DebugHooksOn  { \hook_debug_on:  }
\cs_new_protected:Npn \DebugHooksOff { \hook_debug_off: }
```

(*End of definition for* `\DebugHooksOn` *and* `\DebugHooksOff`. *These functions are documented on page* 15.)

`\DeclareHookRule`

```
\NewDocumentCommand \DeclareHookRule { m m m m }
                    { \hook_gset_rule:nnnn {#1}{#2}{#3}{#4} }
```

(*End of definition for* `\DeclareHookRule`. *This function is documented on page 12.*)

`\DeclareDefaultHookRule` This declaration is only supported before `\begin{document}`.

```
\NewDocumentCommand \DeclareDefaultHookRule { m m m }
                    { \hook_gset_rule:nnnn {??}{#1}{#2}{#3} }
\@onlypreamble\DeclareDefaultHookRule
```

(*End of definition for* `\DeclareDefaultHookRule`. *This function is documented on page 13.*)

`\ClearHookRule` A special setup rule that removes an existing relation. Basically `\__hook_rule_gclear:nnn` plus fixing the property list for debugging.

FMi: Needs perhaps an L3 interface, or maybe it should get dropped?

```
\NewDocumentCommand \ClearHookRule { m m m }
{ \hook_gset_rule:nnnn {#1}{#2}{unrelated}{#3} }
```

(*End of definition for* `\ClearHookRule`. *This function is documented on page 12.*)

`\IfHookEmptyTF` `\IfHookEmptyT` `\IfHookEmptyF` Here we avoid the overhead of xparse, since `\IfHookEmptyTF` is used in `\end` (that is, every LaTeX environment). As a further optimization, use `\let` rather than `\def` to avoid one expansion step.

```
\cs_new_eq:NN \IfHookEmptyTF \hook_if_empty:nTF
\cs_new_eq:NN \IfHookEmptyT \hook_if_empty:nT
\cs_new_eq:NN \IfHookEmptyF \hook_if_empty:nF
```

(*End of definition for* `\IfHookEmptyTF`, `\IfHookEmptyT`, *and* `\IfHookEmptyF`. *These functions are documented on page 14.*)

`\IfHookExistsTF` Marked for removal and no longer documented in the doc section!

PhO: `\IfHookExistsTF` *is used in* `jlreq.cls`, `pxatbegshi.sty`, `pxeverysel.sty`, `pxeveryshi.sty`, *so the public name may be an alias of the internal conditional for a while. Regardless, those packages' use for* `\IfHookExistsTF` *is not really correct and can be changed.*

```
\cs_new_eq:NN \IfHookExistsTF \__hook_if_usable:nTF
```

(*End of definition for* `\IfHookExistsTF`.)

4.13 Deprecated that needs cleanup at some point

`\hook_disable:n` `\hook_provide:n` `\hook_provide_reversed:n` `\hook_provide_pair:nn` `\__hook_activate_generic_reversed:n` `\__hook_activate_generic_pair:nn` Deprecated.

```
\cs_new_protected:Npn \hook_disable:n
  {
    \__hook_deprecated_warn:nn
      { hook_disable:n }
      { hook_disable_generic:n }
    \hook_disable_generic:n
  }
\cs_new_protected:Npn \hook_provide:n
  {
    \__hook_deprecated_warn:nn
      { hook_provide:n }
      { hook_activate_generic:n }
    \hook_activate_generic:n
  }
\cs_new_protected:Npn \hook_provide_reversed:n
```

```
  {
    \__hook_deprecated_warn:nn
      { hook_provide_reversed:n }
      { hook_activate_generic:n }
    \__hook_activate_generic_reversed:n
  }
\cs_new_protected:Npn \hook_provide_pair:nn
  {
    \__hook_deprecated_warn:nn
      { hook_provide_pair:nn }
      { hook_activate_generic:n }
    \__hook_activate_generic_pair:nn
  }
\cs_new_protected:Npn \__hook_activate_generic_reversed:n #1
  { \__hook_normalize_hook_args:Nn \__hook_activate_generic:nn {#1} { - } }
\cs_new_protected:Npn \__hook_activate_generic_pair:nn #1#2
  { \hook_activate_generic:n {#1} \__hook_activate_generic_reversed:n {#2} }
```

(*End of definition for* `\hook_disable:n` *and others.*)

`\DisableHook` `\ProvideHook` `\ProvideReversedHook` `\ProvideMirroredHookPair`

Deprecated.

```
\cs_new_protected:Npn \DisableHook
  {
    \__hook_deprecated_warn:nn
      { DisableHook }
      { DisableGenericHook }
    \hook_disable_generic:n
  }
\cs_new_protected:Npn \ProvideHook
  {
    \__hook_deprecated_warn:nn
      { ProvideHook }
      { ActivateGenericHook }
    \hook_activate_generic:n
  }
\cs_new_protected:Npn \ProvideReversedHook
  {
    \__hook_deprecated_warn:nn
      { ProvideReversedHook }
      { ActivateGenericHook }
    \__hook_activate_generic_reversed:n
  }
\cs_new_protected:Npn \ProvideMirroredHookPair
  {
    \__hook_deprecated_warn:nn
      { ProvideMirroredHookPair }
      { ActivateGenericHook }
    \__hook_activate_generic_pair:nn
  }
```

(*End of definition for* `\DisableHook` *and others.*)

`\__hook_deprecated_warn:nn` Warns about a deprecation, telling what should be used instead.

```
\cs_new_protected:Npn \__hook_deprecated_warn:nn #1 #2
  { \msg_warning:nnnn { hooks } { deprecated } {#1} {#2} }
```

```
\msg_new:nnn { hooks } { deprecated }
  {
    Command~\iow_char:N\\#1~is~deprecated~and~will~be~removed~in~a~
    future~release. \\ \\
    Use~\iow_char:N\\#2~instead.
  }
```

(*End of definition for* `\__hook_deprecated_warn:nn`.)

4.14 Internal commands needed elsewhere

Here we set up a few horrible (but consistent) LaTeX 2_ε names to allow for internal commands to be used outside this module. We have to unset the `@@` since we want double "at" sign in place of double underscores.

```
⟨@@=⟩
```

`\@expl@@@initialize@all@@`
`\@expl@@@hook@curr@name@pop@@`

```
\cs_new_eq:NN \@expl@@@initialize@all@@
              \__hook_initialize_all:
\cs_new_eq:NN \@expl@@@hook@curr@name@pop@@
              \__hook_curr_name_pop:
```

(*End of definition for* `\@expl@@@initialize@all@@` *and* `\@expl@@@hook@curr@name@pop@@`.)

Rolling back here doesn't undefine the interface commands as they may be used in packages without rollback functionality. So we just make them do nothing which may or may not work depending on the code usage.

```
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{lthooks}
⟨latexrelease⟩                  {The~hook~management}%
⟨latexrelease⟩
⟨latexrelease⟩\def \NewHook#1{}
⟨latexrelease⟩\def \NewReversedHook#1{}
⟨latexrelease⟩\def \NewMirroredHookPair#1#2{}
⟨latexrelease⟩
⟨latexrelease⟩\def \DisableGenericHook #1{}
⟨latexrelease⟩
⟨latexrelease⟩\long\def\AddToHookNext#1#2{}
⟨latexrelease⟩
⟨latexrelease⟩\def\AddToHook#1{\@gobble@AddToHook@args}
⟨latexrelease⟩\providecommand\@gobble@AddToHook@args[2][]{}
⟨latexrelease⟩
⟨latexrelease⟩\def\RemoveFromHook#1{\@gobble@RemoveFromHook@arg}
⟨latexrelease⟩\providecommand\@gobble@RemoveFromHook@arg[1][]{}
⟨latexrelease⟩
⟨latexrelease⟩\def \UseHook        #1{}
⟨latexrelease⟩\def \UseOneTimeHook #1{}
⟨latexrelease⟩\def \ShowHook #1{}
⟨latexrelease⟩\let \DebugHooksOn \@empty
⟨latexrelease⟩\let \DebugHooksOff\@empty
⟨latexrelease⟩
⟨latexrelease⟩\def \DeclareHookRule #1#2#3#4{}
⟨latexrelease⟩\def \DeclareDefaultHookRule #1#2#3{}
⟨latexrelease⟩\def \ClearHookRule #1#2#3{}
```

If the hook management is not provided we make the test for existence false and the test for empty true in the hope that this is most of the time reasonable. If not a package would need to guard against running in an old kernel.

```
⟨latexrelease⟩\long\def \IfHookExistsTF #1#2#3{#3}
⟨latexrelease⟩\long\def \IfHookEmptyTF #1#2#3{#2}
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
⟨@@=hook⟩

⟨latexrelease⟩\cs:w __hook_rollback_tidying: \cs_end:
⟨latexrelease⟩\bool_lazy_and:nnT
⟨latexrelease⟩    { \int_compare_p:nNn { \sourceLaTeXdate } > { 20230600 } }
⟨latexrelease⟩    { \int_compare_p:nNn { \requestedLaTeXdate } < { 20230601 } }
⟨latexrelease⟩  {
⟨latexrelease⟩    \cs_gset_protected:Npn \__hook_rollback_tidying:
⟨latexrelease⟩      {
⟨latexrelease⟩        \@latex@error { Rollback~code~executed~twice }
⟨latexrelease⟩          {
⟨latexrelease⟩            Something~went~wrong~(unless~this~was~
⟨latexrelease⟩            done~on~purpose~in~a~testing~environment).
⟨latexrelease⟩          }
⟨latexrelease⟩        \use_none:nnnn
⟨latexrelease⟩      }
⟨latexrelease⟩    \cs_set:Npn \__hook_tmp:w #1 #2
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_tl_gset:ce { __hook#1~#2 }
⟨latexrelease⟩          {
⟨latexrelease⟩            \exp_args:No \exp_not:o
⟨latexrelease⟩              {
⟨latexrelease⟩                \cs:w __hook#1~#2 \exp_last_unbraced:Ne \cs_end:
⟨latexrelease⟩                  { \__hook_braced_cs_parameter:n
⟨latexrelease⟩                      { __hook#1~#2 } }
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩    \seq_map_inline:Nn \g__hook_all_seq
⟨latexrelease⟩      {
⟨latexrelease⟩        \exp_after:wN \cs_gset_nopar:Npn
⟨latexrelease⟩          \cs:w g__hook_#1_code_prop \exp_args:NNo \exp_args:No
⟨latexrelease⟩            \cs_end: { \cs:w g__hook_#1_code_prop \cs_end: }
⟨latexrelease⟩        \__hook_tmp:w { _toplevel } {#1}
⟨latexrelease⟩        \__hook_tmp:w { _next } {#1}
⟨latexrelease⟩      }
⟨latexrelease⟩  }
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨@@=⟩
```

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.

Symbols

\\ 1799, 1810, 2628, 2642, 2643, 2644, 2645, 2646, 2651, 2654, 2655, 2656, 2666, 2667, 2681, 2702, 2706, 2707, 2709, 2715, 2726, 2727, 2732, 2737, 2742, 2758, 2923, 2924, 2925
\⟨*addto-cmd*⟩ *5*
\␣ 2707
__hook\textvisiblespace␣\meta␣{hook} *34*
__hook_next\textvisiblespace␣\meta␣{hook} *34*
__hook_toplevel\textvisiblespace␣\meta␣{hook} *34*

A

A/foo (hook) *13*, *13*
\ActivateGenericHook 2782, 2784, 2789, 2791, *22*
\AddToHook 2794, 2944, *36*
\AddToHookNext 2805, 2942, *8*
\AddToHookNextWithArguments 2805, *8*
\AddToHookWithArguments 2794, *5*
after (hook) *27*
\AfterEndEnvironment *27*
\AtBeginDocument *25*
\AtBeginEnvironment *27*
\AtEndDocument *29*
\AtEndEnvironment *27*
\AtEndPreamble *28*

B

babel/⟨*language*⟩/afterextras (hook) . *22*
\BeforeBeginEnvironment *27*
\begin *3*
begindocument (hook) . . . *25*, *25*, *25*, *28*, *36*
begindocument/before (hook) *28*
begindocument/end (hook) *28*
\bfdefault *30*
bfseries (hook) *30*, *30*
\bfseries *30*
bfseries/defaults (hook) *30*, *30*
bool commands:
  \bool_gset_false:N 15
  \bool_gset_true:N 10
  \bool_if:NTF 1945, 1954, 2045, 2054, 21, 334
  \bool_lazy_and:nnTF 1947, 2047, 2497, 2965, 269
  \bool_lazy_and_p:nn 2500
  \bool_lazy_or:nnTF 908, 1898
  \bool_new:N 6, 24
  \bool_set_false:N 1937, 2037
  \bool_set_true:N 1934, 2034
  \bool_while_do:nn 1614, 1699
  \c_false_bool 430, 478, 391, 397, 405, *52*
  \c_true_bool 398, 406, 408

C

class (hook) *22*
\ClearHookNext 2816, *8*
\ClearHookRule 2853, 2958, *12*
\clearpage *29*
clist commands:
  \clist_gclear:N 1613, 1698
  \clist_gput_left:Nn 1517, 1552
  \clist_gput_right:Nn 1519, 1556
  \clist_if_empty:NTF 1964, 2065
  \clist_if_exist:NTF 115
  \clist_map_inline:nn 939, 948
  \clist_new:N 117, 134
  \clist_use:Nn 1966, 2067
cmd (hook) *55*, *60*, *16*, *22*, *22*, *101*
cmd/⟨*name*⟩/after (hook) *27*
cmd/⟨*name*⟩/before (hook) *27*
cs commands:
  \cs:w 1088, 1145, 1296, 1436, 1543, 1544, 1605, 1625, 1628, 1689, 1714, 1717, 1735, 1736, 1760, 1761, 1762, 1769, 1776, 2254, 2273, 2279, 2293, 2303, 2325, 2379, 2466, 2531, 2581, 2964, 2984, 2993, 2994, 375, *45*
  \cs_end: . . . 789, 1088, 1145, 1299, 1439, 1543, 1544, 1605, 1625, 1628, 1689, 1714, 1717, 1735, 1736, 1751, 1761, 1762, 1769, 1776, 2131, 2253, 2257, 2261, 2273, 2278, 2279, 2293, 2299, 2313, 2326, 2331, 2379, 2465, 2466, 2531, 2581, 2964, 2984, 2994, 375
  \cs_generate_variant:Nn . . . 1102, 1125, 1655, 1738, 37, 47, 48, 51, 60
  \cs_gset:Npe 1069
  \cs_gset:Npn 1101, 1121, 1651, 2092, 2336, 2406

\cs_gset_eq:NN 792, 1446, 1472, 1487, 1488, 1491, 2242, 2343, 2344, 2345, 2346, 2610
\cs_gset_nopar:Npe 44, 46
\cs_gset_nopar:Npn 2992
\cs_gset_protected:Npe 1183, 20
\cs_gset_protected:Npn 603, 606, 624, 650, 699, 707, 1470, 1531, 2187, 2190, 2218, 2223, 2241, 2400, 2430, 2826, 2969, 82, 85, 94, 127, 154, 182, 185, 191, 215, 220, 245, 304, 322
\cs_if_exist:NTF 1073, 1138, 1845, 2512, 2541, 2549
\cs_if_exist_p:N 271
\cs_if_exist_use:NTF 1305, 1349, 1374
\cs_new:Npe 565
\cs_new:Npn 573, 926, 928, 930, 1224, 1236, 1241, 1249, 1258, 1260, 1267, 1294, 1301, 1303, 1418, 1434, 1568, 1569, 1740, 1741, 2088, 2095, 2251, 2276, 2284, 2297, 2305, 2314, 2323, 2358, 2539, 2547, 2561, 2579, 2588, 2593, 2834, 2835, 2836, 2837, 2841, 2842, 39, 40, 324, 330, 345, 355, 356, 358, 372, 378
\cs_new_eq:NN 1208, 1391, 1398, 1441, 1815, 1816, 1817, 1818, 1819, 1820, 2313, 2855, 2856, 2857, 2858, 2928, 2930, 7, 23, 34, 57
\cs_new_protected:Npn . . 425, 433, 449, 457, 467, 482, 493, 499, 505, 521, 534, 577, 594, 654, 673, 681, 688, 720, 726, 732, 742, 784, 786, 794, 796, 799, 801, 982, 984, 1019, 1056, 1068, 1071, 1100, 1113, 1115, 1117, 1119, 1136, 1160, 1167, 1325, 1332, 1363, 1385, 1392, 1399, 1405, 1410, 1415, 1416, 1444, 1497, 1577, 1659, 1742, 1749, 1758, 1765, 1772, 1779, 1787, 1793, 1804, 1821, 1828, 1840, 1847, 1854, 1860, 1862, 1866, 1984, 2106, 2124, 2129, 2138, 2156, 2170, 2192, 2205, 2232, 2236, 2246, 2259, 2268, 2282, 2288, 2311, 2318, 2341, 2364, 2374, 2385, 2391, 2411, 2441, 2443, 2453, 2595, 2601, 2607, 2778, 2779, 2780, 2803, 2814, 2844, 2845, 2846, 2847, 2859, 2866, 2873, 2880, 2887, 2889, 2891, 2898, 2905, 2912, 2919, 8, 13, 18, 41, 43, 45, 49, 52, 58, 63, 65, 67, 98, 142, 166, 168, 170, 195, 197, 199, 225, 260, 262, 279, 284, 286, 379, 388, 393, 401
\cs_parameter_spec:N 1252
\cs_replacement_spec *63*
\cs_replacement_spec:N 1227, 1462, 1484, 1908, 1921, 1930
\cs_set:Npn 1169, 1180, 1187, 1196, 1905, 2599, 2605, 2978
\cs_set_eq:NN 1516, 1517, 1518, 1519, 1549, 1551, 1553, 1555, 1842, 1849, 1856
\cs_set_nopar:Npe 42
\cs_set_protected:Npn 414, 2110, 2141
\cs_to_str:N *45*
\cs_undefine:N 671, 1106, 1107, 1108, 1109, 1129, 1130, 1131, 1132, 1156, 1212, 1232, 1290, 1311, 1312, 1323, 1417, 2351, 2437, 2575, 73, 74, 84, 126, 184, 190, 265
\csname *8*

D

debug commands:
\debug_resume: *33*
\debug_suspend: *33*
\DebugHooksOff 2846, 2954, *15*
\DebugHooksOn 2846, 2953, *15*
\DeclareDefaultHookRule . 2850, 2957, *13*
\DeclareHookRule 2848, 2956, *7*
\def 2791, 2792, 2936, 2937, 2938, 2940, 2942, 2944, 2947, 2950, 2951, 2952, 2956, 2957, 2958, 2959, 2960, *109*
\DisableGenericHook . 2786, 2792, 2940, *36*
\DisableHook 2891, *4*
\DiscardShipoutBox *8*
\document *28*
\documentclass *107*

E

else commands:
\else: 789, 1063, 1220, 1282, 1423, 1430, 1438, 1754, 2300, 2328, 2470, 2486, 2533, 2568, 2583
\end *26*
\endcsname *8*
enddocument (hook) *24*, *25*, *29*
\enddocument *28*
enddocument/afteraux (hook) *29*
enddocument/afterlastpage (hook) . . . *29*
enddocument/end (hook) *29*
enddocument/info (hook) *29*
\EndIncludeInRelease 600, 651, 668, 672, 695, 716, 750, 808, 838, 870, 897, 900, 920, 923, 936, 954, 959, 962, 971, 976, 979, 1016, 1053, 1103, 1110, 1126,

1133, 1153, 1157, 1209, 1213, 1229, 1233, 1261, 1264, 1287, 1291, 1308, 1313, 1320, 1324, 1360, 1384, 1467, 1494, 1528, 1567, 1656, 1739, 1980, 2084, 2089, 2094, 2184, 2191, 2215, 2231, 2238, 2243, 2265, 2285, 2315, 2333, 2338, 2348, 2352, 2355, 2382, 2397, 2408, 2426, 2438, 2450, 2460, 2490, 2509, 2572, 2576, 2658, 2661, 2775, 2781, 2788, 2793, 2800, 2804, 2811, 2815, 2838, 2843, 79, 95, 123, 139, 151, 163, 179, 192, 212, 222, 242, 257, 275, 281, 301, 319, 323
\EndModuleRelease 2962
env (hook) *13*, *22*
env/⟨*env*⟩/after (hook) *26*, *26*, *27*
env/⟨*env*⟩/before (hook) *26*, *26*, *27*
env/⟨*env*⟩/begin (hook) *26*, *27*
env/⟨*env*⟩/end (hook) *26*, *27*
env/document/after (hook) *30*
env/document/before (hook) *28*
env/document/begin (hook) *28*
env/document/end (hook) *30*
env/quote/after (hook) *20*, *20*
\ERROR 1740, 1741
\errorstopmode *18*
exp commands:
\exp:w 779, 833, *55*
\exp_after:wN . . 420, 567, 778, 832, 1062, 1064, 1205, 1370, 1543, 1753, 1872, 1905, 1990, 2134, 2279, 2463, 2466, 2530, 2992, 50, 55, 374, 375, *55*
\exp_args:Nc 1226, 1239, 2220
\exp_args:NcV 591
\exp_args:Ne 427, 475, 551, 736, 738, 2368, 2369
\exp_args:NNe 1524, 1645
\exp_args:Nne 1882
\exp_args:Nnnv 251
\exp_args:NNo . 1086, 1143, 1734, 2993
\exp_args:NNV 1618, 1705
\exp_args:NNx 1562
\exp_args:No 1086, 1143, 1204, 2093, 2152, 2982, 2993
\exp_args:Nv . . 1271, 2022, 2030, 2132
\exp_end: 779, 833
\exp_last_unbraced:Ne 1088, 1121, 1145, 1238, 2984
\exp_last_unbraced:Nf . . . 1251, 1929
\exp_last_unbraced:NNf 1179
\exp_last_unbraced:NNNNo . . 2087, 377
\exp_not:N 567, 568, 569, 573, 574, 1173, 1177, 1196, 1199, 1200, 1202, 1246, 1508, 1510, 1646, 1648, 385
\exp_not:n 554, 557, 1070, 1094, 1193, 1197, 1203, 1204, 1652, 2982, 108, 235
\exp_stop_f: 1272, 1420, 1428
expand@font@defaults (hook) *30*
\expanded *45*
\ExplSyntaxOff 2999
\ExplSyntaxOn 3

F

fi commands:
\fi: 791, 1065, 1222, 1245, 1285, 1424, 1432, 1438, 1756, 2135, 2255, 2263, 2280, 2302, 2330, 2467, 2472, 2488, 2535, 2570, 2585
file (hook) *13*, *22*
file commands:
\g_file_curr_name_str 2829
file/before (hook) *101*
foo (hook) *99*

G

\g__hook_\meta␣{hook}_code_prop *34*
\g__hook_\meta␣{hook}_declared_tl . . *34*
\g__hook_\meta␣{hook}_parameter_tl . *34*
\g__hook_\meta␣{hook}_reversed_tl . . *34*
group commands:
\group_begin: 1172, 1185, 381
\group_end: 1176, 1195, 384

H

hook commands:
\hook_activate_generic:n 2785, 2871, 2890, 2903, 282, 282, 284, 302, 320, 322, *42*
\hook_debug_off: 2847, 7, 13, *18*
\hook_debug_on: 2846, 7, 8, *18*
\hook_disable:n 2859, 2859
\hook_disable_generic:n . . . 2787, 2864, 2896, 258, 258, 260, 276, 279, *16*
\hook_gclear_next_code:n 2232, 2232, 2817, *17*
\hook_gput_code:nnn 491, 491, 493, 601, 603, 703, 724, 2797, *17*
\hook_gput_code_with_args:nnn 491, 499, 650, 2799, *17*
\hook_gput_next_code:nn . 712, 730, 2154, 2154, 2156, 2185, 2187, 2808, *53*
\hook_gput_next_code_with_-args:nn 2170, 2190, 2810, *17*
\hook_gremove_code:nn 980, 980, 982, 1017, 2819, *17*
\hook_gset_rule:nnnn 1325, 1325, 2849, 2851, 2854, *18*
\hook_if_empty:n 2474, 2476, 2491, 2493

\hook_if_empty:nTF 1894, 2000, 2474, 2855, 2856, 2857, *8*
\hook_if_empty_p:n 1949, 2049, 2474, *18*
\hook_log:n 1840, 1840, 2845, *18*
\hook_new:n 883, 2762, 61, 63, 82, 217, *42*
\hook_new_pair:nn 2766, 193, 195, 215, *15*
\hook_new_pair_with_args:nn *16*
\hook_new_pair_with_args:nnn 2774, 193, 193, 197, 213, 220, *16*
\hook_new_reversed:n 2764, 164, 166, 182, 218, *15*
\hook_new_reversed_with_args:nn 2772, 164, 164, 168, 180, 191, *16*
\hook_new_with_args:nn 2770, 61, 61, 65, 80, 94, *16*
\hook_provide:n 2859, 2866
\hook_provide_pair:nn . . . 2859, 2880
\hook_provide_reversed:n . 2859, 2873
\hook_show:n 1840, 1847, 1854, 2844, *84*
\hook_use:n 1487, 2244, 2244, 2246, 2266, 2268, 2286, 2288, 2343, 2834, *97*
\hook_use:nnw . . . 2284, 2314, 2316, 2316, 2318, 2334, 2336, 2344, 2836, *16*
\hook_use_once:n 2383, 2385, 2400, 2835, *16*
\hook_use_once:nnw 2383, 2383, 2391, 2398, 2406, 2837, *16*

hook internal commands:

\c__hook_ 1317, 1321
\g__hook_??_code_prop 1314
\c__hook_??_parameter_tl 1314
\g__hook_??_reversed_tl 1314
\g__hook_⟨*hook*⟩_code_prop *34*
\g__hook_⟨*hook*⟩_labels_clist *38*
\g__hook_⟨*hook*⟩_reversed_tl *34*
__hook_activate_generic:n 282
__hook_activate_generic:nn 2888, 285, 286, 304
__hook_activate_generic_pair:nn 2859, 2885, 2889, 2917
__hook_activate_generic_-reversed:n 2859, 2878, 2887, 2890, 2910
\g__hook_all_seq 1448, 1475, 2990, 28, 102, 131
__hook_apply_-rule_->:nnn . . . 1815
__hook_apply_-rule_<-:nnn . . . 1815
__hook_apply_-rule_<:nnn 1815
__hook_apply_-rule_>:nnn 1815
__hook_apply_-rule_xE:nnn . . . 1815
__hook_apply_-rule_xW:nnn . . . 1815
__hook_apply_label_pair:nnn . . . 1595, 1596, 1677, 1678, 1742, 1742, *78*
__hook_apply_rule:nnn 1752, 1758, 1758, *81*
__hook_apply_rule:nnnN *82*
__hook_apply_rule_->:nnn 1793
__hook_apply_rule_<-:nnn 1793
__hook_apply_rule_<:nnn 1765
__hook_apply_rule_>:nnn 1765
__hook_apply_rule_xE:nnn 1779
__hook_apply_rule_xW:nnn 1779
__hook_braced_cs_parameter:n 1089, 1146, 1171, 1182, 1234, 1234, 1236, 1262, 2985
__hook_braced_hidden_loop:w 1234, 1238, 1241, 1247
__hook_braced_parameter:n 1265, 1265, 1267, 1288, 1290, 1509, 1511, 1646, 1648
__hook_braced_real_loop:w . . . 1265
__hook_chk_args_allowed:nn 507, 652, 652, 654, 669, 671, 2208, *92*
__hook_clear_next:n . 2212, 2228, 2233, 2234, 2234, 2236, 2239, 2241, *89*
__hook_clist_gput:Nn 1517, 1519, 1551, 1555, 1619, 1706, 1740, 1741
__hook_code_gset:nn . 1111, 1111, 1113, 1125, 1127, 1129, 1506, 2445, 112
__hook_code_gset_aux:nnn . 1074, 1111, 1114, 1116, 1118, 1119, 1132
__hook_code_gset_auxi:nnnn 1054, 1075, 1100, 1102, 1109, 1140, *62*
__hook_cs_end:w 1234, 1253, 1254, 1255, 1260
__hook_cs_gput_right:nnn 546, 1054, 1054, 1056, 1104, 1106, 2213, 2416
__hook_cs_gput_right_fast:nnn 1054, 1062, 1068, 1107
__hook_cs_gput_right_slow:nnn 1054, 1064, 1071, 1108
__hook_cs_if_empty:N 1214, 1216, 1230, 1232
__hook_cs_if_empty:NTF . . . 1214, 1919, 1926, 2209, 2211, 2482, 2483
__hook_cs_if_empty_p:N 1214
__hook_cs_parameter_count:N 1234, 1239, 1249
__hook_cs_parameter_count:w 1234, 1251, 1258, 1259
\l__hook_cur_hook_tl 1581, 1663, 1799, 1810, 29, *83*
__hook_curr_name_pop: 449, 2825, 2931, 411, *47*
__hook_curr_name_push:n 425, 2823, 2828, 411, *46*

__hook_curr_name_push_aux:n ... 427, 433, 411
__hook_currname_or_default: ... 543, 633, 2673, 2721, 327, 337, 341, 357, 358, 358, *44*
__hook_debug:n ... 536, 626, 1447, 1456, 1474, 1477, 1499, 1523, 1533, 1561, 1600, 1620, 1682, 1707, 1767, 1774, 1781, 1789, 1795, 1806, 2160, 2174, 7, 7, 20, *31*
\g__hook_debug_bool 6, 10, 15, 21
__hook_debug_gset: 7, 11, 16, 18
__hook_debug_label_data:N 1600, 1642, 1683, 1731, 1828, 1828
__hook_debug_print_rules:n 2138, 2138
__hook_declare_deprecated_-generic:NNn 778, 799, 832
__hook_declare_deprecated_-generic:NNw 794, 800, 801
__hook_deprecated_generic_-warn:n 777, 784, 831, 1009, 1045, 1336, 1367, 1870, 1988, *55*
__hook_deprecated_generic_-warn:Nn 784
__hook_deprecated_generic_-warn:Nw 784
__hook_deprecated_generic_-warn:w 785, 786
__hook_deprecated_warn:nn 2861, 2868, 2875, 2882, 2893, 2900, 2907, 2914, 2919, 2919
__hook_disable:n 258, 261, 262
__hook_do_deprecated_generic:Nn 794, 794, 1010, 1046, 1337, 1368, 1871, 1989
__hook_do_deprecated_generic:Nw 794, 795, 796
__hook_double_hashes:n 558, 1086, 1095, 1143, 1200, *67*
\c__hook_empty_tl 1306, 35
__hook_end_document_label_-check: 456, 457, 464, 411
__hook_file_hook_normalize:n 738, 921, 921, 924, 926, 2369, *58*
\l__hook_front_tl 1570, 1611, 1614, 1617, 1619, 1620, 1621, 1635, 1636, 1696, 1700, 1704, 1706, 1708, 1710, 1724, 1725
\c__hook_generic_⟨*type*⟩/./⟨*place*⟩_tl *55*
\c__hook_generic_class/./after_-tl 937
\c__hook_generic_class/./before_-tl 937
\c__hook_generic_cmd/./after_tl 937
\c__hook_generic_cmd/./before_tl 937
\c__hook_generic_env/./after_tl 937
\c__hook_generic_env/./before_tl 937
\c__hook_generic_env/./begin_tl 937
\c__hook_generic_env/./end_tl .. 937
\c__hook_generic_file/./after_tl 937
\c__hook_generic_file/./before_-tl 937
\c__hook_generic_include/./after_-tl 937
\c__hook_generic_include/./before_-tl 937
\c__hook_generic_include/./end_-tl 937
\c__hook_generic_package/./after_-tl 937
\c__hook_generic_package/./before_-tl 937
__hook_generic_parameter:n 1081, 1301, 1312
__hook_generic_parameter:w 1302, 1303
\c__hook_generics_file_prop 913, 960
\c__hook_generics_prop 849, 881, 937, 955, 957
\c__hook_generics_reversed_ii_-prop 858, 885, 960
\c__hook_generics_reversed_iii_-prop 862, 889, 960
__hook_gput_code:nnn 491, 496, 502, 505, 605, 606, 684
__hook_gput_code_store:nnn 491, 518, 521
__hook_gput_next_code:nn 691, 2159, 2173, 2189, 2192, 2192
__hook_gput_next_do:nn 692, 713, 730, 2198, 2203, 2203, 2205, 2216, 2218, *53*
__hook_gput_next_do:Nnn . 2220, 2223
__hook_gput_undeclared_hook:nnn 673, 673, 685, 704, 724, *53*
__hook_gremove_code:nn 980, 983, 984, 1011, 1019, 1047
__hook_gset_rule:nnnn 1325, 1327, 1330, 1332, 1337, 1361, 1363, 1368
__hook_hash_check:nTF . 491, 556, 565
__hook_hash_check_aux:w 491, 567, 573
\c__hook_hash_tl 1246, *67*
\c__hook_hashes_tl 1173, *66*

\g__hook_hook_curr_name_tl 411, 423, 444, 445, 452, 462, 463, 489, 32, 360, 370, *47*
__hook_hook_gput_code_do:nnn . . . 491, 525, 534, 613, 624, 676, 234, 251
__hook_if_cmd_hook:n 2557, 2559, 2573
__hook_if_cmd_hook:nTF 2100, 2557, 2575
__hook_if_cmd_hook:w . . . 2560, 2561
__hook_if_cmd_hook:wTF 2557
__hook_if_cmd_hook_p:n 2557
__hook_if_cmd_hook_p:w 2557
__hook_if_declared:n 2522
__hook_if_declared:nTF 658, 883, 1059, 1077, 2099, 2522, 69, 87, 172, 201, 204, 291, 309, *35*
__hook_if_declared_p:n 2522
__hook_if_deprecated_generic:n 2545
__hook_if_deprecated_generic:nTF 775, 829, 1007, 1043, 1334, 1365, 1868, 1986, 2537
__hook_if_deprecated_generic:w 2546, 2547
__hook_if_deprecated_generic_-p:n 2537
__hook_if_disabled:n 267
__hook_if_disabled:nTF 529, 617, 1880, 1892, 1975, 1998, 2078, 2194, 258, 288, 306, *36*
__hook_if_disabled_p:n 258
__hook_if_execute_immediately:n 2461
__hook_if_execute_immediately:nTF 508, 608, 1341, 2387, 2393, 2402, 2461
__hook_if_execute_immediately_-p:n 2461
__hook_if_file_hook:w . 898, 901, 903
__hook_if_file_hook:wTF 734, 898, 2366, *54*
__hook_if_file_hook_p:w 898
__hook_if_generic:n 2537
__hook_if_generic:nTF 758, 816, 1080, 1878, 1994, 2537
__hook_if_generic:w 2538, 2539
__hook_if_generic_p:n 2537
__hook_if_generic_reversed:n . 2577
__hook_if_generic_reversed:nTF 770, 824, 2577, 296, 314
__hook_if_generic_reversed:w 2578, 2579
__hook_if_generic_reversed_p:n 2577
__hook_if_label_case:nnnnn 1434, 1434, 1593, 1675, 2115
__hook_if_replacing_args: . . . 2591
__hook_if_replacing_args:TF 510, 553, 580, 656, 1093, 2587, 2593, 2598, 2599, 2604, 2605, 2610
__hook_if_reversed:n 2528
__hook_if_reversed:nTF 1515, 1548, 1915, 1955, 1957, 2016, 2055, 2058, 2528
__hook_if_reversed_p:n 2528
__hook_if_structure_exist:n . 2516
__hook_if_structure_exist:nTF 986, 1021, 2197, 2495, 2516, 2752, 144, 156, *35*
__hook_if_structure_exist_p:n 2516
__hook_if_usable:n 2510
__hook_if_usable:nTF 523, 537, 582, 611, 627, 760, 818, 851, 1003, 1039, 1502, 1537, 1890, 1914, 1973, 1996, 2014, 2076, 2161, 2175, 2419, 2510, 2858, 100, *60*
__hook_if_usable_p:n 1948, 2048, 2510
__hook_if_usable_use:n 2353, 2368, 2371, 2374, *96*
__hook_include_legacy_code_-chunk:n 1501, 1536, 120, 136, 223, 223, 225, 243, 245
__hook_init_structure:n 545, 635, 675, 1347, 1372, 2207, 113, 133, 140, 140, 142, 152, 154, *50*
__hook_initialize_all: . . . 1442, 1442, 1444, 1468, 1470, 1492, 2929
__hook_initialize_hook_code:n 1446, 1473, 1495, 1495, 1497, 1529, 1531, 2262, 2283, 2312, *81*
__hook_initialize_single:NNn 1521, 1559, 1575, 1575, 1577, 1655, 1657, 1659, *73*
\l__hook_label_0_tl 1570
__hook_label_if_exist_apply:nnnTF 1742, 1744, 1746, 1749, *81*
__hook_label_ordered:nn . . . 1426, *73*
__hook_label_ordered:nnTF 1389, 1396, 1403, 1426, *71*
__hook_label_ordered_p:nn . . . 1426
__hook_label_pair:nn 1388, 1395, 1402, 1407, 1412, 1417, 1418, 1418, 1824, 1825, *72*
\l__hook_labels_int 1570, 1580, 1584, 1616, 1638, 1662, 1666, 1702, 1727, *79*
\l__hook_labels_seq . . 1570, 1579, 1585, 1603, 1661, 1667, 1686, 1830
__hook_list_if_rule_exists:nnnTF 2106, 2126, 2127, 2129

__hook_list_one_rule:nnn 2106, 2117, 2118, 2124
__hook_list_rules:nn 1935, 2035, 2106, 2106, 2143, *90*
__hook_log:nN 1840, 1843, 1850, 1857, 1864, 1866, 1871, 1982, 1984, 1989
__hook_log_cmd:n 1842, 1849, 1856, 1861, 1863, 1875, 1993
__hook_log_line:n 1840, 1860, 1891, 1893, 1897, 1911, 1923, 1933, 1951, 1970, 1997, 1999, 2003, 2011, 2025, 2033, 2051, 2072
__hook_log_line_indent:n 1840, 1862, 1901, 1907, 1917, 1924, 1938, 1946, 2005, 2008, 2018, 2026, 2038, 2046
__hook_log_next_code:n 2030, 2085, 2085, 2090, 2092
__hook_log_next_code:w . . 1929, 2088
__hook_make_name:n 351, 357, 366, 372, 372, *45*
__hook_make_name:w . . . 372, 374, 378
__hook_make_usable:n 822, 855, 91, 127, 312
__hook_make_usable:nn 765, 768, 75, 96, 96, 98, 124, 126, 294, *38*
__hook_misused_if_replacing_-args:nn 2587, 2588, 2594
__hook_msg_pair_found:nnn 1767, 1774, 1781, 1789, 1797, 1808, 1821, 1821
\g__hook_name_stack_seq 412, 413, 417, 424, 444, 451, 459, 469, 32
__hook_new:n 83, 85, 187
__hook_new:nn 61, 64, 66, 67, 84, 175, 207
__hook_new_pair:nnn . . 196, 198, 199
__hook_new_reversed:n 183, 185
__hook_new_reversed:nn 164, 167, 169, 170, 184, 190, 208
__hook_next␣⟨*hook*⟩ *60*
__hook_next_gset:nn 992, 1111, 1117, 1131, 2212, 2237, 2446, 148
\c__hook_nine_parameters_tl 1082, 1197, 1906, 35, 108
__hook_normalize_code_pool:n . . . 1158, 1158, 1160, 1210, 1212, 114, *38*
__hook_normalize_cs_args:nn 1134, 1134, 1136, 1154, 1156, 110, 111, *38*
__hook_normalize_fn:nn 586, 1164, 1183, 1208, *65*
__hook_normalize_hook_args:Nn . . 1843, 1850, 1857, 2159, 2173, 2188, 2233, 2388, 2394, 2403, 2888, 64, 66, 83, 167, 169, 183, 261, 285, 379, 388
__hook_normalize_hook_args:Nnn . 496, 502, 604, 983, 196, 198, 379, 393
__hook_normalize_hook_args_-aux:Nn 379, 379, 390, 395, 403
__hook_normalize_hook_rule_-args:Nnnnn 1327, 379, 401
__hook_parameter:n . . 1122, 1180, 1292, 1292, 1294, 1309, 1311, 1885
\c__hook_parameter_cmd 972, 977
\c__hook_parameter_cmd/./after_-tl 972
\c__hook_parameter_cmd/./before_-tl 972
__hook_parse_dot_label:nN 430, 478, 328, 330, 330
__hook_parse_dot_label:w 330, 342, 345
__hook_parse_dot_label_aux:w 330, 348, 356
__hook_parse_dot_label_cleanup:w 330, 352, 355
__hook_parse_label_default:nN . . 324, 324, 391, 397, 398, 405, 406, 408
__hook_patch_cmd_or_delay:Nnn . . *55*
__hook_post_initialization_-defs: 1465, 2339, 2339, 2341, 2346, 2349, 2351
__hook_preamble_hook:n 1488, 1874, 1992, 2244, 2248, 2259, 2272, 2282, 2292, 2311, 2320, 2345, 2378, 2413, 2432, *96*
__hook_print_args:n 2095
__hook_print_args:nn . . . 1882, 2095
__hook_prop_gput_labeled_-cleanup:nnn 491, 551, 577
__hook_prop_gput_labeled_do:Nnn 591, 594
__hook_prop_gput_labeled_-do:Nnnn 491
\l__hook_rear_tl 1570, 1601, 1607, 1608, 1631, 1632, 1684, 1692, 1693, 1720, 1721
__hook_replacement_spec:N 1218, 1224
__hook_replacing_args_false: 495, 663, 1451, 2158, 2415, 2587, 2601, 231, *48*
__hook_replacing_args_reset: 497, 503, 1454, 2168, 2182, 2418, 2587, 2607, 237
__hook_replacing_args_true: 501, 1452, 2172, 2587, 2595
\g__hook_replacing_stack_seq . 2587

\l__hook_return_tl 451, 452, 459, 463, 579, 587, 592, 596, 597, 642, 645, 999, 1034, 1617, 1618, 1704, 1705, 2609, 2610, 25
__hook_rollback_tidying: 2969
__hook_rule_<_gset:nnn 1385
__hook_rule_>_gset:nnn 1385
__hook_rule_after_gset:nnn 1385, 1392, 1398
__hook_rule_before_gset:nnn 1385, 1385, 1391, *78*
__hook_rule_gclear:nnn 1348, 1373, 1415, 1416, *72*
__hook_rule_incompatible-error_-gset:nnn 1405
__hook_rule_incompatible-warning_-gset:nnn 1405
__hook_rule_unrelated_gset:nnn 1415, 1415, *72*
__hook_rule_voids_gset:nnn 1399, 1399
__hook_seq_csname:n 1568, 1569, 1587, 1621, 1669, 1710, 1770, 1777, 1835
__hook_set_default_hook_label:n 467, 467, 2821
__hook_set_default_label:n 467, 475, 482
__hook_set_normalize_fn:nn 584, 1158, 1162, 1167, *65*
__hook_str_compare:nn 1420, 1428, 1437, 23, 23
__hook_strip_double_slash:n 921, 927, 928
__hook_strip_double_slash:w 921, 929, 930, 934
__hook_tl_csname:n 1568, 1568, 1574, 1586, 1602, 1605, 1607, 1611, 1623, 1625, 1628, 1631, 1636, 1668, 1685, 1689, 1691, 1696, 1712, 1714, 1717, 1719, 1725, 1768, 1769, 1775, 1776, 1834
__hook_tl_gclear:N 1026, 1027, 1031, 1612, 1697, 2455, 2456, 2457, 58, 58, 60, 238, 253
__hook_tl_gput:Nn 1516, 1518, 1549, 1553, 1618, 1645, 1705, 1734, 1740, 1740, *79*
__hook_tl_gput_left:Nn 1516, 1550, 52, 52
__hook_tl_gput_right:Nn 636, 1518, 1554, 1647, 1736, 1738, 2229, 49, 49, 51
__hook_tl_gset:Nn 1387, 1394, 1401, 1406, 1411, 1541, 2228, 2442, 2980, 43, 43, 45, 47, 48, 50, 54
__hook_tl_gset_eq:NN 57, 57, 59
__hook_tl_set:Nn 1173, 1586, 1668, 41, 41, *33*
__hook_tmp:w 414, 418, 420, 1169, 1180, 1196, 1202, 1205, 1905, 1908, 2110, 2132, 2141, 2152, 2978, 2995, 2996, 34, 34
\l__hook_tmpa_bool 1934, 1937, 1945, 1954, 2034, 2037, 2045, 2054, 24, *86*
\l__hook_tmpa_tl . . 424, 1181, 1203, 25
\l__hook_tmpb_tl . 1170, 1177, 1205, 25
__hook_toplevel␣⟨*hook*⟩ *60*
__hook_toplevel_gset:nn 991, 996, 1111, 1115, 1130, 2447, 147
__hook_try_declaring_generic_-hook:nnn 531, 619, 678, 679, 681, 697, 699, 718, 721, *53*
__hook_try_declaring_generic_-hook:nNNnn . . 723, 729, 732, 732, *53*
__hook_try_declaring_generic_-hook:wn 752, 755, 810, 813, 840, 843, 872, 875
__hook_try_declaring_generic_-hook:wnTF 683, 690, 701, 710, 745, 751, 804, *55*
__hook_try_declaring_generic_-hook_split:nNNnn 732, 737, 740, 743
__hook_try_declaring_generic_-next_hook:nn 678, 688, 708, 727, 2199, *53*
__hook_try_file_hook:n 2353, 2361, 2364, *96*
__hook_try_put_cmd_hook:n 764, 821, 854
__hook_update_hook_code:n 526, 614, 1004, 1040, 1352, 1377, 1441, 1441, 1446, 1453, 1472, 1476, 2210, 2226, 76, 297, 315, *73*
__hook_use:wn 2295, 2309, 2353, 2353, 2356, 2358, *96*
__hook_use_end: 2303, 2306, 2313
__hook_use_i_delimit_by_s_-mark:nw 1244, 39, 40
__hook_use_initialized:n 1487, 2244, 2249, 2251, 2276, 2297, 2343, 2420, 2434, *93*
__hook_use_initialized:nnw 2316, 2321, 2323, 2344
__hook_use_none_delimit_by_s_-mark:w 1339, 1345, 2463, 2530, 39, 39
__hook_use_once:n 2403, 2430

__hook_use_once:nn 2388, 2394, 2409, 2409, 2411, 2428, 2437
__hook_use_once_clear:n .. 2417, 2435, 2439, 2439, 2443, 2451, 2453, *98*
__hook_use_once_set:n 2414, 2433, 2439, 2441, *98*
__hook_use_undefined:w .. 2301, 2305
\g__hook_used_prop 1447, 1459, 1474, 1480, 1524, 1562, 31
\l__hook_work_prop 587, 1163, 1165, 1199, 1520, 1557, 1582, 1589, 1591, 1600, 1617, 1642, 1664, 1671, 1673, 1683, 1703, 1731, 1802, 1813, 30, *78*
hook␣?? internal commands:
__hook␣?? *69*
hook␣⟨*hook*⟩ internal commands:
__hook␣⟨*hook*⟩ *42*
Hooks:
A/foo *13*, *13*
after *27*
babel/⟨*language*⟩/afterextras *22*
begindocument *25*, *25*, *25*, *28*, *36*
begindocument/before *28*
begindocument/end *28*
bfseries *30*, *30*
bfseries/defaults *30*, *30*
class *22*
cmd *55*, *60*, *16*, *22*, *22*, *101*
cmd/⟨*name*⟩/after *27*
cmd/⟨*name*⟩/before *27*
enddocument *24*, *25*, *29*
enddocument/afteraux *29*
enddocument/afterlastpage *29*
enddocument/end *29*
enddocument/info *29*
env *13*, *22*
env/⟨*env*⟩/after *26*, *26*, *27*
env/⟨*env*⟩/before *26*, *26*, *27*
env/⟨*env*⟩/begin *26*, *27*
env/⟨*env*⟩/end *26*, *27*
env/document/after *30*
env/document/before *28*
env/document/begin *28*
env/document/end *30*
env/quote/after *20*, *20*
expand@font@defaults *30*
file *13*, *22*
file/before *101*
foo *99*
include *22*
insertmark *31*
mdseries *30*
mdseries/defaults *30*
myhook *6*
normalfont *30*
package *22*
rmfamily *30*, *30*, *30*
selectfont *31*
sffamily *30*
shipout/lastpage *29*
test *23*, *23*
top-level *25*, *6*, *6*
ttfamily *30*
hook ?? internal commands:
__hook~?? 1314

I

if commands:
\if:w 1058, 1218, 1243, 2464, 2478, 2531, 2563, *98*
\if_case:w 1269, 1420, 1437
\if_charcode:w 2581
\if_cs_exist:w .. 789, 1751, 2131, 2253, 2261, 2278, 2299, 2326, 2465
\if_int_compare:w 1428
\IfHookEmptyF 2855, *14*
\IfHookEmptyT 2855, *14*
\IfHookEmptyTF 2855, 2960, *8*
\IfHookExistsTF 2858, 2959, *109*
\ignorespaces *28*
\immediate *29*
include (hook) *22*
\include *26*
\IncludeInRelease 491, 601, 652, 669, 678, 696, 717, 751, 809, 839, 871, 898, 901, 921, 924, 937, 955, 960, 963, 972, 977, 980, 1017, 1054, 1104, 1111, 1127, 1134, 1154, 1158, 1210, 1214, 1230, 1234, 1262, 1265, 1288, 1292, 1309, 1317, 1321, 1330, 1361, 1442, 1468, 1495, 1529, 1575, 1657, 1864, 1982, 2085, 2090, 2154, 2185, 2203, 2216, 2234, 2239, 2244, 2266, 2286, 2316, 2334, 2339, 2349, 2353, 2356, 2383, 2398, 2409, 2428, 2439, 2451, 2474, 2491, 2557, 2573, 2630, 2659, 2767, 2776, 2782, 2789, 2794, 2801, 2805, 2812, 2832, 2839, 2933, 61, 80, 96, 124, 140, 152, 164, 180, 193, 213, 223, 243, 258, 276, 282, 302, 320
\input *107*
\InsertMark *31*
insertmark (hook) *31*
int commands:
\int_compare:nNnTF 1605, 1627, 1638, 1688, 1716, 1727, 2097, 2103, 2422, 2619

\int_compare_p:nNn 2966, 2967
\int_decr:N 1616, 1702
\int_eval:n 1270, 1624, 1713, 1769, 1776, 1884
\int_incr:N 1584, 1666
\int_new:N 1571
\int_set:Nn 1186
\int_zero:N 1580, 1662
iow commands:
\iow_char:N 1799, 1810, 2642, 2644, 2646, 2651, 2654, 2656, 2666, 2667, 2707, 2715, 2727, 2732, 2737, 2923, 2925
\iow_log:n 1842
\iow_show:n 1845, 1849
\iow_term:n 536, 626, 1458, 1461, 1479, 1482, 1500, 1534, 1620, 1640, 1641, 1643, 1707, 1729, 1730, 1732, 1798, 1809, 1823, 1829, 1830, 1831, 1834, 1838, 1856, 2140, 2145, 2160, 2174

K

kernel internal commands:
__kernel_cs_parm_from_arg_count:nnTF 103
__kernel_exp_not:w ... 42, 44, 50, 55

L

\let 2953, 2954, *109*
\listfiles *29*
\LogHook 2844, *14*
\long 2942, 2959, 2960

M

mdseries (hook) *30*
\mdseries *30*
mdseries/defaults (hook) *30*
msg commands:
\msg_error:nn 436, 453
\msg_error:nnn 461, 530, 548, 618, 638, 2195, 70, 88, 173, 202, 205
\msg_error:nnnn 512, 662, 106
\msg_error:nnnnn 440, 471, 486
\msg_error:nnnnnn 1343, 1355, 1380, 1782
\msg_expandable_error:nn ... 335, 336
\msg_expandable_error:nnn 1283, 2590, 364
\msg_line_context: 585, 2672, 2677, 2681, 2728
\g_msg_module_name_prop 2614
\g_msg_module_type_prop .. 2612, 2613
\msg_new:nnn 2670, 2675, 2679, 2724, 2730, 2735, 2740, 2745, 2749, 2756, 2921
\msg_new:nnnn ... 2615, 2625, 2632, 2639, 2648, 2662, 2684, 2700, 2713
\msg_warning:nnn 289, 307
\msg_warning:nnnn 1000, 1013, 1035, 1049, 2920
\msg_warning:nnnnn 790
\msg_warning:nnnnnn 1790
myhook (hook) *6*

N

\NewDocumentCommand 2761, 2763, 2765, 2769, 2771, 2773, 2784, 2786, 2796, 2798, 2807, 2809, 2816, 2818, 2820, 2822, 2824, 2848, 2850, 2853
\NewDocumentEnvironment *26*
\newenvironment *26*
\NewHook 2761, 2936, *20*
\NewHookWithArguments 2767, *22*
\NewMirroredHookPair 2761, 2938, *3*
\NewMirroredHookPairWithArguments 2767, *3*
\NewModuleRelease 4
\NewReversedHook 2761, 2937, *3*
\NewReversedHookWithArguments .. 2767, *3*
normalfont (hook) *30*
\normalfont *30*
\normalsize *7*

O

or commands:
\or: .. 1273, 1274, 1275, 1276, 1277, 1278, 1279, 1280, 1281, 1422, 1438

P

package (hook) *22*
\PopDefaultHookLabel 2820, *10*
prg commands:
\prg_do_nothing: 2346
\prg_new_conditional:Npnn 903, 1216, 1426, 2461, 2476, 2493, 2510, 2516, 2522, 2528, 2537, 2545, 2559, 2577, 267
\prg_new_protected_conditional:Npnn 754, 812, 842, 874
\prg_replicate:nn 1190, 2329, 2337, 2407, 2423
\prg_return_false: 781, 835, 847, 867, 879, 894, 911, 915, 918, 1221, 1431, 2471, 2487, 2505, 2514, 2520, 2526, 2534, 2543, 2552, 2555, 2569, 2584, 273, *55*

\prg_return_true: 772, 826, 865, 892, 914, 1219, 1429, 2469, 2485, 2504, 2507, 2513, 2519, 2525, 2532, 2542, 2553, 2567, 2582, 272
prop commands:
\prop_clear:N 1163
\prop_const_from_keyval:Nn 957, 965, 967, 969
\prop_gclear:N . 990, 1025, 1447, 1474
\prop_gclear_new:N 2448, 2458
\prop_get:NnN 587, 1617, 1703
\prop_get:NnNTF 596, 641
\prop_gpop:NnNTF 998, 1033
\prop_gput:Nnn 597, 598, 644, 647, 1524, 1562, 2612, 2613, 2614
\prop_gset_eq:NN 1165
\prop_if_empty:NTF 1504, 1539, 2004, 2481
\prop_if_empty_p:N 1900, 2498
\prop_if_exist:NTF . . 2108, 2480, 2518
\prop_if_exist_p:N 1899
\prop_if_in:NnTF 849, 857, 861, 881, 884, 888, 913
\prop_map_break: 1594, 1676, 2116
\prop_map_function:NN 1164, *65*
\prop_map_inline:Nn 1459, 1480, 1582, 1589, 1591, 1664, 1671, 1673, 1832, 1903, 2007, 2111, 2113
\prop_new:N 1314, 1315, 30, 31, 146, 158
\prop_put:Nnn 1199, 1802, 1813
\prop_set_eq:NN 1520, 1557
\prop_show:N *90*
\providecommand 2945, 2948
\ProvideHook 2891
\ProvideMirroredHookPair 2891
\ProvideReversedHook 2891
\PushDefaultHookLabel 2820, *10*

Q

quark commands:
\quark_if_recursion_tail_stop:n 416
\q_recursion_stop 422
\q_recursion_tail 421, 422
\q_stop 569, 574

R

\RemoveFromHook 2818, 2947, *6*
\requestedLaTeXdate 2967
\RequirePackage *10*
rmfamily (hook) *30*, *30*, *30*
\rmfamily *30*

S

scan commands:
\scan_new:N 38
\scan_stop: 683, 690, 701, 710, 745, 756, 792, 804, 814, 844, 876, 1218, 2578, 2579
scan internal commands:
\s__hook_mark 734, 785, 787, 795, 797, 800, 802, 904, 929, 930, 934, 1239, 1256, 1260, 1302, 1303, 1358, 2295, 2309, 2358, 2366, 2469, 2471, 2532, 2534, 2538, 2539, 2546, 2547, 2560, 2561, 38, 39, 40, 342, 345, 348, 352, 355, 356, *33*
selectfont (hook) *31*
\selectfont *30*
seq commands:
\seq_clear:N 1579, 1661
\seq_clear_new:N 1587, 1669
\seq_gpop:NN 2609
\seq_gpop:NNTF 451, 459
\seq_gpop_right:NN 424
\seq_gpush:Nn 444, 2597, 2603
\seq_gput_right:Nn . 413, 417, 102, 131
\seq_if_empty:NTF 412, 469
\seq_map_inline:Nn 1448, 1475, 1603, 1621, 1686, 1709, 2990
\seq_new:N 1570, 2587, 28, 33
\seq_put_right:Nn 1585, 1667, 1770, 1777
\seq_use:Nnnn 1830, 1835
\SetDefaultHookLabel 2820, *8*
sffamily (hook) *30*
\sffamily *30*
\shipout *30*
shipout/lastpage (hook) *29*
show commands:
\show_hook:n *84*
\ShowHook 2844, 2952, *14*
\small *7*
\sourceLaTeXdate 2966
\space 629
\special *29*
str commands:
\c_hash_str 568, 574
\c_right_brace_str 2088
\str_count:n 1271, 1885, 37, 37
\str_gset:Nn 2829
\str_if_eq:nn *73*
\str_if_eq:nnTF . . . 438, 484, 541, 543, 630, 632, 762, 820, 853, 906, 988, 995, 1023, 1030, 1823, 1941, 2041, 2148, 2360, 2564, 2618, 2688, 340
\str_if_eq_p:nn 910, 1614, 1700
str internal commands:
__str_if_eq:nn 23, *32*
\strut *31*

T

test (hook) *23*, *23*
TEX and LATEX 2ε commands:
\@...hook *41*
\@begindocumenthook *25*
\@cls@pkg 2721
\@currname 423, 362, 368, *108*
\@currnamestack 420, *108*
\@empty 2953, 2954
\@expl@@@hook@curr@name@pop@@ . 2928
\@expl@@@initialize@all@@ 1491, 2928
\@expl@push@filename@aux@@ 2826, *108*
\@firstofone *5*
\@gobble@AddToHook@args . . 2944, 2945
\@gobble@RemoveFromHook@arg 2947, 2948
\@kernel@after@⟨*hook*⟩ *24*
\@kernel@after@enddocument@afterlastpage 455
\@kernel@before@⟨*hook*⟩ *24*
\@latex@error 2971
\@onefilewithoptions *107*
\@onlypreamble 2852
\@pushfilename *108*
\@spaces 539, 1863, 1963, 1973, 2064, 2075, 2164, 2178
\@@end *29*
\expand@font@defaults *30*
\g@addto@macro *38*
\on@line 538, 629, 1500, 1535, 2163, 2177
\protected *93*
tex commands:
\tex_escapechar:D 1186, *66*
tl commands:
\c_empty_tl 2242, 59
\c_novalue_tl *44*
\tl_const:Nn 941, 942, 944, 945, 946, 947, 950, 951, 953, 974, 975, 1319, 35, 36, 104, 107
\tl_gclear:N *92*
\tl_gclear_new:N 264
\tl_gput_right:Nn 455, *75*
\tl_gset:Nn 411, 423, 445, 489, 771, 825, 859, 863, 886, 890, 176, 188, 295, 313
\tl_gset_eq:NN 452, 463, 57
\tl_if_blank:nTF 429, 435, 477
\tl_if_empty:N *99*
\tl_if_empty:NTF 659, 1060, 1450, 2020, 2028, 2225, 2227, 229, 249, 360, 362
\tl_if_empty:nTF 576, 846, 878, 932, 2551, 332, 347, 350
\tl_if_empty_p:N 2501, 2502
\tl_if_empty_p:n 909
\tl_if_exist:N *93*
\tl_if_exist:NTF 1297, 2270, 2290, 2376, 2524, 2565, 129, 227, 247
\tl_if_exist_p:N 270
\tl_if_novalue:nTF 326
\tl_log:n 1843
\tl_new:N 1316, 1572, 1573, 1574, 25, 26, 27, 29, 32, 72, 90, 118, 132, 135, 159, 160, 293, 311
\tl_set:Nn . . 579, 1170, 1177, 1181, 1581, 1601, 1602, 1607, 1608, 1623, 1630, 1632, 1663, 1684, 1685, 1691, 1693, 1712, 1719, 1721, 1768, 1775
\tl_set_eq:NN . 1611, 1635, 1696, 1724
\tl_show:n 1850, 1857, 1961, 2062
\tl_to_str:n 540, 568, 629, 2009, 2022, 2093, 2165, 2179, 378
\tl_trim_spaces:n 407
\tl_trim_spaces_apply:nN 328
\tl_use:N 1078, 1141, 1834
token commands:
\token_if_macro:NTF 1226, 1252
\token_to_str:N 375
top-level (hook) *25*, *6*, *6*
ttfamily (hook) *30*
\ttfamily *30*

U

use commands:
\use:N 1651, 1784, 1785, 1825, 2337, 2407, 2423
\use:n 516, 571, 1174, 1649, 2102, 2136, 2389, 2395, 2404, 232, 382
\use:nn 1755, 2152, 2727
\use_i:nn 2254, 2599
\use_i:nnn 2101, *73*
\use_ii:nn 659, 2605, *73*
\use_iii:nn *73*
\use_none:n 660, 1370, 1441, 1488, 1753, 2256, 2345, 7, *93*
\use_none:nn 2093, 2134
\use_none:nnn 766, 2099
\use_none:nnnn 2976
\use_none:nnnnn 1872, 1990
\use_none:nnnnnnnnn . 1370, 1872, 1990
\UseHook 2832, 2950, *4*
\UseHookWithArguments 2832, *22*
\UseOneTimeHook 2832, 2951, *5*
\UseOneTimeHookWithArguments . . . 2832, *5*
\usepackage *27*
\usetikzlibrary *10*

W

\write *29*